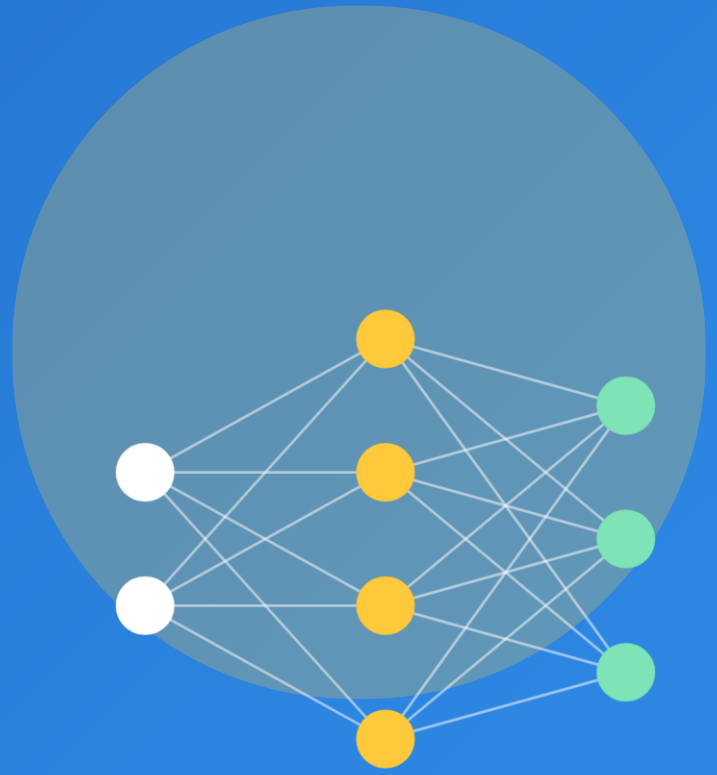
CHƯƠNG: MẠNG NƠ-RON NHÂN TẠO · MỤC 1

Từ mô hình tuyến tính đến mạng nơ-ron

Vì sao Softmax Regression chưa đủ, và nơ-ron nhân tạo giải quyết điều đó như thế nào



LỘ TRÌNH

Hôm nay ta trả lời ba câu hỏi

1.1

Mô hình tuyến tính không làm được gì?

XOR và các lớp bị bao quanh: giới hạn của vùng quyết định lồi.

1.2

Làm sao để mô hình tự học đặc trưng?

Từ đặc trưng đa thức tự chọn đến đặc trưng có tham số, học được.

1.3

Nơ-ron nhân tạo là gì?

Nơ-ron, mạng nhiều lớp, lịch sử và kết quả sơ bộ.

Nhắc lại: vùng quyết định luôn lồi

Mô hình làm gì?

Điểm số mỗi lớp là một biểu thức tuyến tính: $s = w_0 + w_1x_1 + w_2x_2 + \dots$

Dự đoán: lớp có điểm số lớn nhất

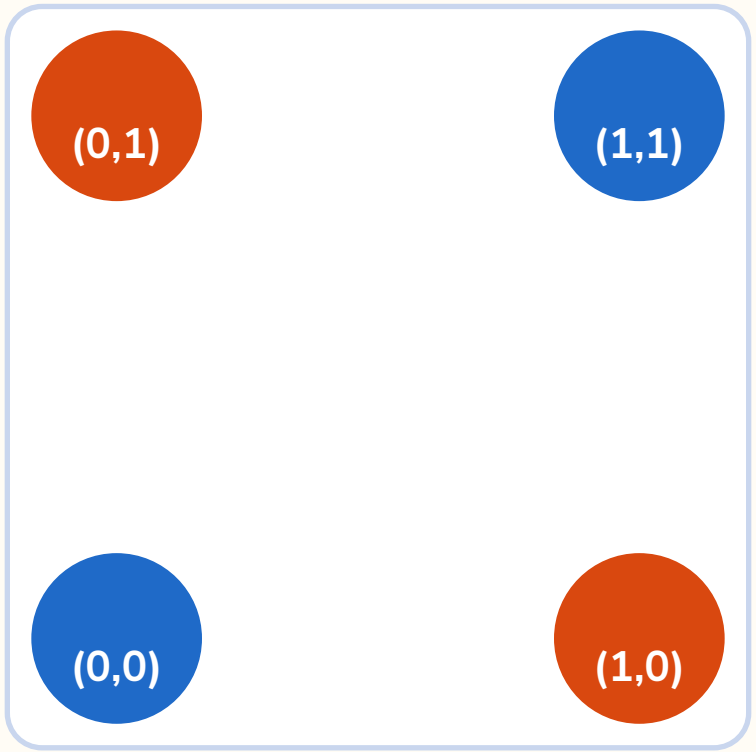
Biên giữa lớp k và l: các điểm có $s_k = s_l$, tức là một **siêu phẳng** (đường thẳng khi d = 2)

Vùng của mỗi lớp = giao của các nửa không gian

⇒ luôn lồi

Rất tốt cho các cụm tách biệt (rớt, đạt, giỏi). Nhưng dữ liệu thật không phải lúc nào cũng thân thiện như vậy.

Bài toán XOR



x_1	x_2	Nhãn y
0	0	0 (xanh)
1	1	0 (xanh)
0	1	1 (cam)
1	0	1 (cam)

Hai điểm cùng nhãn nằm ở hai đỉnh **chéo nhau**.

Không có đường thẳng nào tách được cả hai lớp.

Chứng minh bằng bốn bất đẳng thức

Cần điểm số $s = w_0 + w_1x_1 + w_2x_2$ **âm** với nhãn 0 và **dương** với nhãn 1:

(0,0)	$w_0 < 0$
(1,1)	$w_0 + w_1 + w_2 < 0$
(0,1)	$w_0 + w_2 > 0$
(1,0)	$w_0 + w_1 > 0$

Cộng hai bất đẳng thức **đầu**:

$$2w_0 + w_1 + w_2 < 0$$

Cộng hai bất đẳng thức **sau**:

$$2w_0 + w_1 + w_2 > 0$$

⇒ Mâu thuẫn!

Không tồn tại w nào: kết luận đúng cho cả **Softmax**, **Logistic Regression** và **Perceptron**.

1.1 · HẠN CHẾ THỨ HAI

Lớp bị lớp khác bao quanh



Ba vòng tròn đồng tâm (bán kính 1, 3, 5), mỗi lớp 200 điểm. Lớp 0 ở giữa, lớp 1 và lớp 2 là hai chiếc nhẫn bao quanh.

35,0%

độ chính xác kiểm tra của Softmax

33,3%

mức đoán mò của 3 lớp

Hàm mất mát J = 1,0964, gần như bằng mốc $J_0 = \ln 3 = 1{,}0986$: mô hình **gần như không học được gì**.

1.1 · CHẨN ĐOÁN

Huấn luyện kỹ hơn có cứu được không?

Nguyên nhân	Biểu hiện	Sửa bằng huấn luyện kỹ hơn?
Thuật toán tối ưu chưa tốt	J giảm chậm hoặc dao động	Có: chỉnh η, chuẩn hóa, thêm epoch
Nhiễu hoặc ít dữ liệu	Độ chính xác huấn luyện cao hơn nhiều so với kiểm tra	Một phần: thêm dữ liệu, điều chuẩn hóa
Mô hình không đủ khả năng biểu diễn	J đã hội tụ mà vẫn cao; sai cả trên tập huấn luyện	Không: phải đổi mô hình

XOR và vòng tròn thuộc nhóm thứ ba: hàm mất mát lồi đã về **cực tiểu toàn cục** mà vẫn cao. Trần chất lượng nằm ở lớp hàm mô hình biểu diễn được, ta cần mô hình **phi tuyến**.

Cách chữa quen thuộc: thêm đặc trưng đa thức

XOR: thêm x_1x_2

$$s = x_1 + x_2 - 2x_1x_2 - 0{,}5$$

(x_1, x_2)	s	Dự đoán
(0,0)	−0,5	0
(1,1)	−0,5	0
(0,1)	+0,5	1
(1,0)	+0,5	1

Vòng tròn: thêm x_1^2, x_2^2, x_1x_2

Biên trở thành đường conic (có thể là đường tròn).

99,4%

độ chính xác kiểm tra, tăng từ 35,0%

Mô hình vẫn **tuyến tính theo tham số**; ta chỉ đổi đặc trưng đầu vào thành $\varphi(x)$.

1.2 · CÁI GIÁ

Số đặc trưng đa thức bùng nổ

Với d đặc trưng gốc và bậc tối đa k: số đặc trưng mới là **C(d + k, k) − 1**; số tham số là **K · C(d + k, k)**.

Bài toán	d	k	K	Đặc trưng mới	Số tham số
Vòng tròn đồng tâm	2	2	3	5	18
Dữ liệu bảng cỡ vừa	10	2	10	65	660
Ảnh chữ số 28×28	784	2	10	308.504	3.085.050
Ảnh chữ số 28×28	784	3	10	80.931.144	809.311.450

1. Bùng nổ

Số tham số vượt xa số mẫu, dễ quá khớp.

2. Phải biết trước dạng

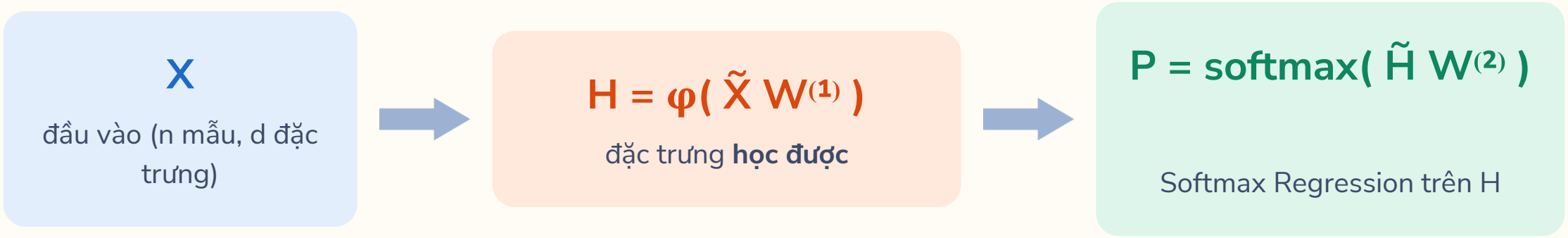
Với ảnh, tiếng nói, văn bản: không ai biết nên dùng tổ hợp nào.

3. Tách rời

Đặc trưng chọn trước, bộ phân loại không sửa được.

Để dữ liệu quyết định đặc trưng

Nếu đặc trưng quan trọng như vậy, sao không **tham số hóa đặc trưng rồi học chúng** cùng với bộ phân loại?



$W^{(1)}$ cỡ (d+1)×m. Mỗi **cột** định nghĩa một đặc trưng $h_j = \varphi(\tilde{x}\, w_j^{(1)})$.

φ là hàm phi tuyến áp dụng lên từng phần tử. Cả hai lớp được huấn luyện chung bằng cross-entropy.

1.2 · MỘT CHI TIẾT BẮT BUỘC

Vì sao φ phải phi tuyến?

Nếu φ là hàm đồng nhất

H = $\tilde{X}$ $W^{(1)}$. Tách $W^{(2)}$ thành hàng chệch b và phần còn lại V:

$$S = 1b + \tilde{X}\, W^{(1)}\, V$$

Đây vẫn chỉ là hàm **tuyến tính** của x, với ma trận tham số $W^{(1)}V$.

Mạng sâu bao nhiêu cũng chỉ bằng Softmax Regression.

Chính φ phi tuyến làm cho việc xếp chồng có ý nghĩa.

Ba lựa chọn sẽ gặp ở mục 2.3: sigmoid · tanh · ReLU

1.2 · VÍ DỤ

Hai “bộ dò” làm XOR tách được

$h_1 = \varphi(x_1 + x_2 - 0{,}5)$

Bật khi **ít nhất một** đầu vào bằng 1 (phép OR)

$h_2 = \varphi(x_1 + x_2 - 1{,}5)$

Bật khi **cả hai** đầu vào bằng 1 (phép AND)

φ là hàm bước (bằng 1 khi đối số không âm, ngược lại bằng 0). Mỗi bộ dò là một Perceptron nhỏ.

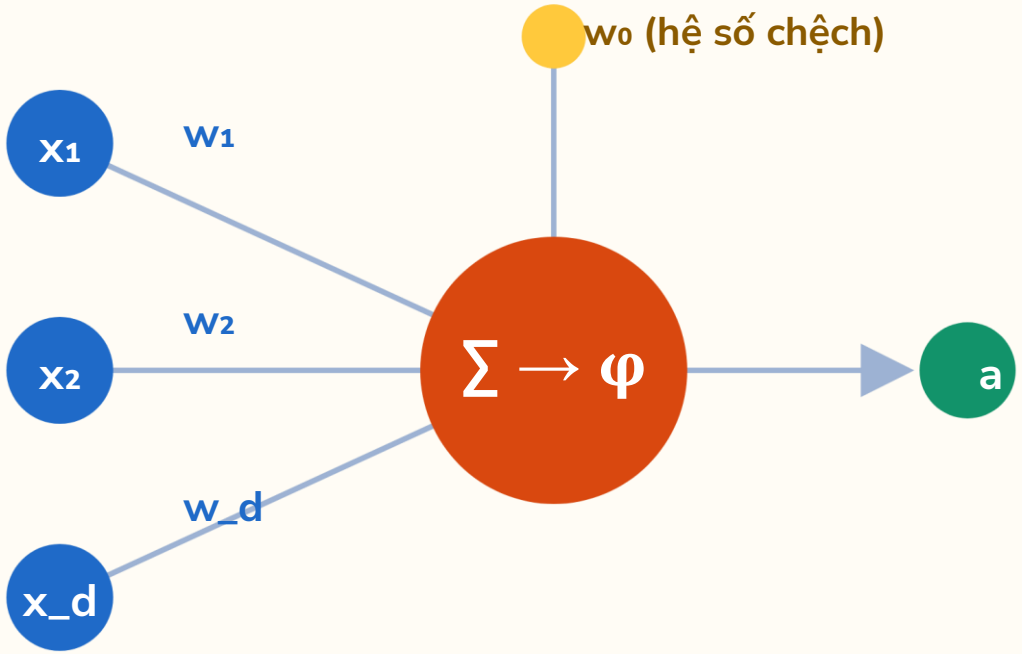
(x_1, x_2)	h_1	h_2	Nhãn
(0,0)	0	0	0
(1,1)	1	1	0
(0,1)	1	0	1
(1,0)	1	0	1

Hai điểm lớp 1 **trùng nhau** tại (1,0); hai điểm lớp 0 nằm hai phía.

$s = -0{,}5 + h_1 - h_2$

cho −0,5; −0,5; +0,5; +0,5: đúng cả bốn. Không tách được ở (x_1, x_2) nhưng **tách được ở (h_1, h_2)**. Dữ liệu nhiễu: 96,0% so với 50,5%.

Một nơ-ron: tổng có trọng số rồi kích hoạt



$$s = w_0 + \sum w_j x_j = \tilde{x}\, w$$

$$a = \varphi(s)$$

Toán học	Gợi hứng sinh học
x_j	tín hiệu vào
w_j	độ mạnh của kết nối
$-w_0$	ngưỡng kích hoạt
φ	cách nơ-ron “phát xung”

Chỉ là gợi hứng: mạng nơ-ron nhân tạo là một họ hàm huấn luyện bằng gradient, không mô phỏng chính xác bộ não.

1.3 · KẾT NỐI VỚI CÁC BÀI TRƯỚC

Các mô hình đã học đều là nơ-ron

Mô hình	Hàm kích hoạt φ	Nơ-ron ra	Hàm mất mát
Hồi quy tuyến tính	Đồng nhất: φ(s) = s	1	Bình phương sai số
Perceptron	Hàm dấu: sign(s)	1	Mất mát Perceptron
Logistic Regression	Sigmoid: σ(s)	1	Cross-entropy nhị phân
Softmax Regression	Softmax (chung cho K nơ-ron)	K	Cross-entropy nhiều lớp

Perceptron làm mềm hàm dấu **Logistic** 1 điểm số thành K **Softmax** xếp chồng + phi tuyến **Mạng nơ-ron**

Mỗi mô hình cũ là **một lớp nơ-ron duy nhất**. Mạng nơ-ron chỉ làm thêm một việc: xếp chồng các lớp.

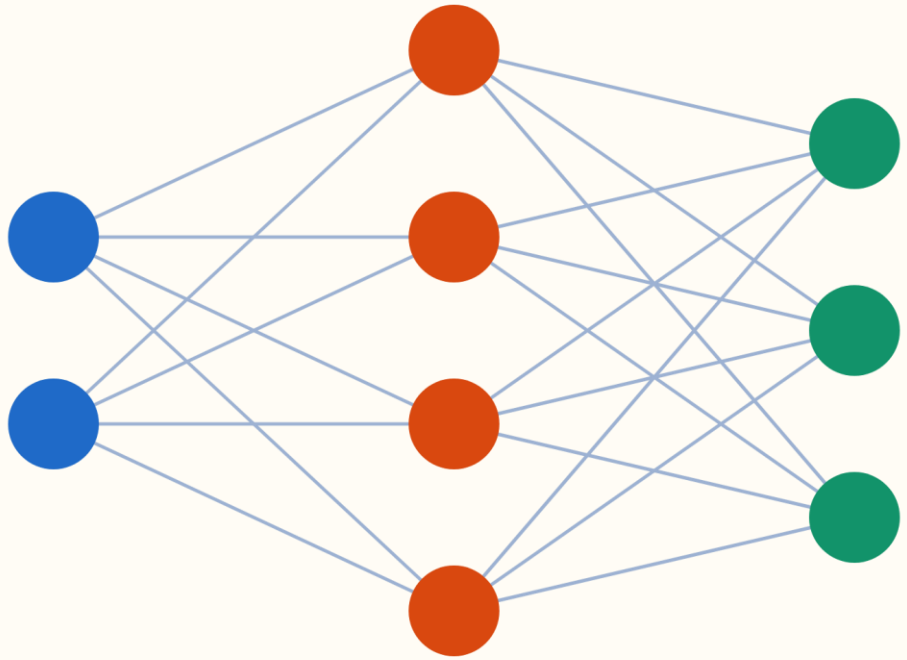
1.3 · MỘT CHÚT LỊCH SỬ

Nhiều lần lên xuống

Năm	Sự kiện
1943	McCulloch và Pitts: nơ-ron ngưỡng thực hiện được phép logic
1958	Rosenblatt công bố Perceptron, học tự động từ dữ liệu
1969	Minsky và Papert chỉ ra giới hạn (như XOR); quan tâm giảm mạnh
1974–1986	Lan truyền ngược được phổ biến: huấn luyện được các lớp ẩn
1989	Định lý xấp xỉ phổ quát; mạng tích chập nhận dạng chữ số
2012	AlexNet thắng ImageNet: mở đầu làn sóng học sâu
2017	Kiến trúc Transformer, nền tảng của các mô hình ngôn ngữ lớn

1958 là điểm xuất phát, **1969** là bài học (XOR), lan truyền ngược là chìa khóa để **huấn luyện** mạng nhiều lớp.

Xếp chồng nơ-ron: mạng 2–4–3



Lớp vào: các đặc trưng $x_1,\ldots,x_d$; không có tham số.

Lớp ẩn: m nơ-ron. "Ẩn" vì ta **không có nhãn trực tiếp** cho chúng, nên cần lan truyền ngược (mục 3).

Lớp ra: K nơ-ron softmax, chính là Softmax Regression.

Số tham số: (d+1)m + (m+1)K

Mạng 2–4–3: 3×4 + 5×3 = **27**. Softmax 2–3 chỉ có 9.

1.3 · KẾT QUẢ SƠ BỘ

Ba vòng tròn: mạng tự học biên cong



Mỗi cỡ lớp ẩn chạy với 5 khởi tạo ngẫu nhiên; thanh đậm = giá trị thấp nhất, phần nhạt = đến giá trị cao nhất. Trục từ 0% đến 100%.

Tự học biên cong chỉ từ x_1, x_2, ngang mô hình đa thức ta phải tự chọn.

Cỡ lớp ẩn quan trọng: m = 2 quá nhỏ cho ba lớp lồng nhau.

Phụ thuộc khởi tạo: hàm mất mát không còn lồi.

TÓM TẮT MỤC 1

Năm điều cần nhớ

1. Softmax, Logistic, Perceptron có **vùng quyết định lồi**: không giải được XOR (50,5%) và vòng tròn (35,0%).

2. Giới hạn nằm ở **khả năng biểu diễn**, không phải ở thuật toán hay lượng dữ liệu.

3. Ý tưởng chính: **học đặc trưng** $H = \varphi(\tilde{X}W^{(1)})$, $P = \text{softmax}(\tilde{H}W^{(2)})$; φ bắt buộc phi tuyến.

4. Nơ-ron $a = \varphi(\tilde{x}w)$. Mô hình cũ là mạng **không có lớp ẩn**; mạng một lớp ẩn có $(d+1)m + (m+1)K$ tham số.

5. Mạng 2–8–3 đạt **99,4%** chỉ từ x_1, x_2; đổi lại hàm mất mát không lồi nên kết quả phụ thuộc khởi tạo.

BẢN ĐỒ CỦA CHƯƠNG

Mỗi mục trả lời một câu hỏi từ mục này

Mục	Câu hỏi chính
2	Nơ-ron, lớp và mạng viết bằng ma trận ra sao? Hàm kích hoạt nào, vì sao phi tuyến? Mạng biểu diễn được gì?
3	Làm sao huấn luyện khi nơ-ron ẩn không có nhãn? Gradient tính thế nào (lan truyền ngược)?
4	Mạng học ra sao khi một mẫu sai? Vì sao mất mát không lồi, và gradient của mạng sâu?
5	Đánh giá, chọn siêu tham số và chẩn đoán lỗi như thế nào?
6	Ưu và nhược điểm, con đường đến học sâu (CNN, RNN, Transformer)

Cái “mới” duy nhất so với bài trước là **các lớp ẩn**; mọi thứ khác (cross-entropy, gradient P – Y ở lớp ra) vẫn dùng được.

THẢO LUẬN

Bốn câu hỏi nhanh

Câu 1 · OR

Dữ liệu OR có nhãn 0, 1, 1, 1 tại (0,0), (0,1), (1,0), (1,1). Tìm w_0, w_1, w_2 phân loại đúng. Vì sao làm được với OR mà không với XOR?

Câu 2 · Thêm epoch?

Softmax đã hội tụ trên ba vòng tròn nhưng chỉ đạt 35%. Chạy thêm 10 lần số epoch có giúp không? Vì sao?

Câu 3 · Đếm tham số

Có d = 20 đặc trưng, K = 5 lớp. Thêm mọi đặc trưng đa thức bậc 2 thì có bao nhiêu đặc trưng mới và bao nhiêu tham số?

Câu 4 · Mạng nhỏ

Mạng một lớp ẩn có d = 4, m = 6, K = 3: kích thước $W^{(1)}$, $W^{(2)}$ và tổng số tham số là bao nhiêu?

CHƯƠNG: MẠNG NƠ-RON NHÂN TẠO · MỤC 2

Mô hình mạng nơ-ron truyền thẳng (MLP)

Từ một nơ-ron, một lớp, đến nhiều lớp; hàm kích hoạt, lan truyền thuận và khả năng biểu diễn



LỘ TRÌNH

Từ nhỏ đến lớn, năm câu hỏi

Một nơ-ron → Một lớp nơ-ron → Nhiều lớp (MLP)

2.1	Một nơ-ron làm gì? Perceptron, Logistic, Softmax là trường hợp riêng nào?
2.2	Gộp nhiều nơ-ron thành một lớp viết bằng ma trận thế nào? Kích thước khớp ra sao?
2.3	Hàm kích hoạt nào (sigmoid, tanh, ReLU)? Vì sao bắt buộc phi tuyến?
2.4	Lan truyền thuận qua nhiều lớp; chọn lớp ra và hàm mất mát cho từng bài toán?
2.5	Mạng biểu diễn được gì? Giải XOR, biên cong, định lý xấp xỉ phổ quát.

KÝ HIỆU

Ký hiệu cho mạng nhiều lớp

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Số lớp có tham số	L	vô hướng	Lớp ẩn + lớp ra (mạng 1 lớp ẩn: $L = 2$)
Số nơ-ron lớp l	m_l	vô hướng	$m_0 = d$ (vào), $m_L = K$ (ra)
Ma trận trọng số	$W^{(l)}$	$(m_{l-1}+1) \times m_l$	Cột j: tham số nơ-ron j; hàng đầu là chệch
Điểm số	$Z^{(l)}$	$n \times m_l$	$Z^{(l)} = \tilde{A}^{(l-1)}W^{(l)}$
Kích hoạt (đầu ra lớp l)	$A^{(l)}$	$n \times m_l$	$A^{(l)} = \varphi_l(Z^{(l)})$, $A^{(0)} = X$, $A^{(L)} = P$
Một mẫu	$z_i^{(l)}$, $a_i^{(l)}$	$1 \times m_l$	Hàng i của $Z^{(l)}$, $A^{(l)}$

Chỉ số trên trong ngoặc, như $^{(l)}$, là **lớp thứ l**, không phải lũy thừa. Dấu ngã ($\tilde{A}$) nghĩa là đã thêm cột số 1. **Ma trận: chữ hoa in đậm; vector: chữ thường in đậm.**

Một nơ-ron là một hàm hai bước

BƯỚC 1 · Tổ hợp tuyến tính

$$s = w_0 + w_1x_1 + \dots + w_dx_d = \tilde{x}\, w$$

BƯỚC 2 · Kích hoạt

$$a = \varphi(s)$$

Đầu vào là vector hàng $\mathbf{x} = [x_1 \dots x_d]$; $\tilde{x}$ là $\mathbf{x}$ sau khi thêm số 1 ở đầu.

Với **n mẫu**, tính cùng lúc bằng một phép nhân ma trận: $\mathbf{s} = \tilde{\mathbf{X}}\,\mathbf{w}$ và $\mathbf{a} = \varphi(\mathbf{s})$, φ áp dụng lên từng phần tử.

Ba mô hình cũ là ba cách chọn φ

Mô hình	φ(s)	Đầu ra a	Hàm mất mát
Hồi quy tuyến tính	s (đồng nhất)	số thực	Bình phương sai số
Perceptron	sign(s)	nhãn ±1	Mất mát Perceptron
Logistic Regression	$\sigma(s) = 1/(1 + e^{-s})$	xác suất $p \in (0,1)$	Cross-entropy nhị phân
Softmax Regression	softmax trên K điểm số	K xác suất	Cross-entropy nhiều lớp

✓ Mỗi mô hình đã học là **mạng nơ-ron không có lớp ẩn**: đầu vào nối thẳng tới lớp nơ-ron ra.

2.1 · HAI ĐIỂM CẦN LƯU Ý

Softmax: K nơ-ron chung bước chuẩn hóa

Một lớp K nơ-ron

Nơ-ron k có vector w_k và điểm số $s_k = \tilde{x}\, w_k$.

Khác sigmoid, softmax cần **cả K điểm số** để chuẩn hóa:

$$p_k = e^{s_k} / \sum_j e^{s_j}$$

Xếp w_k thành cột của **W**: **S** = $\tilde{\mathbf{X}}$**W**, **P** = softmax(**S**).

Với K = 2 trở về Logistic

$$p_1 = \sigma(\tilde{x}\,(w_1 - w_0))$$

Một nơ-ron sigmoid với $w = w_1 - w_0$ làm đúng việc của hai nơ-ron softmax.

Quy ước trong mạng: nhị phân dùng **1** nơ-ron ra sigmoid; K > 2 lớp dùng **K** nơ-ron ra softmax.

2.1 · HÌNH HỌC

Một nơ-ron cắt không gian bằng một siêu phẳng

Đường biên của nơ-ron ngưỡng:

$$w_0 + w_1x_1 + \dots + w_dx_d = 0$$

Hướng của biên

Vector pháp tuyến là $[w_1 \dots w_d]$.

Vị trí của biên

Hệ số chệch w_0 quyết định siêu phẳng **dịch** đi đâu.

Với sigmoid

Biên $p = 0{,}5$ vẫn là siêu phẳng đó, nhưng xác suất đổi **mượt** khi rời xa nó.

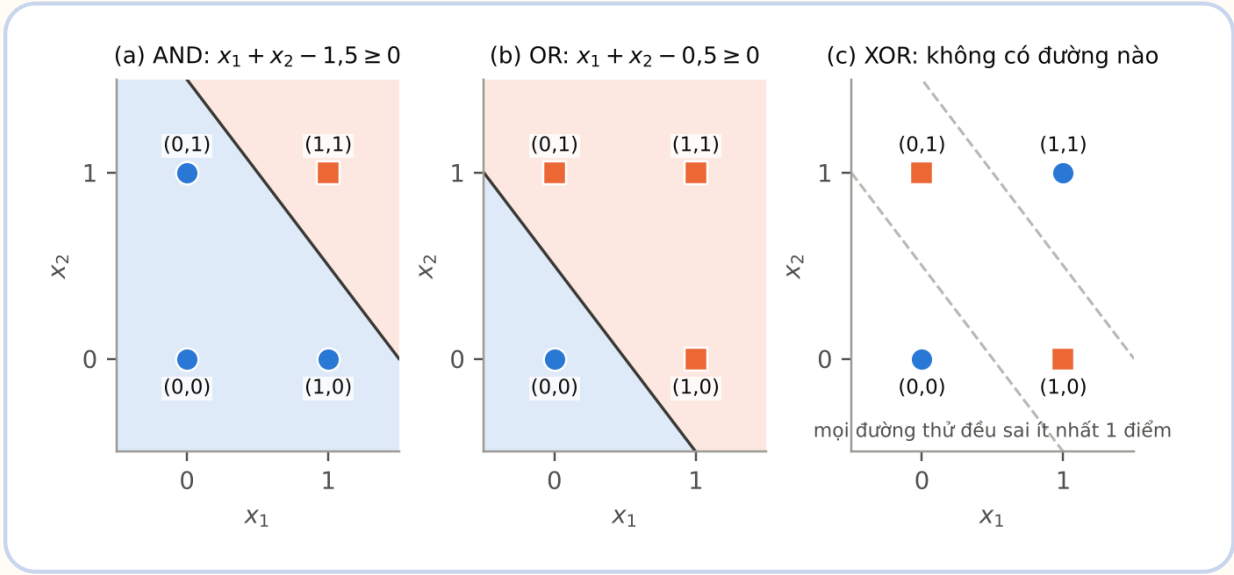
Mỗi nơ-ron chỉ vẽ được **một** siêu phẳng, vì vậy bộ ba mô hình cũ có **vùng quyết định lồi** (mục 1.1).

2.1 · VÍ DỤ

Nơ-ron làm cổng logic

Phép	w_0	w_1	w_2	Điều kiện $s \geq 0$
AND	−1,5	1	1	$x_1 + x_2 \geq 1{,}5$
OR	−0,5	1	1	$x_1 + x_2 \geq 0{,}5$
NOT x_1	0,5	−1	0	$x_1 = 0$

φ là hàm bước (1 nếu s ≥ 0, ngược lại 0). Với AND: (1,1) → −1,5 + 1 + 1 = 0,5 ≥ 0 nên ra 1; (1,0) → −0,5 < 0 nên ra 0.



Hình 5. (a) AND, (b) OR: mỗi phép chỉ cần một đường thẳng; (c) XOR: không có đường thẳng nào.

Từ một nơ-ron đến một lớp

Một lớp là m_l nơ-ron chạy song song, **cùng nhận một đầu vào**, mỗi nơ-ron có vector tham số riêng. Nơ-ron j của lớp l:

$$z_j^{(l)} = \tilde{a}^{(l-1)} w_j^{(l)},\ a_j^{(l)} = \varphi_l(z_j^{(l)}),\ j = 1, \ldots, m_l$$

b_1	b_2	...	b_m
w_{11}	w_{12}	...	w_{1m}
⋮	⋮		⋮
w_d1	w_d2	...	w_dm

$W^{(l)}$ cỡ $(m_{l-1} + 1) \times m_l$: **mỗi cột là một nơ-ron**, **hàng đầu là hệ số chệch**.

w_{ij} nối nơ-ron i của lớp trước tới nơ-ron j của lớp này.

Đúng cách xếp của **W** ở Softmax Regression, chỉ khác: cột là nơ-ron ẩn thay vì lớp.

$$Z^{(l)} = \tilde{A}^{(l-1)} W^{(l)},\ A^{(l)} = \varphi_l(Z^{(l)})$$ cho cả n mẫu, một phép nhân ma trận

Kiểm tra kích thước: thói quen cần có

$\tilde{A}^{(l-1)}$
$n \times (m_{l-1} + 1)$

$\times$

$W^{(l)}$
$(m_{l-1} + 1) \times m_l$

$=$

$Z^{(l)}, A^{(l)}$
$n \times m_l$

Số tham số của lớp

$$(m_{l-1} + 1) \cdot m_l$$

Lớp kết nối đầy đủ

(fully connected, dense): mỗi nơ-ron nhận tín hiệu từ **mọi** nơ-ron lớp trước.

Đọc theo cột: cột j là "bộ lọc" của nơ-ron j, tìm một mẫu hình trong đầu vào.

Đọc theo hàng: hàng i cho biết đặc trưng i đóng góp vào mọi nơ-ron của lớp này ra sao.

2.2 · VÍ DỤ

Một lớp ba nơ-ron ReLU

Hai mẫu $\mathbf{x}_1 = [1\ 2]$, $\mathbf{x}_2 = [3\ -1]$; lớp ba nơ-ron, $\varphi(z) = \max(0, z)$. Hàng đầu của $\mathbf{W}$ là ba hệ số chệch.

$$\underset{\tilde{\mathbf{X}}}{\begin{bmatrix} 1 & 1 & 2 \\ 1 & 3 & -1 \end{bmatrix}} \times \underset{\mathbf{W}^{(1)}}{\begin{bmatrix} \mathbf{0} & \mathbf{1} & \mathbf{-2} \\ 1 & -1 & 1 \\ -1 & 0 & 1 \end{bmatrix}} = \underset{\mathbf{Z}^{(1)}}{\begin{bmatrix} -1 & 0 & 1 \\ 4 & -2 & 0 \end{bmatrix}} \xrightarrow{\text{ReLU}} \underset{\mathbf{A}^{(1)}}{\begin{bmatrix} 0 & 0 & \mathbf{1} \\ \mathbf{4} & 0 & 0 \end{bmatrix}}$$

Phần tử đầu của $\mathbf{Z}$: $1\cdot 0 + 1\cdot 1 + 2\cdot(-1) = -1$. Phần tử âm bị ReLU đưa về 0: mẫu 1 chỉ nơ-ron thứ ba “bật”, mẫu 2 chỉ nơ-ron thứ nhất “bật”. **Mỗi nơ-ron chuyên phản ứng với một kiểu đầu vào.**

2.3 · HÀM KÍCH HOẠT

Ba hàm kích hoạt phổ biến

Hàm	$\varphi(z)$	Đạo hàm $\varphi'(z)$	Miền giá trị
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	$\sigma(z)(1 - \sigma(z))$	$(0, 1)$
Tanh	$(e^{z} - e^{-z})/(e^{z} + e^{-z})$	$1 - \tanh^2(z)$	$(-1, 1)$
ReLU	$\max(0, z)$	1 nếu $z > 0$; 0 nếu $z < 0$	$[0, \infty)$

Đạo hàm viết qua chính đầu ra $a = \varphi(z)$

$\sigma' = a(1 - a)$, $\tanh' = 1 - a^2$

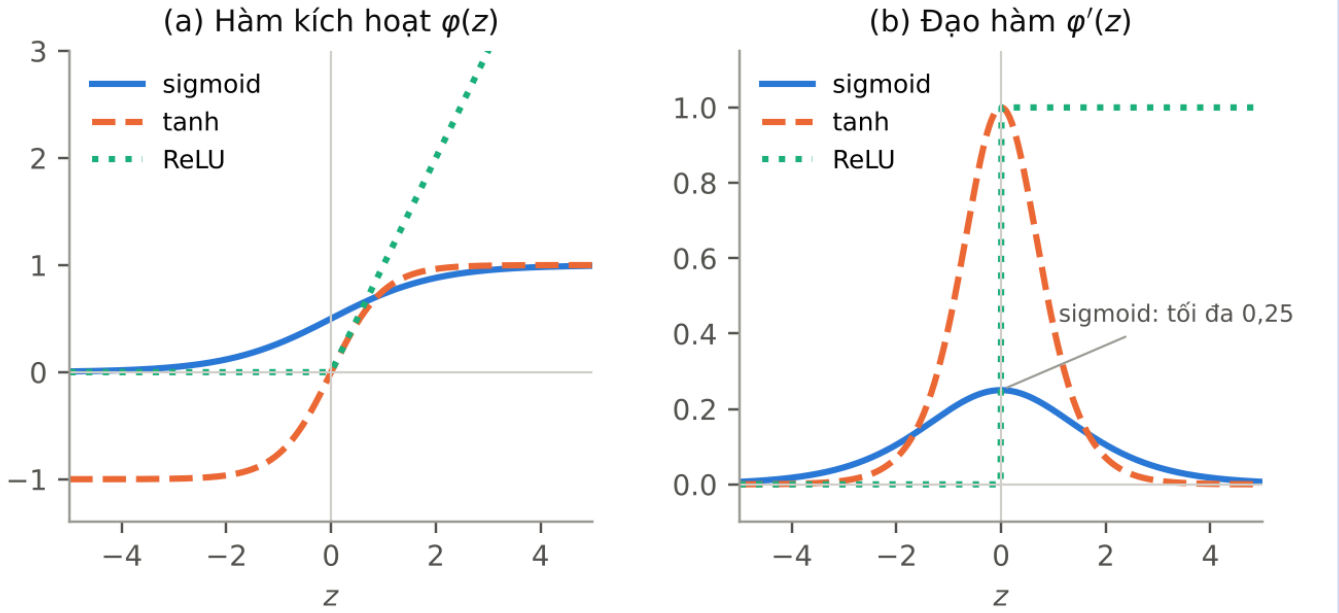
Đã tính A ở lượt thuận thì đạo hàm gần như không tốn thêm (dùng ở mục 3).

Hai ghi chú nhỏ

$\tanh(z) = 2\sigma(2z) - 1$ nên $\tanh'(z) = 4\sigma'(2z) = 1 - \tanh^2(z)$.

ReLU không khả vi tại 0; quy ước $\varphi'(0) = 0$ (hoặc 1), không ảnh hưởng huấn luyện.

Hàm kích hoạt và đạo hàm của chúng



Sigmoid: đạo hàm không vượt quá 0,25.

Sigmoid và tanh bão hòa: đạo hàm rất nhỏ khi |z| lớn.

ReLU: đạo hàm bằng 1 trên toàn nửa dương.

Hình 6. (a) Ba hàm kích hoạt phổ biến. (b) Đạo hàm của chúng.

2.3 · SỐ LIỆU CỤ THỂ

Bão hòa nghĩa là gì bằng số

z	σ(z)	σ'(z)	tanh(z)	tanh'(z)	ReLU	ReLU'
−4	0,0180	0,0177	−0,9993	0,0013	0	0
−2	0,1192	0,1050	−0,9640	0,0707	0	0
0	0,5000	0,2500	0,0000	1,0000	0	0 (quy ước)
2	0,8808	0,1050	0,9640	0,0707	2	1
4	0,9820	0,0177	0,9993	0,0013	4	1

⚠ Tại |z| = 4, tanh có đạo hàm chỉ **0,0013** và sigmoid **0,0177**. Nhân qua nhiều lớp, tín hiệu gradient co lại rất nhanh (mục 4.3). ReLU giữ nguyên đạo hàm 1 ở phía dương.

2.3 · SO SÁNH VÀ CÁCH CHỌN

Chọn hàm kích hoạt cho lớp ẩn

Sigmoid

Đầu ra trong (0,1), hợp để diễn giải là xác suất: thường dùng ở **lớp ra nhị phân**.

Ở lớp ẩn: đầu ra luôn dương, đạo hàm ≤ 0,25 nên gradient co nhanh.

Tanh

Sigmoid "căn giữa" về 0: $\tanh(z) = 2\sigma(2z) - 1$.

Cân xứng quanh 0 giúp học dễ hơn sigmoid ở lớp ẩn, nhưng vẫn bão hòa khi $|z|$ lớn.

ReLU

Tính nhanh, không bão hòa ở phía dương, cho nơ-ron "thưa".

Nhược điểm: nơ-ron có $z < 0$ với mọi mẫu sẽ "chết". Leaky ReLU: $\max(\alpha z, z)$, $\alpha \approx 0{,}01$.

☆ **ReLU và các biến thể** là mặc định cho lớp ẩn trong học sâu hiện đại. Trong chương này dùng **tanh** cho các ví dụ nhỏ vì đạo hàm gọn, dễ kiểm tra bằng tay.

2.3 · MỘT CHI TIẾT BẮT BUỘC

Vì sao φ bắt buộc phải phi tuyến?

Giả sử mọi lớp ẩn có φ(z) = z. Tách $\mathbf{W}^{(l)}$ thành hàng chệch $\mathbf{b}^{(l)}$ và phần còn lại $\mathbf{V}^{(l)}$:

$$\mathbf{A}^{(l)} = \mathbf{1}\, \mathbf{b}^{(l)} + \mathbf{A}^{(l-1)}\, \mathbf{V}^{(l)}$$

Thay lần lượt $A^{(1)}$ vào $A^{(2)}$, rồi $A^{(3)}$, ...:

$$\mathbf{A}^{(L)} = \mathbf{1}\, \mathbf{b}_\mathbf{eff} + \mathbf{X} \cdot \mathbf{V}^{(1)}\mathbf{V}^{(2)} \dots \mathbf{V}^{(L)}$$ một hàm affine của x

Mạng sâu bao nhiêu cũng chỉ **tương đương một lớp** với ma trận [b_eff; V_eff]. Lớp ra softmax thì chỉ có lại Softmax Regression.

Phi tuyến biến việc xếp chồng thành khả năng biểu diễn mới. φ cũng nên khả vi gần như khắp nơi, và **không** là đa thức.

2.3 · LỚP RA

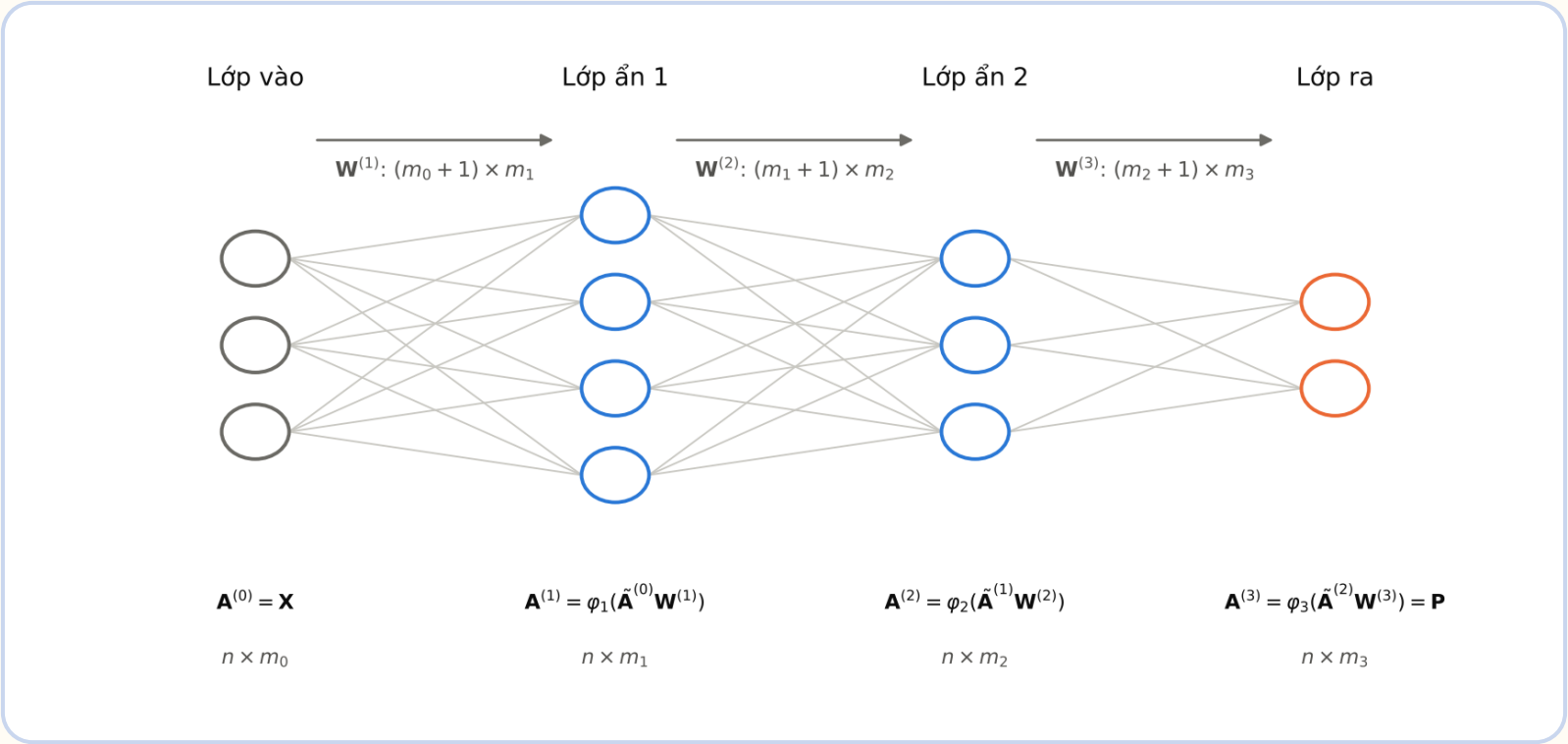
Hàm kích hoạt ở lớp ra khác lớp ẩn

Lớp ra phải cho đầu ra **đúng dạng mà bài toán cần**:

Bài toán	Nơ-ron ra	Kích hoạt ra	Ý nghĩa đầu ra
Hồi quy (số thực)	1	Đồng nhất	Giá trị dự đoán
Phân loại nhị phân	1	Sigmoid	$p = P(y = 1 \mid x)$
Phân loại K lớp (mỗi mẫu một lớp)	K	Softmax	p_k, cộng lại bằng 1
Nhiều nhãn (một mẫu nhiều nhãn)	K	K sigmoid độc lập	K xác suất không ràng buộc nhau

Mẹo nhớ: các nhãn **loại trừ nhau** thì dùng softmax; nhãn **cùng tồn tại** được thì mỗi nhãn một sigmoid.

Mạng truyền thẳng L lớp (MLP)



Hình 7. Mạng 3–4–3–2, L = 3. Dưới mỗi lớp: công thức và kích thước ma trận kích hoạt; trên các mũi tên: kích thước ma trận trọng số. Hệ số chệch không vẽ. MLP là phép hợp của L lớp, thông tin đi một chiều, không có vòng lặp ngược.

2.4 · LAN TRUYỀN THUẬN

Tính đầu ra từ lớp này sang lớp kia

Khởi tạo

$A^{(0)} = X$

Với l = 1, 2, …, L

$Z^{(l)} = \tilde{A}^{(l-1)} W^{(l)}$, $A^{(l)} = \varphi_l(Z^{(l)})$

Đầu ra

$\hat{Y} = A^{(L)} = P$

Mạng là hàm hợp: **P** = f_L(f_{L−1}(… f_1(**X**) …)), với f_l(**A**) = $\varphi_l(\tilde{A}\, W^{(l)})$. Ta cần **lưu lại** các $A^{(l)}$ vì lan truyền ngược (mục 3) dùng lại chúng.

Lớp l	$\tilde{A}^{(l-1)}$	$W^{(l)}$	$Z^{(l)}$, $A^{(l)}$
1	n × (m_0 + 1)	(m_0 + 1) × m_1	n × m_1
2	n × (m_1 + 1)	(m_1 + 1) × m_2	n × m_2
L	n × (m_{L−1} + 1)	(m_{L−1} + 1) × m_L	n × m_L

2.4 · ĐẾM THAM SỐ

Mạng có bao nhiêu tham số?

$$\sum_l (m_{l-1} + 1) \cdot m_l$$ tổng số phần tử của mọi ma trận $W^{(l)}$, kể cả hàng chệch

Ví dụ: mạng 784–128–64–10 (ảnh chữ số 28×28, hai lớp ẩn, 10 lớp):

Lớp 1	Lớp 2	Lớp 3	Tổng
785 × 128	129 × 64	65 × 10	=
100.480	8.256	650	109.386

So với **3.085.050** tham số của Softmax Regression trên đặc trưng đa thức bậc hai (mục 1.2), mạng này nhỏ hơn khoảng **28 lần** mà vẫn học được đặc trưng phi tuyến.

Mạng 2–3–3: lớp ẩn tanh

Dùng lại hai mẫu ở mục 2.2, đổi kích hoạt thành **tanh**, thêm lớp ra softmax 3 lớp. Nhãn thật: $y_1 = 2$, $y_2 = 0$, tức **Y** = [[0 0 1], [1 0 0]].

$Z^{(1)}$ (như 2.2)

−1	0	1
4	−2	0

tanh →

$A^{(1)}$

−0,7616	0	0,7616
0,9993	−0,9640	0

$W^{(2)}$ (4 × 3)

0	**0,5**	**−0,5**
1	0	−1
−1	1	0
0	−1	1

Lớp ra có kích thước (3 + 1) × 3: hàng đầu là chệch. **Bước tiếp:** thêm cột 1 vào $A^{(1)}$, nhân với $W^{(2)}$ để được điểm số, rồi softmax từng hàng.

Lớp ra softmax và hàm mất mát

$S = \tilde{A}^{(1)}W^{(2)}$

−0,7616	−0,2616	1,0232
1,9634	−0,4640	−1,4993

softmax →

P

0,1162	0,1916	**0,6923**
0,8932	0,0788	0,0280

Ví dụ $s_{11} = 0 + 1\cdot(-0{,}7616) + (-1)\cdot 0 + 0\cdot 0{,}7616 = -0{,}7616$. Mỗi hàng của **P** cộng lại bằng 1: một phân phối xác suất.

Xác suất của lớp thật

$q_1 = p_{1,2} = 0{,}6923$

$q_2 = p_{2,0} = 0{,}8932$

Mất mát từng mẫu

$l_1 = -\ln 0{,}6923 \approx 0{,}3678$

$l_2 = -\ln 0{,}8932 \approx 0{,}1130$

Hàm mất mát

$J \approx 0{,}2404$

Mạng đoán đúng cả hai mẫu (lớp 2 và lớp 0) dù trọng số chọn tay; mẫu thứ nhất kém tự tin hơn.

Lớp ra và mất mát phải đi đôi

Bài toán	Lớp ra	Mất mát một mẫu	Gradient theo điểm số $z^{(L)}$
Hồi quy	Đồng nhất, $\hat{y}$	$\frac{1}{2}(\hat{y} - y)^2$	$\hat{y} - y$
Nhị phân	Sigmoid, p	$-y \ln p - (1 - y) \ln(1 - p)$	$p - y$
K lớp	Softmax, p	$-\sum_k y_k \ln p_k$	$p - y$

Mất mát là âm log-hợp lý của mô hình xác suất tương ứng, nên gradient ở lớp ra luôn có dạng "**dự đoán trừ thực tế**". Với K lớp:

$$J = -(1/n) \sum_i \sum_k y_{ik} \ln p_{ik}$$

Vì sao cross-entropy tốt hơn MSE cho phân loại: gradient chỉ còn **p − y**, không có thừa số p(1 − p) làm gradient tắt khi mô hình sai chắc chắn.

Siêu tham số kiến trúc (dữ liệu không tự quyết): số lớp ẩn, số nơ-ron mỗi lớp, hàm kích hoạt. Chọn bằng tập kiểm định (mục 5.2).

Mạng 2–2–1 giải XOR

$W^{(1)}$

−0,5	−1,5
1	1
1	1

$W^{(2)}$

−0,5
1
−1

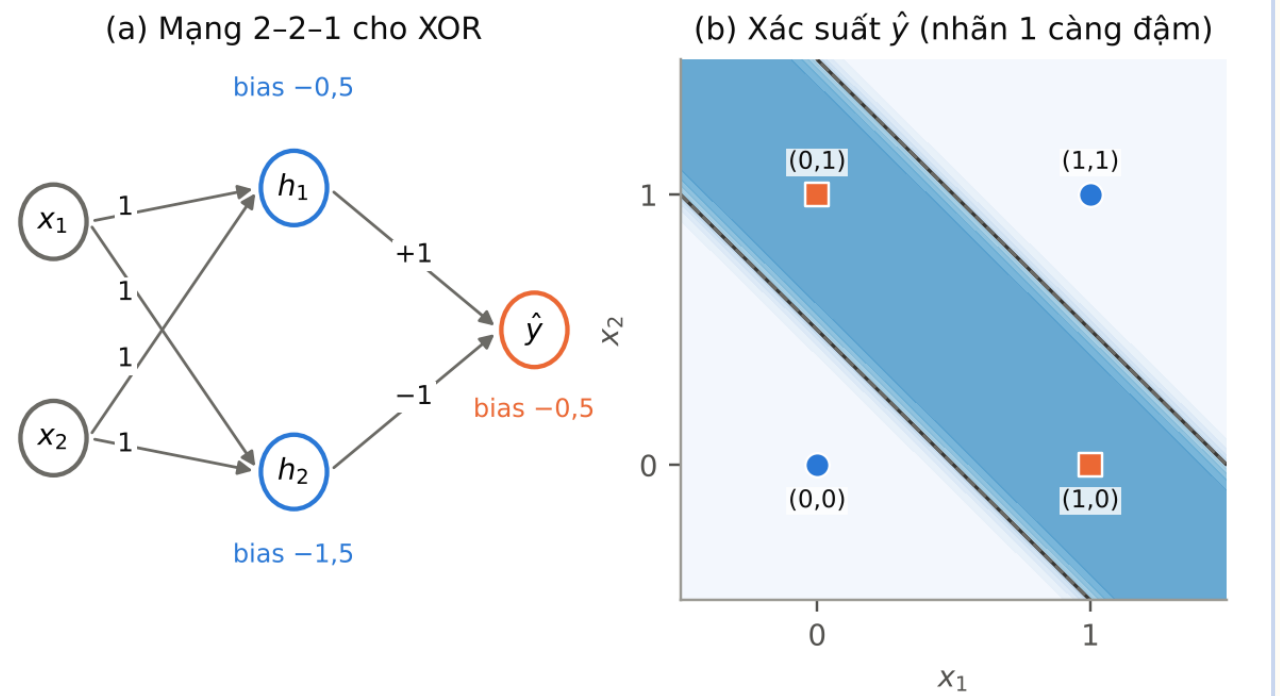
Cột 1 của $W^{(1)}$: nơ-ron **OR**. **Cột 2**: nơ-ron **AND**.

$W^{(2)}$ kết hợp: "OR nhưng không AND". Hàm bước ở cả hai lớp.

(x_1, x_2)	$z_1^{(1)}$	$z_2^{(1)}$	h_1	h_2	$z^{(2)} = -0{,}5 + h_1 - h_2$	$\hat{y}$	Nhãn
(0,0)	−0,5	−1,5	0	0	−0,5	0	0
(0,1)	0,5	−0,5	1	0	0,5	1	1
(1,0)	0,5	−0,5	1	0	0,5	1	1
(1,1)	1,5	0,5	1	1	−0,5	0	0

Cả bốn mẫu đúng. Nguyên nhân: lớp ẩn **phi tuyến** biến 4 điểm XOR thành 3 điểm (0,0), (1,0), (1,1) trong không gian (h_1, h_2), nơi một đường thẳng đã đủ.

Vùng quyết định của mạng giải XOR



Lớp 1 là **dải** giữa hai đường $x_1 + x_2 = 0{,}5$ và $x_1 + x_2 = 1{,}5$ (giao của hai nửa mặt phẳng).

Lớp 0 là **hợp của hai** nửa mặt phẳng ở hai phía dải: **không lồi**, gồm hai nhóm rời nhau.

Đúng điều mà Softmax Regression **không làm được**.

Hình 8. (a) Mạng 2–2–1; số trên cạnh là trọng số, số cạnh nơ-ron là hệ số chệch. (b) Xác suất lớp 1 với sigmoid $\sigma(10z)$; nét đứt là biên của hai nơ-ron ẩn.

2.5 · KHẢ NĂNG BIỂU DIỄN

Biên cong và vùng không lồi

Một nơ-ron ẩn
tạo một "đường thẳng"

→

m nơ-ron ẩn
tạo m đường thẳng

→

Lớp ra tổ hợp
giao, hợp, trừ các nửa mặt phẳng

Với kích hoạt mềm như tanh, biên gấp khúc trở thành **đường cong trơn**. Đó là cách mạng 2–8–3 (mục 1.3) tạo các vòng khép kín ôm lớp ở giữa.

Nguyên lý: số nơ-ron ẩn càng nhiều thì vùng quyết định càng có thể phức tạp. Ba vòng tròn đồng tâm:

m = 2
63–70%
mới có 2 đường biên, chưa đủ để lồng hai vòng

m = 4
96,1–98,9%
đủ "nguyên liệu"

m = 8
96,1–99,4%
cho kết quả tốt nhất, trên 96%

Mạng có giới hạn biểu diễn nào không?

Cybenko 1989; Hornik và cộng sự 1989, 1991

Cho hàm liên tục g trên một tập compact và sai số $\varepsilon > 0$ bất kỳ. Với hàm kích hoạt phi tuyến, bị chặn, tăng (ví dụ sigmoid), tồn tại mạng **một lớp ẩn** với số nơ-ron m đủ lớn sao cho hàm mạng tính được khác g **không quá ε tại mọi điểm** của tập đó.

Một lớp ẩn đủ rộng **xấp xỉ được mọi hàm liên tục** với độ chính xác tùy ý.

Cũng đúng với ReLU và mọi kích hoạt liên tục không phải đa thức (Leshno và cộng sự 1993). Với phân loại, vùng quyết định cũng xấp xỉ được tùy ý.

Xếp các “bậc thang” sigmoid

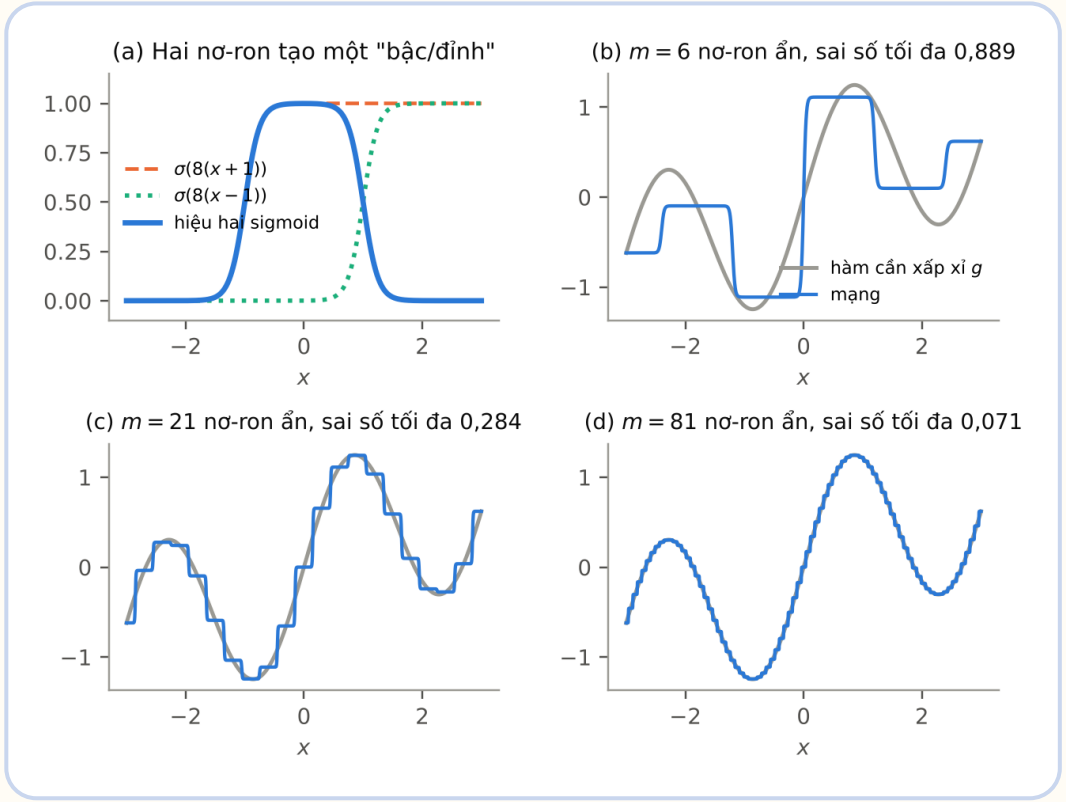
Bậc thang: $\sigma(k(x - t))$ với k lớn ≈ 0 khi $x < t$, ≈ 1 khi $x > t$.

Hiệu hai bậc thang cho một “đỉnh” trên (a, b) (Hình 9a).

$$f(x) = \sum_j c_j \, \sigma(k(x - t_j))$$

$c_j = g(x_j) - g(x_{j-1})$: độ cao = phần tăng thêm của g

Đúng là **mạng một lớp ẩn** m nơ-ron: trọng số vào k, chệch $-k \cdot t_j$, trọng số ra c_j.

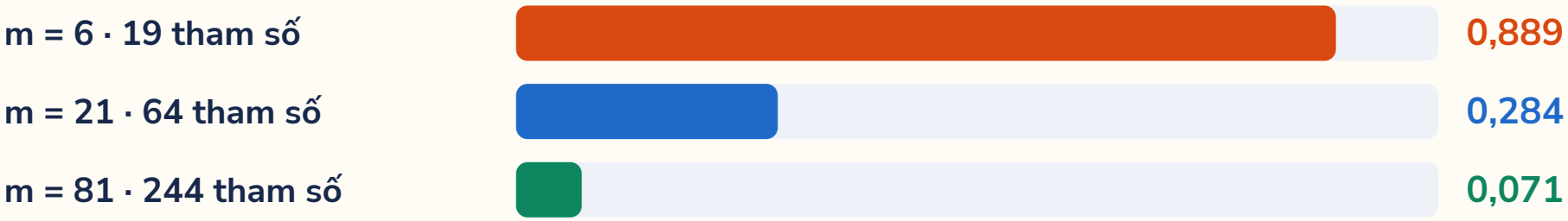


Hình 9. (a) Hiệu của hai sigmoid dốc tạo một “đỉnh”. (b), (c), (d) Tổng m bậc thang xấp xỉ g(x) = sin(2x) + 0,3x (nét xám); m càng lớn sai số càng nhỏ.

2.5 · THỬ BẰNG SỐ

Sai số giảm khi tăng số nơ-ron

Xấp xỉ g(x) = sin(2x) + 0,3x trên [–3, 3] bằng mạng một lớp ẩn (3m + 1 tham số):



Độ dài thanh = sai số lớn nhất max |f(x) – g(x)| (thang 0 đến 1).

Từ m = 21 lên m = 81 (gấp khoảng 4 lần), sai số giảm từ 0,284 xuống 0,071, cũng khoảng **4 lần**: sai số tỉ lệ với độ rộng mỗi đoạn, tức tỉ lệ với **1/m**.

2.5 · ĐỌC ĐÚNG ĐỊNH LÝ

Định lý nói gì, và không nói gì

Định lý NÓI

Tồn tại mạng một lớp ẩn đủ rộng xấp xỉ được hàm đích.

Mạng không bị giới hạn về **khả năng biểu diễn** như mô hình tuyến tính.

Về lý thuyết, một lớp ẩn là đủ.

Định lý KHÔNG NÓI

Cần **bao nhiêu** nơ-ron: có hàm đòi số nơ-ron rất lớn, tăng nhanh theo số chiều d.

Thuật toán huấn luyện có **tìm được** trọng số đó không (hàm mất mát không lồi, mục 4.2).

Mạng có **khái quát hóa** tốt không (có thể khớp quá mức nhiễu, mục 4.4).

Vì thế thực hành thường dùng mạng **sâu hơn** thay vì rộng hơn: mỗi lớp tái sử dụng đặc trưng của lớp trước nên biểu diễn gọn hơn nhiều hàm. So sánh rộng và sâu ở mục 4.5.

TỔNG KẾT · LIÊN HỆ

Softmax Regression và MLP

Thành phần	Softmax Regression	Mạng nơ-ron (MLP)
Mô hình	$P = \text{softmax}(\tilde{X}W)$	$A^{(l)} = \varphi_l(\tilde{A}^{(l-1)}W^{(l)})$, $P = A^{(L)}$
Hàm mất mát	Cross-entropy, lồi theo W	Cùng cross-entropy, không lồi theo $\{W^{(l)}\}$
Gradient	$(1/n)\,\tilde{X}^{T}(P - Y)$	Lan truyền ngược (mục 3)

Mục 1 đặt vấn đề; mục 2 hoàn thiện **mô hình**. Điểm khác biệt còn lại, **dòng gradient**, là chủ đề mục 3.

Mạng có hàng trăm ma trận W lồng nhau, và các nơ-ron ẩn **không có nhãn "đúng"** để so sánh: cần một cách có hệ thống để tính gradient theo từng $W^{(l)}$.

TÓM TẮT MỤC 2 · PHẦN 1/2

Từ nơ-ron đến lớp và kích hoạt

1. Nơ-ron

Tính tổng có trọng số rồi áp hàm kích hoạt: $s = \tilde{x}w$, $a = \varphi(s)$. Perceptron, Logistic, Softmax đều là mạng **không có lớp ẩn**.

2. Lớp

Nhiều nơ-ron song song gộp thành ma trận $W^{(l)}$ cỡ $(m_{(l-1)} + 1) \times m_l$ có hàng chệch: $Z = \tilde{A}W$, $A = \varphi(Z)$.

3. Hàm kích hoạt

Sigmoid, tanh, ReLU và đạo hàm của chúng. Phải **phi tuyến**: nếu không, tích nhiều ma trận vẫn chỉ là một phép biến đổi affine.

TÓM TẮT MỤC 2 · PHẦN 2/2

Lan truyền thuận và khả năng biểu diễn

4. Lan truyền thuận

Lặp $Z^{(l)} = \tilde{A}^{(l-1)}W^{(l)}$, $A^{(l)} = \varphi_l(Z^{(l)})$ từ đầu vào đến đầu ra. Lớp ra và hàm mất mát **đi theo cặp** (identity + MSE; sigmoid + BCE; softmax + CE), cho gradient ở đầu ra là $p - y$. Số tham số: $\sum (m_{(l-1)} + 1)\, m_l$.

5. Khả năng biểu diễn

Lớp ẩn phi tuyến cho phép biên cong (XOR giải được bằng mạng 2-2-1). Định lý xấp xỉ phổ quát: một lớp ẩn đủ rộng xấp xỉ mọi hàm liên tục, nhưng **không** nói cần bao nhiêu nơ-ron, có huấn luyện được hay có khái quát hóa tốt.

[icon: Arrow-right] **Mục 3:** mô hình đã đủ mạnh, vậy làm sao tính gradient của hàng loạt ma trận $W^{(l)}$? Đó là thuật toán lan truyền ngược.

KIỂM TRA NHANH · CÂU 1 ĐẾN 5

Bạn đã nắm được chưa?

1. $w = [-1\ 2\ -3]^T$, $x = [1\ 2]$. Tính s và a với sigmoid, tanh, ReLU.

2. Softmax có $w_0 = [1\ 0\ 2]^T$, $w_1 = [0\ 1\ 1]^T$. Tìm w của sigmoid tương đương và p_1 tại $x = [2\ 1]$.

3. Mạng 5–8–4–3: kích thước $W^{(1)}$, $W^{(2)}$, $W^{(3)}$ và tổng số tham số.

4. Tính $\sigma'(0)$, $\tanh'(1)$ (biết $\tanh 1 \approx 0{,}7616$) và $\text{ReLU}'(-3)$.

5. Chứng minh mạng 2 lớp có kích hoạt ẩn là identity và đầu ra softmax tương đương Softmax Regression. Viết ma trận tương đương.

KIỂM TRA NHANH · CÂU 6 ĐẾN 9

Vận dụng và suy luận

6 Mạng XOR ở mục 2.5, tại (1, 0): tính $z^{(1)}$, h và ŷ.

7 Nơ-ron ra sigmoid có $w = [0{,}5\ 1\ -1\ 2]^T$ và $a^{(1)} = [0\ 0\ 1]$. Tính xác suất.

8 Chọn lớp ra và hàm mất mát cho: (a) dự đoán giá nhà; (b) có người hay không; (c) chữ số viết tay; (d) gắn nhiều thẻ chủ đề cho bài báo (đa nhãn).

9 Sai số giảm từ 0,284 xuống 0,071 khi m tăng từ 21 lên 81. Tỉ lệ là bao nhiêu và nó cho biết điều gì?

ĐÁP ÁN · CÂU 1 ĐẾN 5

Lời giải ngắn gọn

1 Với $\tilde{x} = [1\ 1\ 2]$: $s = -1 + 2 - 6 = \mathbf{-5}$. $\sigma(-5) \approx 0{,}0067$; $\tanh(-5) \approx -0{,}9999$; ReLU = 0.

2 $w = w_1 - w_0 = \mathbf{[-1\ 1\ -1]}^T$. Tại $x = [2\ 1]$: $s = -1 + 2 - 1 = 0$, nên $p_1 = \mathbf{0{,}5}$.

3 $W^{(1)}$: 6×8; $W^{(2)}$: 9×4; $W^{(3)}$: 5×3. Tham số: 48 + 36 + 15 = **99**.

4 $\sigma'(0) = \mathbf{0{,}25}$; $\tanh'(1) = 1 - 0{,}7616^2 \approx \mathbf{0{,}4200}$; $\text{ReLU}'(-3) = \mathbf{0}$.

5 Với kích hoạt ẩn identity: $S = 1b^T + \tilde{X}W^{(1)}V$. Ma trận tương đương $W_{eff} = W^{(1)}V + e_1 b^T$ (e_1 cộng b vào hàng chệch), nên mô hình vẫn là Softmax Regression.

ĐÁP ÁN · CÂU 6 ĐẾN 9

Lời giải ngắn gọn

6 $z^{(1)} = [0{,}5\ -0{,}5]$, $h = [1\ 0]$ (ReLU), điểm ra 0,5 nên $\hat{y} =$ **1**.

7 Với $\tilde{a} = [1\ 0\ 0\ 1]$: $s = 0{,}5 + 2 =$ **2,5**; $p = \sigma(2{,}5) \approx$ **0,9241**.

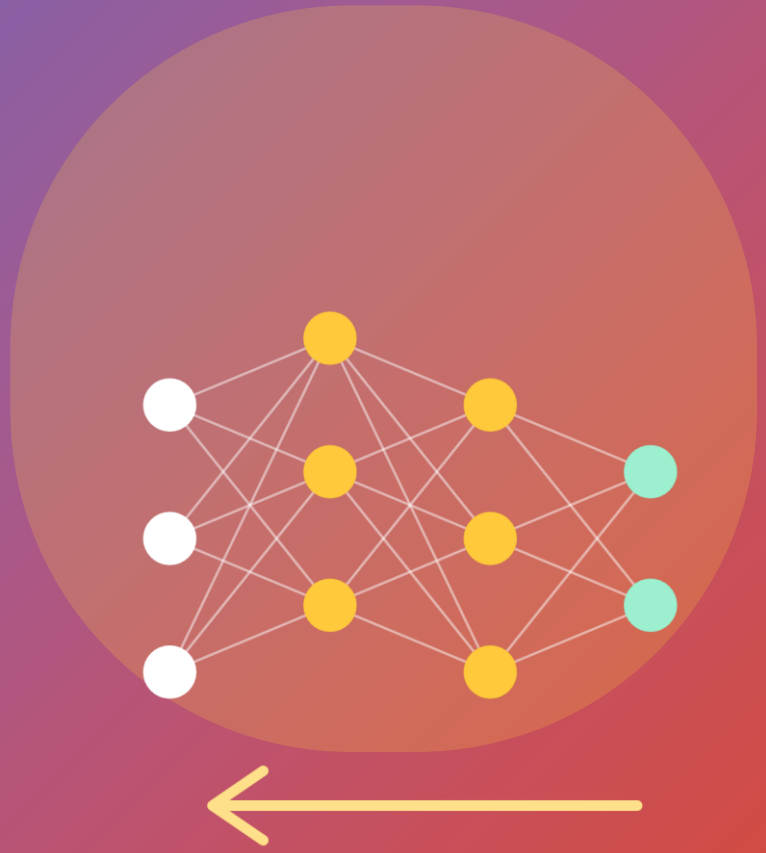
8 (a) identity + MSE. (b) một sigmoid + BCE. (c) 10 nơ-ron softmax + cross-entropy. (d) một sigmoid độc lập cho mỗi chủ đề, tổng BCE, **không** dùng softmax.

9 $0{,}284 / 0{,}071 \approx 4{,}0 \approx 81 / 21 \approx 3{,}9$: sai số **tỉ lệ nghịch với m**.

CHƯƠNG: MẠNG NƠ-RON NHÂN TẠO · MỤC 3

Huấn luyện mạng nơ-ron: lan truyền ngược

Làm sao để máy tự tìm các ma trận trọng số $W^{(l)}$ từ dữ liệu



KẾ HOẠCH BÀI HỌC

Còn thiếu hai mảnh ghép để huấn luyện

Mô hình ✓ + Hàm mất mát ✓ + Thuật toán tối ưu: $W^{(l)} \leftarrow W^{(l)} - \eta \nabla J$, cần ∇J cho mọi lớp

1 · Vì sao

Vì sao các quy tắc cũ và đạo hàm số không đủ?

2 · Công cụ

Đạo hàm hàm hợp và đồ thị tính toán cho ta gì?

3 · Một lớp ẩn

Suy ra gradient từng ma trận W của mạng một lớp ẩn.

4 · L lớp

Thuật toán tổng quát và cách kiểm tra gradient.

5 · Tối ưu

Khởi tạo, tốc độ học, SGD, momentum, Adam.

6 · Thực hành

Tính tay một vòng và viết mã Python từ đầu.

KÝ HIỆU

Ký hiệu bổ sung cho mục 3

Đối tượng	Ký hiệu	Kích thước	Ý nghĩa
Sai số tại lớp l	$D^{(l)} = \partial J / \partial Z^{(l)}$	$n \times m_l$	“Tín hiệu lỗi”: đạo hàm của J theo điểm số của lớp l
Trọng số bỏ chệch	$V^{(l)}$	$m_{(l-1)} \times m_l$	$W^{(l)}$ bỏ hàng đầu (hàng hệ số chệch)
Đạo hàm kích hoạt	$\varphi'(Z^{(l)})$	$n \times m_l$	Áp dụng lên từng phần tử của $Z^{(l)}$
Nhân từng phần tử	$A \odot B$	cùng cỡ A, B	$(A \odot B)_{ij} = a_{ij}\, b_{ij}$
Gradient theo trọng số	∇J theo $W^{(l)}$	$(m_{(l-1)} + 1) \times m_l$	Cùng kích thước với $W^{(l)}$

Quy tắc nhớ nhanh: **gradient của một đại lượng luôn có cùng kích thước với chính đại lượng đó.**

3.1 · VÌ SAO CẦN LAN TRUYỀN NGƯỢC?

Điều đã biết: nơ-ron nào cũng có “đáp án”

Perceptron

Mẫu bị phân loại sai thì kéo w về phía mẫu đó:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i\, \tilde{\mathbf{x}}_i^T$$

Logistic · Softmax

Gradient dùng sai số “dự đoán trừ thực tế”:

$$\nabla J = (1/n)\, \tilde{\mathbf{X}}^T\, (\mathbf{P} - \mathbf{Y})$$

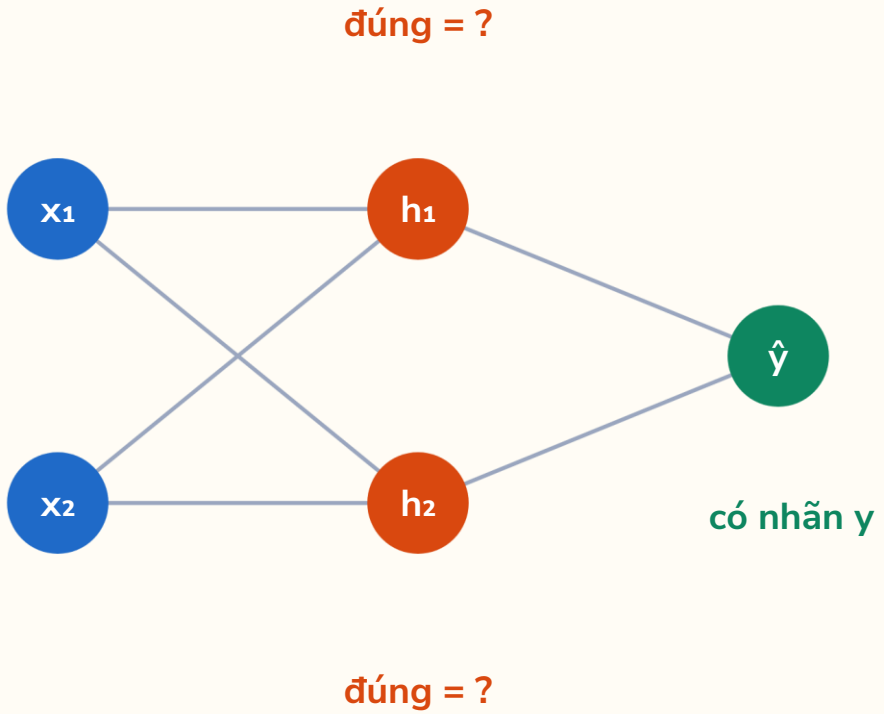
Điểm chung then chốt: nơ-ron ra thứ k được so với cột k của Y, nên sai số $p_{ik} - y_{ik}$ **tính được ngay từ dữ liệu**. Chỉ cần nhân sai số với đầu vào $\tilde{x}_i^T$ của nơ-ron đó.

Điều khó: nơ-ron ẩn không có đáp án

Nhãn y_i chỉ cho biết đầu ra cuối $\hat{y}_i$ phải là 0 hay 1. **Không ai cho biết h_1 "đúng" phải bằng bao nhiêu** với mẫu (0, 1).

Bài toán gán trách nhiệm (credit assignment): dự đoán sai thì lỗi ở nơ-ron ra, ở h_1, ở h_2 hay cả ba? Mỗi trọng số góp bao nhiêu?

Quy tắc Perceptron cần sai số của chính nơ-ron đang cập nhật, nên **bó tay với lớp ẩn**. Đó là lý do mạng nhiều lớp bị bỏ quên sau năm 1969.



Phương án “ngây thơ”: đạo hàm số

Với mỗi trọng số w, thay w bằng w ± ε rồi chạy lại lan truyền thuận:

$$\partial J/\partial w \approx [\, J(w + \varepsilon) - J(w - \varepsilon) \,] / (2\varepsilon)$$

Đúng nhưng đắt: mỗi tham số cần **hai** lượt lan truyền thuận, nên một gradient cho P tham số tốn **2P** lượt.

Mạng	Số tham số P	Số lượt thuận cho một gradient
2–4–3	27	54
2–8–3	51	102
784–128–64–10	109 386	218 772
Mạng cỡ 10^8 tham số	10^8	2×10^8

3.1 · ĐO THỰC NGHIỆM

Đạo hàm số chậm gấp khoảng 98 lần

Mạng 2–8–3 với 420 mẫu dữ liệu vòng tròn, đo thời gian cho **một gradient**:

0,063 ms

một lượt lan truyền thuận

0,098 ms

thuận + ngược (chỉ gấp 1,56 lần)

9,6 ms

đạo hàm số (gấp khoảng 98 lần, cỡ 2P)

Mạng càng lớn, chênh lệch càng khủng khiếp: lan truyền ngược chỉ tốn cỡ **3 lượt thuận** bất kể P, đạo hàm số tốn **2P** lượt.

3.1 · Ý TƯỞNG LAN TRUYỀN NGƯỢC

Đi ngược từ đầu ra, tái sử dụng kết quả

Mạng là hàm hợp của nhiều phép tính đơn giản (mục 2.4), nên gradient tính được bằng **quy tắc đạo hàm hàm hợp**.

Tái sử dụng: đạo hàm của các lớp sau được dùng lại cho các lớp trước, nên cả P đạo hàm chỉ tốn cỡ vài lượt thuận, không phải 2P.

Mỗi lớp đều có dạng quen thuộc của Softmax:

$$\nabla J \text{ theo } W^{(l)} = \tilde{A}^{(l-1)T} \cdot D^{(l)}$$

- ∇J theo $W^{(l)}$: gradient
- $\tilde{A}^{(l-1)T}$: đầu vào của lớp
- $D^{(l)}$: tín hiệu lỗi tại lớp

Điều mới: $D^{(l)}$ của lớp ẩn **không đọc từ nhãn mà được truyền ngược** từ lớp sau. Ý tưởng có từ thập niên 1960–1970 (như Werbos 1974), được Rumelhart, Hinton và Williams phổ biến năm 1986.

3.2 · CÔNG CỤ TOÁN

Quy tắc đạo hàm hàm hợp

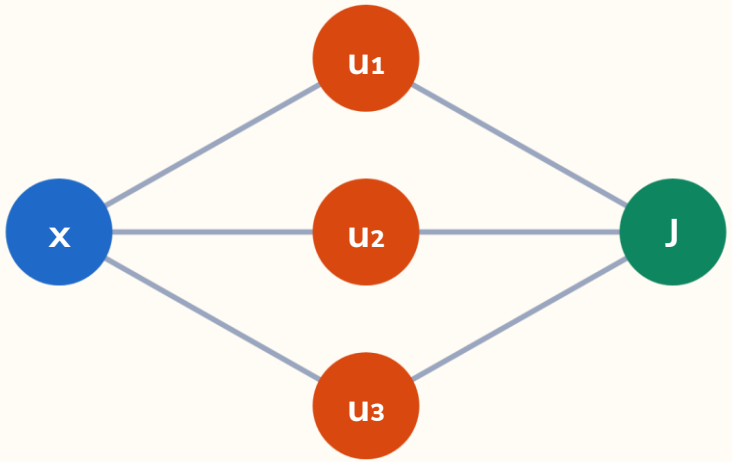
Một đường: $y = f(g(x))$, $u = g(x)$

$$dy/dx = (dy/du) \cdot (du/dx) = f'(u)\, g'(x)$$

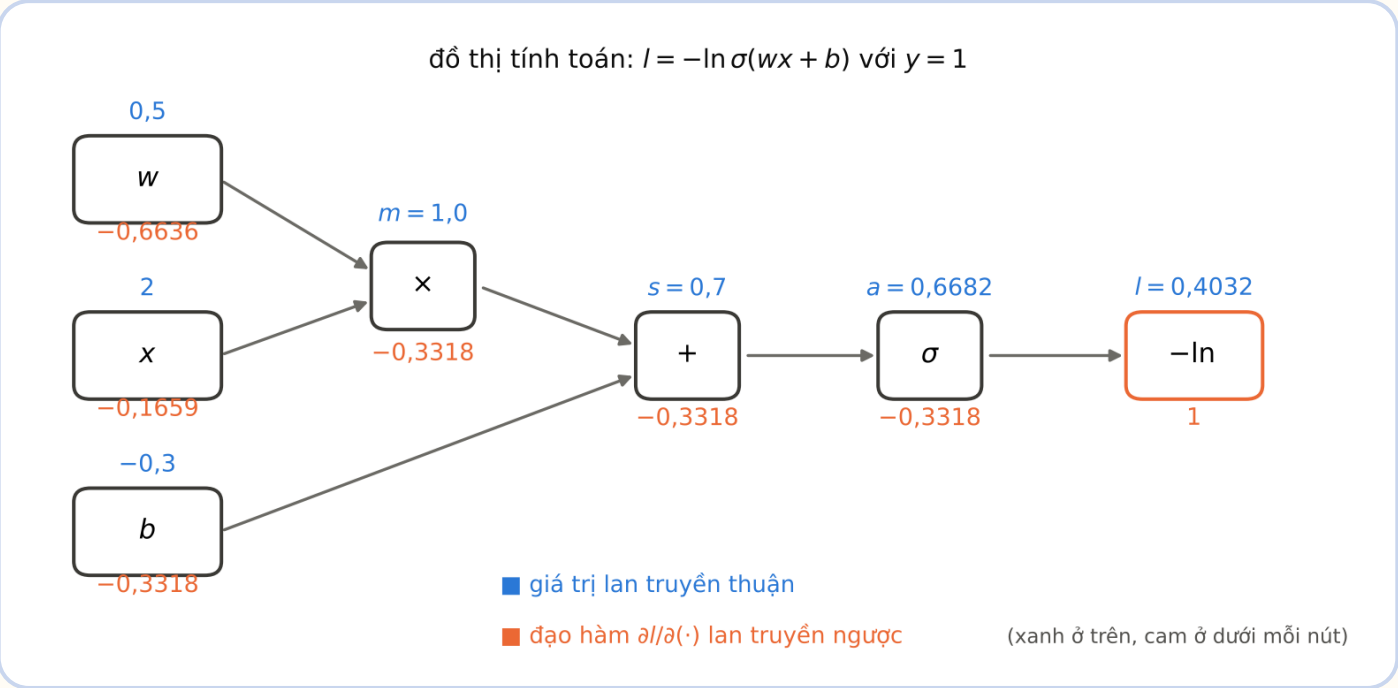
Nhiều đường: cộng đóng góp của mọi đường

$$\partial J/\partial x = \sum_k (\partial J/\partial u_k) \cdot (\partial u_k/\partial x)$$

Công thức cộng theo đường chính là lý do mục 3.3 có phép nhân ma trận $\mathbf{D}^{(2)}\mathbf{V}^{(2)T}$: tổng $\sum$ trên mọi nơ-ron ra mà nơ-ron ẩn nối tới.



Hai chiều: thuận để tính, ngược để học



Chiều thuận: tính giá trị tại mỗi nút từ trái sang phải và **lưu lại** giá trị trung gian.

Chiều ngược: bắt đầu từ ∂J/∂J = 1; ở mỗi nút, lấy đạo hàm nhận từ sau **nhân đạo hàm cục bộ**, rồi chuyển về trước (cộng nếu nhiều đường).

Hình 10. Đồ thị tính toán của l = −ln σ(wx + b). Giá trị thuận ghi ở trên mỗi nút (xanh), đạo hàm ∂l/∂(·) ghi ở dưới (cam).

3.2 · VÍ DỤ SỐ: MỘT NƠ-RON SIGMOID

$w = 0{,}5 \cdot x = 2 \cdot b = -0{,}3 \cdot y = 1$

Chiều thuận

$m = wx = 1{,}0$

$s = m + b = 0{,}7$

$a = \sigma(0{,}7) \approx 0{,}6682$

$l = -\ln a \approx 0{,}4032$

Chiều ngược

$\partial l/\partial a = -1/a \approx -1{,}4966$; $\partial a/\partial s = a(1-a) \approx 0{,}2217$

$\partial l/\partial s = a - 1 \approx -0{,}3318$ $(= p - y)$

$\partial l/\partial b = -0{,}3318$

$\partial l/\partial w = -0{,}3318 \times 2 =$ **$-0{,}6636$**; $\partial l/\partial x = -0{,}3318 \times 0{,}5 = -0{,}1659$

Kiểm tra bằng đạo hàm số ($\varepsilon = 10^{-6}$): $-0{,}66362446$ và $-0{,}33181223$, khớp đến 10^{-10}.

Một bước GD ($\eta = 0{,}5$): $w = 0{,}8318$, $b = -0{,}1341$, nên $a \approx 0{,}8219$ và l giảm từ $0{,}4032$ xuống $0{,}1961$.

3.2 · DẠNG MA TRẬN CHO MỘT LỚP

Ba quy tắc, biết D = ∂J/∂Z (n × m)

Quy tắc 1 · theo trọng số

$$\nabla J = \tilde{A}^T D$$

Từ Z = ÃW. Kích thước: $(m_{(l-1)}+1) \times m$, đúng cỡ W.

Quy tắc 2 · theo đầu vào

$$\partial J/\partial A = D V^T$$

V là W bỏ hàng chệch (cột 1 của Ã là hằng số 1). Kích thước $n \times m_{(l-1)}$.

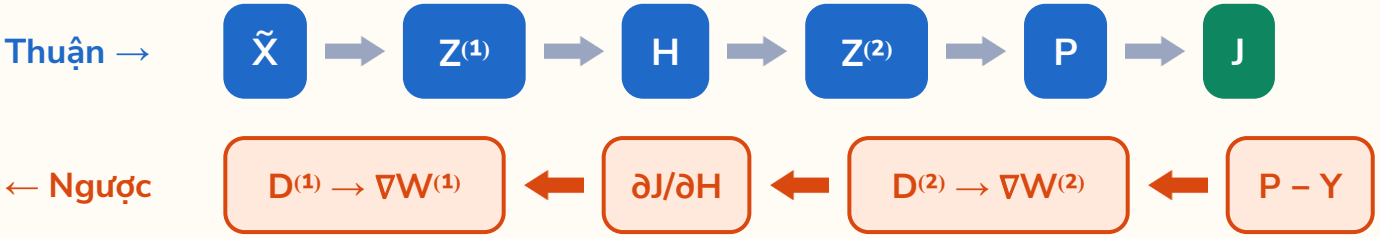
Quy tắc 3 · qua kích hoạt

$$\partial J/\partial Z = \partial J/\partial A \odot \varphi'(Z)$$

Vì a_{ij} chỉ phụ thuộc z_{ij} nên nhân từng phần tử.

✓ **Kiểm tra kích thước:** $\tilde{A}^T$ là $(m_{(l-1)}+1) \times n$, D là $n \times m$, tích là $(m_{(l-1)}+1) \times m$. Hàng đầu của $\tilde{A}^T D$ là $1^T D$, tức **tổng các cột của D theo mẫu** (gradient của hệ số chệch).

Mô hình và đường đi ngược



$$Z^{(1)} = \tilde{X}W^{(1)} \cdot H = \varphi(Z^{(1)}) \cdot Z^{(2)} = \tilde{H}W^{(2)} \cdot P = \mathrm{softmax}(Z^{(2)})$$

$$J = -(1/n) \sum_i \sum_k y_{ik} \ln p_{ik}$$

Ta đi **ngược** theo sơ đồ, từ J về $W^{(1)}$, bằng bốn bước và ba quy tắc ở mục 3.2.

Sai số ở lớp ra và gradient của $W^{(2)}$

Bước 1 · sai số ở lớp ra (chuỗi $J \to P \to Z^{(2)}$)

$$D^{(2)} = (1/n)(P - Y)$$

kích thước $n \times K$

Bước 2 · quy tắc 1 cho $Z^{(2)} = \tilde{H}W^{(2)}$

$$\nabla J \text{ theo } W^{(2)} = \tilde{H}^T D^{(2)}$$

kích thước $(m + 1) \times K$

Đây **đúng là công thức của Softmax Regression**, chỉ khác là đầu vào $\tilde{X}$ được thay bằng đặc trưng $\tilde{H}$ mà mạng **tự học**. Phần này ta không phải làm lại.

Softmax: $\nabla J = (1/n)\,\tilde{X}^T(P - Y)$

Lớp ra của MLP: $\nabla J = \tilde{H}^T \cdot (1/n)(P - Y)$

3.3 · BƯỚC 3: TRUYỀN SAI SỐ VỀ LỚP ẨN

Giải bài toán gán trách nhiệm

Quy tắc 2 cho $Z^{(2)} = \tilde{H}W^{(2)}$

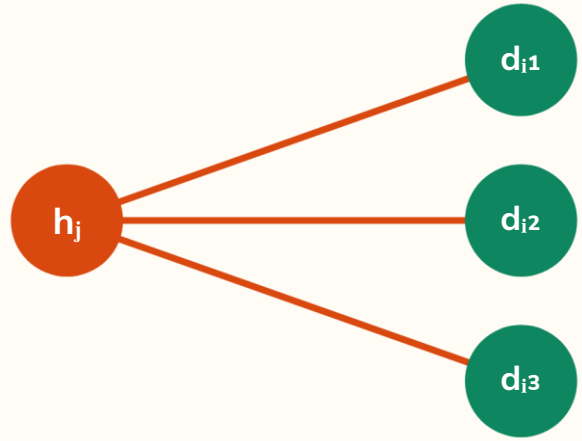
$$\partial J/\partial H = D^{(2)}\, V^{(2)T} \ (n \times m)$$

Đọc phần tử (i, j)

$$[D^{(2)}V^{(2)T}]_{ij} = \sum_k d^{(2)}_{ik} \cdot v^{(2)}_{jk}$$

Sai số của nơ-ron ẩn j = **tổng sai số các nơ-ron ra, mỗi cái nhân với trọng số nối từ j đến nơ-ron ra đó**.

Nơ-ron ẩn nối mạnh tới nơ-ron ra đang sai nhiều thì "chịu trách nhiệm" nhiều. Mạng dùng **chính $V^{(2)}$ theo chiều ngược** để phân bổ lỗi.



Sai số chảy ngược dọc các kết nối, mỗi sai số nhân với trọng số v_{jk}

3.3 · BƯỚC 4: QUA HÀM KÍCH HOẠT

Sai số lớp ẩn và gradient của $W^{(1)}$

Quy tắc 3, rồi quy tắc 1

$$D^{(1)} = (D^{(2)}V^{(2)T}) \odot \varphi'(Z^{(1)})$$

$$\nabla J \text{ theo } W^{(1)} = \tilde{X}^T D^{(1)}$$

kích thước $n \times m$ và $(d + 1) \times m$

φ	$\varphi'(Z^{(1)})$, tính từ H đã lưu
tanh	$1 - H \odot H$
sigmoid	$H \odot (1 - H)$
ReLU	1 nếu $Z^{(1)} > 0$, ngược lại 0

φ' là "cái cổng". Nơ-ron bão hòa ($\varphi' \approx 0$ với sigmoid, tanh khi $|z|$ lớn) hoặc đang tắt (ReLU với $z < 0$) thì sai số truyền về bị chặn, trọng số đi vào gần như không được cập nhật. Đây là mầm mống của **gradient biến mất** (mục 4.3).

3.3 · TỔNG HỢP

Bốn công thức cho mạng một lớp ẩn

$D^{(2)} = (1/n)(P - Y)$	$n \times K$
∇J theo $W^{(2)} = \tilde{H}^T D^{(2)}$	$(m + 1) \times K$
$D^{(1)} = (D^{(2)} V^{(2)T}) \odot \varphi'(Z^{(1)})$	$n \times m$
∇J theo $W^{(1)} = \tilde{X}^T D^{(1)}$	$(d + 1) \times m$

Cái cổng φ': nơ-ron bão hòa hoặc tắt thì nhận gần như không có gradient.

Bỏ lớp ẩn (H = X): chỉ còn bước 1 và 2, đúng công thức Softmax Regression.

3.3 · VÍ DỤ SỐ MẠNG 2–3–3 · BƯỚC 1 ĐẾN 3

Tính tay các ma trận sai số

Dùng lại mạng ở mục 2.4: tanh, n = 2, Y = [0 0 1; 1 0 0], đã biết J ≈ 0,2404.

Bước 1 · $D^{(2)} = \frac{1}{2}(P - Y)$

$$\begin{pmatrix} 0{,}0581 & 0{,}0958 & -0{,}1539 \\ -0{,}0534 & 0{,}0394 & 0{,}0140 \end{pmatrix}$$

Bước 2 · ∇J theo $W^{(2)} = \tilde{H}^{T} D^{(2)}$

$$\begin{pmatrix} 0{,}0047 & 0{,}1352 & -0{,}1399 \\ -0{,}0976 & -0{,}0336 & 0{,}1312 \\ 0{,}0515 & -0{,}0380 & -0{,}0135 \\ 0{,}0442 & 0{,}0729 & -0{,}1172 \end{pmatrix}$$

Bước 3 · $V^{(2)} = W^{(2)}$ bỏ hàng đầu

$$\begin{pmatrix} 1 & 0 & -1 \\ -1 & 1 & 0 \\ 0 & -1 & 1 \end{pmatrix}$$

Bước 3 · $D^{(2)} V^{(2)T}$ (truyền về lớp ẩn)

$$\begin{pmatrix} 0{,}2120 & 0{,}0377 & -0{,}2496 \\ -0{,}0674 & 0{,}0928 & -0{,}0254 \end{pmatrix}$$

Qua cổng φ' và gradient của $W^{(1)}$

$1 - H \odot H$ (tanh')

$$\begin{pmatrix} 0{,}4200 & 1 & 0{,}4200 \\ \mathbf{0{,}0013} & 0{,}0707 & 1 \end{pmatrix}$$

$D^{(1)} = (D^{(2)}V^{(2)T}) \odot \varphi'$

$$\begin{pmatrix} 0{,}0890 & 0{,}0377 & -0{,}1048 \\ \mathbf{-0{,}0001} & 0{,}0066 & -0{,}0254 \end{pmatrix}$$

∇J theo $W^{(1)} = \tilde{X}^T D^{(1)}$

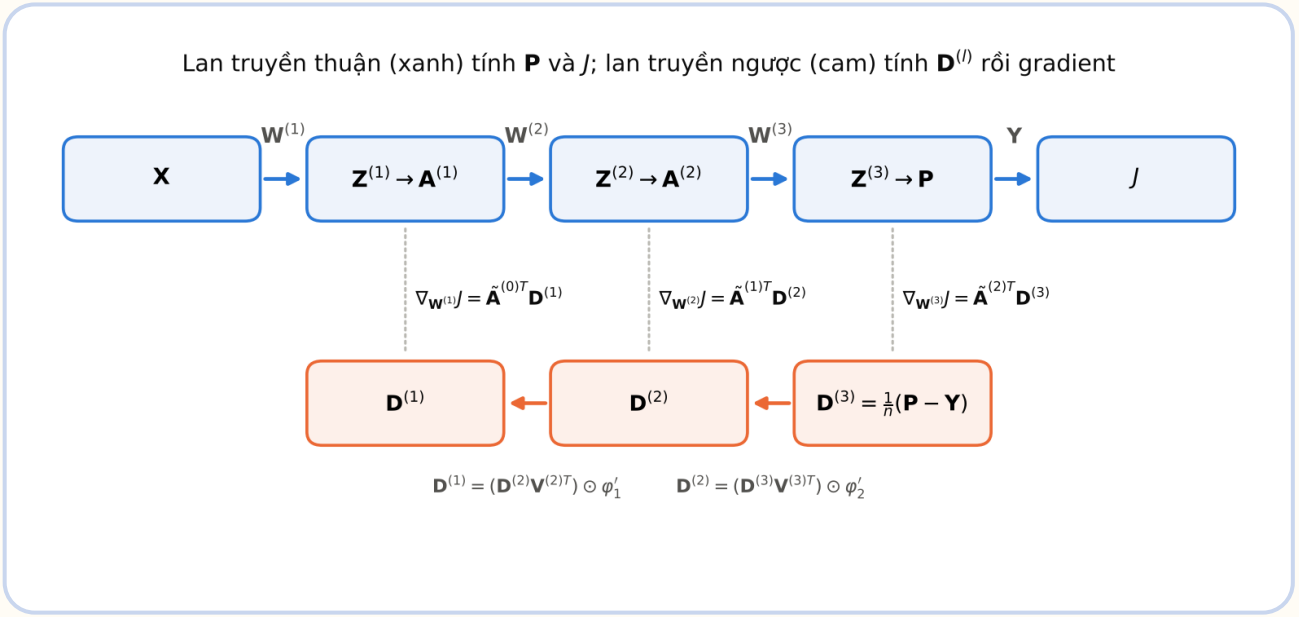
$$\begin{pmatrix} 0{,}0889 & 0{,}0442 & -0{,}1303 \\ 0{,}0887 & 0{,}0574 & -0{,}1811 \\ 0{,}1781 & 0{,}0688 & -0{,}1843 \end{pmatrix}$$

Cái cổng: mẫu 2, nơ-ron ẩn đầu có tanh = 0,9993 nên φ' = 0,0013. Sai số −0,0674 đi qua cổng chỉ còn −0,0001.

Kiểm tra số: $w^{(2)}_{31}$: đạo hàm số 0,0514955164, ngược 0,0514955165. Sai số tương đối lớn nhất trên 21 trọng số: $1{,}6 \times 10^{-9}$.

✓ Một bước Gradient Descent với η = 0,5 đưa J từ **0,2404** xuống **0,1562**.

Từ hai lớp lên L lớp: một công thức lặp



Lớp ra (softmax + cross-entropy)

$$D^{(L)} = (1/n)(P - Y)$$

Gradient, l = L, L−1, ..., 1

$$\nabla J \text{ theo } W^{(l)} = \tilde{A}^{(l-1)T} D^{(l)}$$

Truyền ngược, l = L, ..., 2

$$D^{(l-1)} = (D^{(l)}V^{(l)T}) \odot \varphi'(Z^{(l-1)})$$

Hình 11. Lan truyền thuận (xanh) tính $Z^{(l)}$, $A^{(l)}$ và J từ trái sang phải; lan truyền ngược (cam) đi từ $D^{(L)}$ về trái.

Thuật toán lan truyền ngược

1 · Lan truyền thuận

$A^{(0)} = X$; với $l = 1..L$ tính $Z^{(l)} = \tilde{A}^{(l-1)}W^{(l)}$, $A^{(l)} = \varphi_l(Z^{(l)})$. **Lưu lại** mọi A, Z. Tính J.

2 · Sai số ra

$D^{(L)} = (1/n)(P - Y)$.

3 · Lan truyền ngược

Với $l = L..1$: $\nabla J = \tilde{A}^{(l-1)T}D^{(l)}$; nếu $l > 1$: $D^{(l-1)} = (D^{(l)}V^{(l)T}) \odot \varphi'(Z^{(l-1)})$.

4 · Cập nhật

$W^{(l)} \leftarrow W^{(l)} - \eta\nabla J$, **sau khi** tính xong mọi gradient.

Lưu ý 1. Cập nhật $W^{(l)}$ ngay rồi mới truyền ngược thì $V^{(l)}$ đã đổi: sai số truyền đi là "nửa cũ nửa mới", không còn là gradient thật.

Lưu ý 2. Phải lưu ma trận trung gian, nên bộ nhớ tăng theo số lớp và kích thước mini-batch.

Chi phí: mỗi lớp có 2 phép nhân ma trận ở chiều ngược so với 1 ở chiều thuận, tổng cỡ **3 lượt thuận**, bất kể số tham số. Đạo hàm số: 2P lượt.

Đạo hàm số làm “giám khảo”

Sai phân trung tâm (chính xác hơn sai phân tiến: $O(\varepsilon^2)$ thay vì $O(\varepsilon)$):

$$g_{số} = [J(w + \varepsilon) - J(w - \varepsilon)] / (2\varepsilon)$$

$$\text{sai số tương đối} = |g_{số} - g_{ngược}| / \max(10^{-8}, |g_{số}| + |g_{ngược}|)$$

Sai số tương đối	Nhận định ($\varepsilon \approx 10^{-6}$, số thực 64 bit)
$< 10^{-6}$	Gần như chắc chắn đúng
10^{-6} đến 10^{-4}	Đáng ngờ, nên xem lại (có thể do ReLU gần điểm gãy)
$> 10^{-3}$	Gần như chắc chắn có lỗi

Mạng 3–5–4–3 (59 tham số, 6 mẫu ngẫu nhiên): sai số lớn nhất **$1{,}9 \times 10^{-7}$** với tanh và **$3{,}2 \times 10^{-7}$** với sigmoid, đều thuộc nhóm “gần như chắc chắn đúng”.

Lỗi thường gặp và lưu ý khi kiểm tra

Lỗi	Biểu hiện khi kiểm tra
Quên chia cho n ở $D^{(L)}$	Gradient lớn gấp n lần đạo hàm số
Nhầm V với W (quên bỏ hàng chệch)	Lỗi kích thước, hoặc gradient lớp trước sai
Quên nhân với φ'	Gradient các lớp ẩn sai, lớp ra vẫn đúng
Cập nhật W trước khi truyền ngược	Gradient các lớp trước sai
Quên gradient của hệ số chệch	Hàng đầu của gradient sai

Năm lưu ý khi kiểm tra

(i) mạng nhỏ, vài mẫu (cần 2P lượt thuận) · (ii) tắt yếu tố ngẫu nhiên như dropout (mục 4.4) · (iii) số thực 64 bit · (iv) tránh ReLU ngay tại $z = 0$ · (v) chỉ kiểm tra khi cài đặt hoặc sửa mã, **không** bật lúc huấn luyện thật.

Vì sao không thể khởi tạo W = 0?

Trọng số bằng nhau → **Cùng đầu ra** → **Cùng tín hiệu lỗi** → **Cùng gradient** → **Mãi giống nhau**

Hiện tượng **đối xứng** (symmetry): lớp ẩn m nơ-ron chỉ có sức biểu diễn của **đúng một** nơ-ron. Với W = 0 còn tệ hơn: $V^{(2)} = 0$ nên $D^{(1)} = 0$, lớp ẩn không nhận gradient nào.

Thí nghiệm XOR, mạng 2–3–1

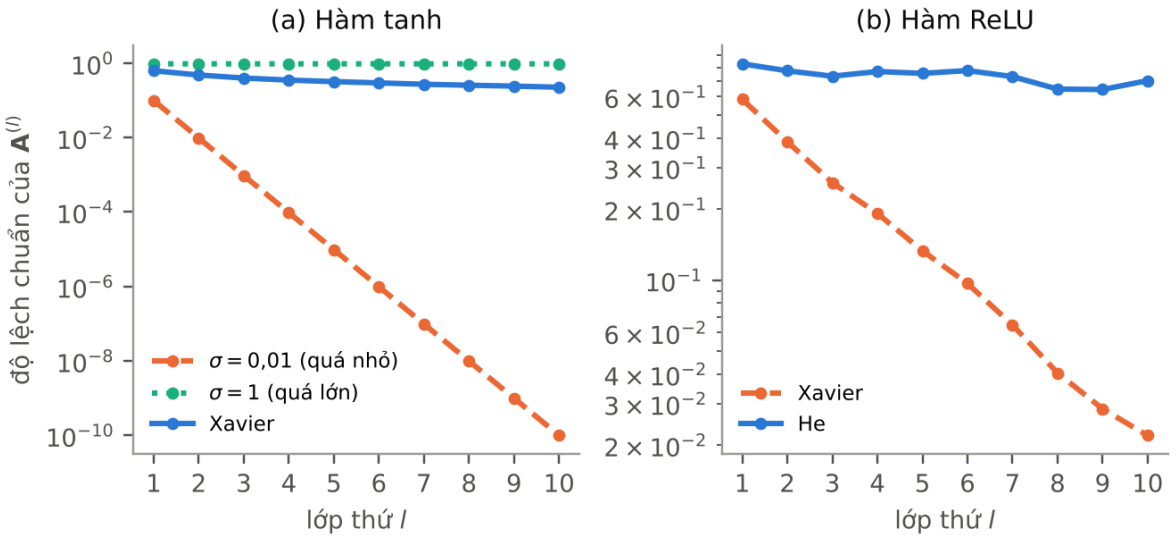
Mọi trọng số bằng 0,3. Sau 3000 epoch, J dừng ở **0,4832**, ba cột của $W^{(1)}$ vẫn giống hệt nhau: $[-0{,}5901\ 6{,}5197\ 6{,}5197]^T$.

Cách phá đối xứng

Khởi tạo ngẫu nhiên. Hệ số chệch có thể bằng 0 vì các trọng số ngẫu nhiên đã phá đối xứng.

Tín hiệu co lại hay bão hòa qua các lớp

Thử với mạng 10 lớp, mỗi lớp 100 nơ-ron, tanh, đầu vào N(0, 1); đo độ lệch chuẩn đầu ra từng lớp:



Quá nhỏ N(0; 0,01²): 0,099 ở lớp 1, ≈ 10^{-5} ở lớp 5, ≈ 10^{-10} ở lớp 10. Tín hiệu và gradient **co về 0**.

Quá lớn N(0; 1): ≈ 0,96 ở mọi lớp. Đầu ra gần ±1: **bão hòa**, tanh' ≈ 0.

Xavier: 0,625 → 0,319 → 0,226 (lớp 1, 5, 10). Tín hiệu giữ ở cỡ hợp lý.

Hình 12. Độ lệch chuẩn đầu ra qua 10 lớp (thang logarit). (a) tanh với ba cách khởi tạo. (b) ReLU: Xavier co dần, He giữ được độ lớn.

3.5 · KHỞI TẠO XAVIER VÀ HE

Giữ phương sai của tín hiệu không đổi

Với đầu vào độc lập, kỳ vọng 0: $z = \sum_i a_i w_{ij}$ có $\mathrm{Var}(z) = m_{(l-1)} \cdot \mathrm{Var}(w) \cdot \mathrm{Var}(a)$. Giữ phương sai ⇒ **$\mathrm{Var}(w) = 1/m_{(l-1)}$** (chiều ngược gợi ý $1/m_l$).

Xavier (Glorot)

Thỏa hiệp giữa hai chiều:

$$w_{ij} \sim N(0, 2/(m_{(l-1)} + m_l))$$

Cho tanh, sigmoid

He

ReLU đưa khoảng nửa số điểm số về 0, nên nhân đôi phương sai:

$$w_{ij} \sim N(0, 2/m_{(l-1)})$$

Cho ReLU và biến thể

Hình 12b: với ReLU, Xavier làm độ lệch chuẩn co từ **0,585 xuống 0,022** sau 10 lớp; He giữ ở khoảng **0,6 đến 1,0**.

η tốt nhất phụ thuộc thuật toán

Mạng 2–8–3 (tanh, mini-batch 32), 420 mẫu vòng tròn; J sau 100 epoch, trung bình 3 khởi tạo (ô trống: không thử):

η	SGD	Momentum (β = 0,9)	Adam
0,001	–	–	0,863
0,003	–	–	0,412
0,01	0,998	0,293	0,148
0,03	0,665	0,118	0,070
0,1	0,292	0,062	0,030
0,3	0,173	0,140	–
1	0,478	–	–

Vùng η tốt (chữ lục): SGD quanh 0,3; Momentum và Adam quanh 0,1. Hai phía đều tệ hơn.

Mặc định Adam η = 0,001 quá nhỏ khi chỉ chạy 100 epoch: “mặc định” chỉ là **điểm xuất phát**.

Thực hành: thử η trên lưới logarit (10^{-3}, $3\cdot10^{-3}$, 10^{-2}, …) và theo dõi đường cong J.

3.5 · MINI-BATCH VÀ SGD

Ước lượng gradient bằng một tập con

Mini-batch

Gradient là trung bình của n gradient từng mẫu, nên ước lượng bằng b mẫu: thay X, Y, n bằng X_B, Y_B, b trong thuật toán mục 3.4.

Epoch

Một lượt duyệt qua cả n mẫu, gồm [n/b] lần cập nhật. Ví dụ n = 420, b = 32: [13,1] = **14** lần cập nhật mỗi epoch.

SGD: đi ngược gradient G (của mini-batch) một đoạn η

$$W \leftarrow W - \eta G$$

Ngoài việc **nhanh hơn**, nhiễu của mini-batch còn giúp mạng **thoát khỏi một số vùng phẳng** của hàm mất mát.

3.5 · MOMENTUM

Giữ lại “vận tốc” như quả bóng lăn

M là tổng tích lũy có suy giảm của các gradient trước đó, β thường 0,9:

$$M \leftarrow \beta M + G,\ W \leftarrow W - \eta M$$

Gradient hằng G = 1, β = 0,9: vận tốc tăng dần về **G/(1 − β) = 10**.



Cùng hướng: vận tốc tích lũy, bước hiệu dụng lớn gấp 1/(1 − β) = 10 lần.

Đổi dấu liên tục (dao động ở thung lũng hẹp): các đóng góp triệt tiêu nhau.

3.5 · ADAM

Chuẩn hóa bước đi riêng cho từng tham số

$$M \leftarrow \beta_1 M + (1-\beta_1) G \cdot V \leftarrow \beta_2 V + (1-\beta_2)\, G \odot G$$

$$\hat{M} = M/(1-\beta_1^t) \cdot \hat{V} = V/(1-\beta_2^t)$$

$$W \leftarrow W - \eta\, \hat{M} / (\sqrt{\hat{V}} + \varepsilon)$$

t: số bước đã thực hiện; $\beta_1 = 0{,}9$; $\beta_2 = 0{,}999$; $\varepsilon = 10^{-8}$ (phép chia, căn, bình phương theo từng phần tử).

Hai trung bình trượt
M của gradient (hướng), V của **bình phương** gradient (độ lớn). Chia cho $\sqrt{\hat{V}}$ để chuẩn hóa từng tham số.

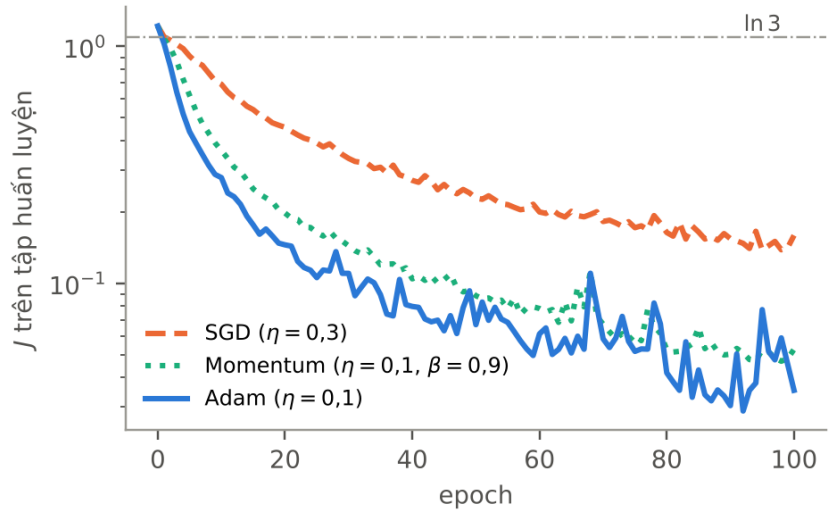
Hiệu chỉnh độ chệch
M, V khởi tạo bằng 0 nên các bước đầu bị kéo về 0; thừa số $1/(1-\beta^t)$ bù lại.

Ví dụ t = 1, g = 0,5
m = 0,05; v = 0,00025; m̂ = 0,5; v̂ = 0,25. Độ dời = η · 0,5/0,5 = **η**.

Tham số có gradient luôn nhỏ vẫn được bước đi cỡ **η**: một điểm khác hẳn SGD. Adam ít nhạy với thang đo gradient nên dễ dùng.

3.5 · SO SÁNH THỰC NGHIỆM

SGD, Momentum và Adam



Thuật toán	η	J cuối (TB ± độ lệch)	Độ chính xác kiểm tra
SGD	0,3	0,159 ± 0,046	94,8%
Momentum	0,1	0,053 ± 0,035	96,2%
Adam	0,1	0,035 ± 0,010	97,2%

Mạng 2–8–3 (tanh, softmax, mini-batch 32, 100 epoch), η tốt nhất của mỗi thuật toán, 5 khởi tạo khác nhau.

Momentum và Adam giảm J nhanh hơn SGD; hai đường này dao động nhiều hơn ở cuối vì η lớn.

Chênh lệch 1% đến 2% chỉ là 2 đến 4 trên 180 mẫu: **chưa kết luận chắc**. Không thuật toán nào luôn thắng.

Hình 13. J huấn luyện theo epoch (thang logarit, trung bình 5 khởi tạo), dữ liệu ba vòng tròn.

Mạng 2–2–1 cho XOR: dữ liệu và khởi tạo

Hai nơ-ron ẩn sigmoid, một nơ-ron ra sigmoid, cross-entropy nhị phân, n = 4, η = 0,5; làm tròn 4 chữ số thập phân.

$\tilde{X}$ (4 × 3)

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{pmatrix}$$

y (4 × 1)

$$\begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix}$$

$W^{(1)}$ (3 × 2)

$$\begin{pmatrix} 0{,}1 & -0{,}1 \\ 0{,}5 & 0{,}3 \\ -0{,}4 & 0{,}6 \end{pmatrix}$$

$W^{(2)}$ (3 × 1)

$$\begin{pmatrix} 0{,}2 \\ 0{,}7 \\ -0{,}6 \end{pmatrix}$$

Ta tính trọn một lượt: thuận → mất mát → ngược → cập nhật. Với bài toán nhị phân, P và Y chỉ có **một cột**, $D^{(2)} = (1/n)(p - y)$ vẫn đúng.

Từ $\tilde{X}$ đến J ≈ 0,7011

$Z^{(1)} = \tilde{X}W^{(1)}$

$$\begin{pmatrix} 0,1 & -0,1 \\ -0,3 & 0,5 \\ 0,6 & 0,2 \\ 0,2 & 0,8 \end{pmatrix}$$

$H = \sigma(Z^{(1)})$

$$\begin{pmatrix} 0,5250 & 0,4750 \\ 0,4256 & 0,6225 \\ 0,6457 & 0,5498 \\ 0,5498 & 0,6900 \end{pmatrix}$$

$z^{(2)} = \tilde{H}W^{(2)}$

$$\begin{pmatrix} 0,2825 \\ 0,1244 \\ 0,3221 \\ 0,1709 \end{pmatrix}$$

$p = \sigma(z^{(2)})$

$$\begin{pmatrix} 0,5702 \\ 0,5311 \\ 0,5798 \\ 0,5426 \end{pmatrix}$$

Mẫu	(x_1, x_2)	y	p	l = −ln q (q: xác suất của nhãn thật)
1	(0, 0)	0	0,5702	0,8443
2	(0, 1)	1	0,5311	0,6329
3	(1, 0)	1	0,5798	0,5450
4	(1, 1)	0	0,5426	0,7822

J = (0,8443 + 0,6329 + 0,5450 + 0,7822)/4 ≈ **0,7011**, chỉ cao hơn mốc ln 2 = 0,6931 của mô hình “chưa biết gì”.

Từ p − y về hai ma trận gradient

$D^{(2)} = \frac{1}{4}(p - y)$

$$\begin{pmatrix} 0{,}1425 \\ -0{,}1172 \\ -0{,}1050 \\ 0{,}1357 \end{pmatrix}$$

∇J theo $W^{(2)} = \tilde{H}^T D^{(2)}$

$$\begin{pmatrix} 0{,}0559 \\ 0{,}0317 \\ 0{,}0306 \end{pmatrix}$$

$D^{(2)}V^{(2)T}$, $V^{(2)} = [0{,}7; -0{,}6]$

$$\begin{pmatrix} 0{,}0998 & -0{,}0855 \\ -0{,}0821 & 0{,}0703 \\ -0{,}0735 & 0{,}0630 \\ 0{,}0950 & -0{,}0814 \end{pmatrix}$$

$H \odot (1 - H)$

$$\begin{pmatrix} 0{,}2494 & 0{,}2494 \\ 0{,}2445 & 0{,}2350 \\ 0{,}2288 & 0{,}2475 \\ 0{,}2475 & 0{,}2139 \end{pmatrix}$$

$D^{(1)} = (D^{(2)}V^{(2)T}) \odot \sigma'$

$$\begin{pmatrix} 0{,}0249 & -0{,}0213 \\ -0{,}0201 & 0{,}0165 \\ -0{,}0168 & 0{,}0156 \\ 0{,}0235 & -0{,}0174 \end{pmatrix}$$

∇J theo $W^{(1)} = \tilde{X}^T D^{(1)}$

$$\begin{pmatrix} 0{,}0115 & -0{,}0066 \\ 0{,}0067 & -0{,}0018 \\ 0{,}0034 & -0{,}0009 \end{pmatrix}$$

Kiểm tra nhanh: hàng đầu = tổng các cột của $D^{(1)}$ (0,0249 − 0,0201 − 0,0168 + 0,0235 = 0,0115). Hàng x_1 cộng mẫu 3, 4: 0,0067. Hàng x_2 cộng mẫu 2, 4: 0,0034.

3.6 · BƯỚC 4: CẬP NHẬT

W ← W − 0,5 ∇J: J giảm từ 0,7011 xuống 0,6987

$W^{(1)}$ mới

$$\begin{pmatrix} 0{,}0942 & -0{,}0967 \\ 0{,}4967 & 0{,}3009 \\ -0{,}4017 & 0{,}6004 \end{pmatrix}$$

$W^{(2)}$ mới

$$\begin{pmatrix} 0{,}1720 \\ 0{,}6841 \\ -0{,}6153 \end{pmatrix}$$

p mới

$$\begin{pmatrix} 0{,}5591 \\ 0{,}5196 \\ 0{,}5679 \\ 0{,}5303 \end{pmatrix}$$

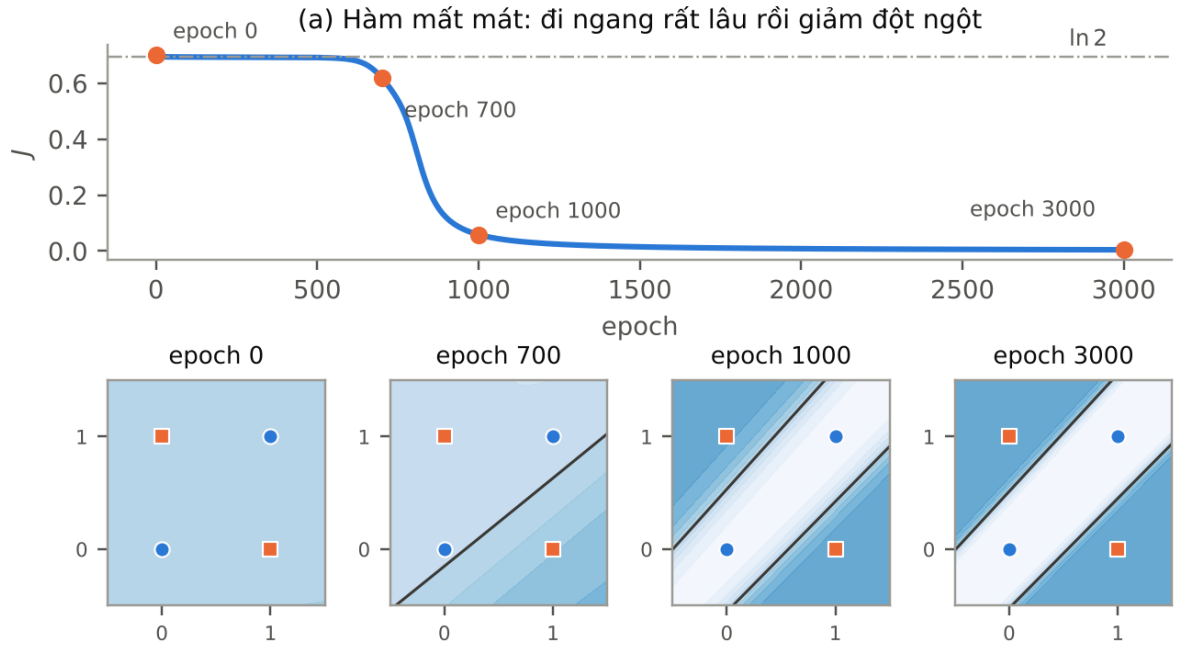
Chạy lại lan truyền thuận

J: 0,7011 → 0,6987

Có gì lạ? p của mẫu 2, 3 (nhãn 1) cũng giảm dù nhãn là 1. Không mâu thuẫn: bước đầu chủ yếu dịch cả bốn đầu ra cùng chiều (hệ số chệch lớp ra giảm).

Vì sao? Gradient quanh điểm khởi tạo rất nhỏ (cỡ 10^{-2}), chưa đủ để phân biệt các mẫu. Việc phân hóa chỉ xảy ra sau **nhiều bước**.

Một cao nguyên dài rồi giảm đột ngột



Epoch	J (η = 1)
0	0,7011
100	0,6935
500	0,6917
700	0,6194
800	0,3886
1000	0,0587
5000	0,0023

J đi ngang quanh **ln 2** khoảng 600 epoch đầu, rồi giảm rất nhanh: dấu hiệu của hàm mất mát **không lồi**.

Hình 14. (a) J đi ngang gần ln 2 rồi giảm đột ngột. (b) Xác suất đầu ra tại epoch 0, 700, 1000, 3000: ban đầu ≈ 0,5 khắp nơi, sau đó hình thành một dải ở giữa nhận lớp 1.

Lan truyền ngược chỉ khoảng mười dòng

```
def lan_truyen_nguoc(As, Zs, Y, Ws, kich_hoat):
  dphi = KICH_HOAT[kich_hoat][1]
  n, L = len(Y), len(Ws)
  grads = [None] * L
  D = (As[-1] - Y) / n # sai số lớp ra
  for l in range(L - 1, -1, -1):
    grads[l] = them_bias(As[l]).T @ D
    if l > 0:
      V = Ws[l][1:] # bỏ hàng chệch
      D = (D @ V.T) * dphi(As[l], Zs[l - 1])
  return grads
```

Đúng bằng công thức

$D^{(L)} = (P - Y)/n$; $\nabla W = \tilde{A}^T D$; $D^{(l-1)} = (D\,V^T) \odot \varphi'$. Chỉ khoảng 10 dòng.

Ba chỗ dễ sai

Quên chia n; quên bỏ hàng chệch khi lấy V; quên nhân φ'.

Kiểm tra gradient

Mạng 3–5–4–3 (59 tham số): sai số tương đối $1{,}9\times10^{-7}$ (tanh) và $3{,}2\times10^{-7}$ (sigmoid).

Chạy thử: XOR và ba vòng tròn

```
for epoch in range(so_epoch):
  idx = rng.permutation(n) # xáo trộn mẫu
  for k in range(0, n, b): # mỗi mini-batch
    lo = idx[k:k + b]
    As, Zs = lan_truyen_thuan(X[lo], Ws)
    grads = lan_truyen_nguoc(As, Zs, Y[lo], Ws)
    cap_nhat(Ws, grads, tt, thuat_toan, eta) # sau cùng
```

XOR, mạng 2–2–1

J cuối 0,0021; p = [0,002 0,998 0,998 0,002]; nhãn [0 1 1 0].

Ba vòng tròn, mạng 2–8–3, Adam

J: 1,122 → 0,0582 sau 300 epoch. Độ chính xác huấn luyện 100%, kiểm tra 96,7%.

So với Softmax

Softmax Regression chỉ đạt 35,0% trên cùng dữ liệu: mạng tự học biên cong.

J khởi đầu cỡ **ln 3 ≈ 1,0986** (chưa biết gì) rồi giảm đều. Năm khởi tạo khác nhau cho độ chính xác kiểm tra 96,1% đến 98,9%: **kết quả phụ thuộc khởi tạo**, điều không có ở Softmax.

TỔNG KẾT · LIÊN HỆ

Softmax Regression và MLP

Thành phần	Softmax Regression	Mạng nơ-ron (MLP)
Gradient	$(1/n)\,\tilde{X}^T(P - Y)$	$\nabla W^{(l)}J = \tilde{A}^{(l-1)T}D^{(l)}$, với $D^{(l)}$ truyền ngược
Khởi tạo	$W = 0$ được	Ngẫu nhiên (Xavier hoặc He), không được là 0
Hội tụ	Về cùng một J^* từ mọi điểm xuất phát	Phụ thuộc điểm xuất phát; có cao nguyên
Tối ưu	Gradient Descent là đủ	Mini-batch, thường kèm momentum hoặc Adam

Công thức **P – Y** vẫn nằm đúng ở đầu mọi chuỗi lan truyền ngược. Mục 4 sẽ hỏi “vì sao hoạt động”: điều gì xảy ra khi phân loại sai, vì sao có cao nguyên, và vì sao gradient biến mất hay bùng nổ khi mạng sâu hơn.

TÓM TẮT MỤC 3 · PHẦN 1/2

Từ ý tưởng đến thuật toán

1. Vì sao cần lan truyền ngược

Nơ-ron ẩn không có đáp án (bài toán gán trách nhiệm). Đạo hàm số tốn 2P lượt thuận (gấp khoảng 98 lần trên mạng 2–8–3); lan truyền ngược chỉ cỡ 3 lượt thuận, bất kể P.

2. Công cụ

Cộng theo mọi đường và ba quy tắc với $D = \partial J/\partial Z$: $\nabla W = \tilde{A}^T D$; $\partial J/\partial A = D\, V^T$; $\partial J/\partial Z = \partial J/\partial A \odot \varphi'(Z)$. Gradient luôn cùng kích thước với đại lượng của nó.

3. Mạng một lớp ẩn

$D^{(2)} = (P - Y)/n$; $\nabla W^{(2)} = \tilde{H}^T D^{(2)}$; $D^{(1)} = (D^{(2)} V^{(2)T}) \odot \varphi'(Z^{(1)})$; $\nabla W^{(1)} = \tilde{X}^T D^{(1)}$. Hàm φ' là cái cổng: nơ-ron bão hòa hoặc tắt gần như không nhận gradient.

4. Mạng L lớp

Lặp công thức từ lớp ra về lớp vào; lưu ma trận trung gian; cập nhật sau khi tính xong mọi gradient. Kiểm tra bằng sai phân trung tâm: sai số tương đối dưới 10^{-6} là đúng.

TÓM TẮT MỤC 3 · PHẦN 2/2

Khởi tạo, tối ưu và ví dụ XOR

5. Khởi tạo

Không được là 0 (đối xứng: các nơ-ron ẩn giống hệt nhau mãi mãi). Ngẫu nhiên với độ lớn hợp lý: Xavier $\mathrm{Var}(w) = 2/(m_{(l-1)} + m_l)$ cho tanh, sigmoid; He $\mathrm{Var}(w) = 2/m_{(l-1)}$ cho ReLU.

6. Tối ưu

η tốt nhất phụ thuộc thuật toán. Momentum $M \leftarrow \beta M + G$ tích lũy hướng nhất quán; Adam chuẩn hóa bước theo từng tham số, có hiệu chỉnh độ chệch. Momentum và Adam giảm J nhanh hơn SGD.

7. Ví dụ XOR 2–2–1

Một bước tính tay giảm J từ 0,7011 xuống 0,6987. Huấn luyện đầy đủ: cao nguyên gần ln 2 rồi giảm đột ngột (hàm mất mát không lồi). Mạng 2–8–3 đạt 96% đến 99% nhờ mã chỉ vài chục dòng.

[icon: Arrow-right] **Mục 4:** vì sao hoạt động? Cao nguyên, tối ưu không lồi, gradient biến mất hay bùng nổ, quá khớp.

KIỂM TRA NHANH · CÂU 1 ĐẾN 6

Bạn đã nắm được chưa?

1. Với $l = -\ln \sigma(wx + b)$, $w = 1$, $x = 1$, $b = 0$, $y = 1$: tính $\partial l/\partial s$, $\partial l/\partial w$, $\partial l/\partial b$ bằng đồ thị tính toán.

2. Mạng 4–6–3 (tanh, softmax), mini-batch $n = 10$. Cho kích thước của $Z^{(1)}$, H, $D^{(2)}$, $D^{(1)}$, $\nabla W^{(1)} J$ và $\nabla W^{(2)} J$.

3. Chứng minh $\nabla W J = \tilde{A}^T D$ cho $Z = \tilde{A} W$ bằng cách viết theo từng phần tử w_{kj}.

4. Vì sao phải tính xong gradient của tất cả các lớp rồi mới cập nhật? Sai gì nếu cập nhật $W^{(2)}$ ngay rồi mới truyền ngược về lớp 1?

5. $n = 1$, $K = 2$, $D^{(2)} = [0{,}2\ -0{,}2]$, $V^{(2)} = [[1, 2], [-1, 0]]$, $H = [0{,}5\ -0{,}5]$ (tanh). Tính $D^{(1)}$.

6. $g_{số} = 0{,}05149552$ và $g_{ngược} = 0{,}05149551$: tính sai số tương đối. Nếu lan truyền ngược cho 0,0258 thì kết luận gì?

KIỂM TRA NHANH · CÂU 7 ĐẾN 11

Khởi tạo, tối ưu và suy luận

7 Tính độ lệch chuẩn Xavier cho lớp 100 vào, 50 ra, và độ lệch chuẩn He cho lớp ReLU 100 vào.

8 Momentum $\beta = 0{,}9$, $M_0 = 0$, gradient hằng $g = 1$: tính M_1, M_2, M_3 và giới hạn. Nói gì về bước đi hiệu dụng?

9 Adam, bước $t = 1$, gradient $g = 0{,}5$: tính m, v, $\hat{m}$, $\hat{v}$ và độ dời của tham số (lấy $\varepsilon \approx 0$).

10 Vì sao $W = 0$ dùng được cho Softmax Regression nhưng không dùng được cho mạng nhiều lớp?

11 Ở Hình 14, vì sao J đi ngang quanh ln 2 rất lâu ở đầu quá trình huấn luyện?

ĐÁP ÁN · CÂU 1 ĐẾN 4

Lời giải ngắn gọn

1 $a = \sigma(1) \approx 0{,}7311$ nên $\partial l/\partial s = a - 1 \approx$ **−0,2689**. Vì $x = 1$: $\partial l/\partial w = -0{,}2689 \times 1 =$ **−0,2689**, $\partial l/\partial b =$ **−0,2689**.

2 $Z^{(1)}$ và H: **10×6**. $D^{(2)}$: **10×3**. $D^{(1)}$: **10×6**. $\nabla W^{(1)} J$: **5×6**. $\nabla W^{(2)} J$: **7×3**.

3 $z_{ij} = \sum_k \tilde{a}_{ik} w_{kj}$. w_{kj} chỉ có trong cột j của Z nhưng ở cả n hàng, nên $\partial J/\partial w_{kj} = \sum_i d_{ij} \tilde{a}_{ik}$, đúng phần tử (k, j) của $\tilde{A}^T D$.

4 $D^{(1)}$ dùng $V^{(2)}$ **tại điểm đang xét**. Nếu $W^{(2)}$ đã đổi thì sai số truyền về là của mạng "nửa cũ nửa mới", không phải gradient của J tại điểm hiện tại.

ĐÁP ÁN · CÂU 5 ĐẾN 8

Lời giải ngắn gọn

5 $D^{(2)}V^{(2)T} = [0{,}2\cdot 1 + (-0{,}2)\cdot 2,\ 0{,}2\cdot(-1) + 0] = [-0{,}2\ -0{,}2]$; $\varphi' = 1 - H \odot H = [0{,}75\ 0{,}75]$; vậy $D^{(1)}$ = **[–0,15 –0,15]**.

6 Sai số tương đối ≈ $10^{-8}/0{,}103 \approx$ **10^{-7}**: gần như chắc chắn đúng. Với 0,0258: ≈ 0,33, **rất lớn**, gần như chắc chắn có lỗi (có thể chia thừa hệ số 2, ví dụ chia n hai lần).

7 Xavier: $\sqrt{(2/150)} \approx$ **0,1155**. He: $\sqrt{(2/100)} \approx$ **0,1414**.

8 $M_1 = 1$; $M_2 = 0{,}9\cdot 1 + 1 =$ **1,9**; $M_3 = 0{,}9\cdot 1{,}9 + 1 =$ **2,71**; giới hạn $1/(1 - 0{,}9) =$ **10**: bước đi hiệu dụng ổn định gấp 10 lần SGD cùng η.

ĐÁP ÁN · CÂU 9 ĐẾN 11

Lời giải ngắn gọn

9 $m = 0{,}1 \cdot 0{,}5 =$ **0,05**; $v = 0{,}001 \cdot 0{,}25 =$ **0,00025**; $\hat{m} = 0{,}05/0{,}1 =$ **0,5**; $\hat{v} = 0{,}00025/0{,}001 =$ **0,25**; độ dời $= \eta \cdot 0{,}5/0{,}5 =$ **η** theo dấu của gradient.

10 Softmax có một lớp có tham số, hàm mất mát lồi, và cột k của W nhận gradient riêng nhờ cột k của Y nên các cột phân hóa dù khởi tạo 0. Ở mạng nhiều lớp, nơ-ron ẩn cùng lớp có cùng đầu vào nên nếu cùng trọng số thì mãi giống nhau (đối xứng); với $W = 0$ còn có $V^{(2)} = 0$ nên $D^{(1)} = 0$.

11 Trọng số khởi tạo nhỏ nên mọi nơ-ron ẩn gần như không phân biệt được các mẫu, $p \approx 0{,}5$ cho mọi mẫu. Đó là vùng phẳng của hàm mất mát không lồi: gradient rất nhỏ nên trọng số đổi chậm. Khi trọng số lệch dần khỏi gần-đối-xứng, gradient tăng và J giảm đột ngột.

CHƯƠNG: MẠNG NƠ-RON NHÂN TẠO · MỤC 4

Phân tích cơ chế học của mạng nơ-ron

Khi nào và vì sao lan truyền ngược hoạt động, hoặc thất bại



KẾ HOẠCH BÀI HỌC

Chạy được chưa có nghĩa là hiểu

Mục 3 cho ta **thuật toán huấn luyện**. Mục 4 giải thích **khi nào và vì sao** nó hoạt động hoặc thất bại. Năm câu hỏi:

4.1 · Mẫu sai

Khi phân loại sai một mẫu, tín hiệu lỗi đi qua các lớp thế nào?

4.2 · Không lồi

Vì sao J đi ngang rất lâu rồi mới giảm đột ngột?

4.3 · Mạng sâu

Vì sao mạng sâu hơn có thể không học được gì?

4.4 · Quá khớp

Mạng khớp hoàn hảo tập huấn luyện thì dữ liệu mới ra sao?

4.5 · Rộng hay sâu

Nên thêm nơ-ron vào mỗi lớp hay thêm lớp mới?

Cách làm

Đọc hiện tượng từ công thức, rồi kiểm chứng bằng thí nghiệm.

KÝ HIỆU

Ký hiệu bổ sung cho mục 4

Đối tượng	Ký hiệu	Ý nghĩa
Tín hiệu lỗi riêng mẫu	$d_i^{(l)}$	Hàng i của $n \cdot D^{(l)}$ (vector hàng $1 \times m_l$), không chia n
Ma trận Hessian	H_J	Các đạo hàm riêng cấp hai của J theo mọi tham số
Hệ số điều chuẩn hóa	λ	Hệ số của số hạng phạt trọng số (L2)
Xác suất bỏ nơ-ron	p	Xác suất một nơ-ron ẩn bị tắt trong dropout
Mặt nạ dropout	$M^{(l)}$	Cùng cỡ $A^{(l)}$, phần tử bằng 0 hoặc $1/(1 - p)$

Quy ước cũ vẫn giữ: **ma trận chữ hoa in đậm, vector chữ thường in đậm.** Mục 4 thêm "lời phàn nàn" của từng mẫu d_i và các công cụ phân tích (Hessian) cùng điều chuẩn hóa (λ, p).

ĐIỂM XUẤT PHÁT

Ba công thức lan truyền ngược

Lớp ra	$D^{(L)} = (1/n)(P - Y)$
Gradient	$\nabla W^{(l)} J = \tilde{A}^{(l-1)T} D^{(l)}$
Truyền ngược	$D^{(l-1)} = (D^{(l)} V^{(l)T}) \odot \varphi'(Z^{(l-1)})$

Mọi hiện tượng của mục 4, từ **cao nguyên**, **bão hòa** đến **gradient biến mất**, đều đọc ra được từ ba công thức này.

4.1 · MẪU BỊ PHÂN LOẠI SAI

Gradient là trung bình các “lời phàn nàn”

$$\nabla W^{(l)} J = (1/n) \sum_i \tilde{a}_i^{(l-1)T} d_i^{(l)}$$

$$d_i^{(L)} = p_i - y_i;\ d_i^{(l-1)} = (d_i^{(l)} V^{(l)T}) \odot \varphi'(z_i^{(l-1)})$$

J là trung bình

J = (1/n) ∑ l_i, nên gradient cũng là trung bình của gradient từng mẫu.

Mỗi mẫu một lực kéo

Mỗi mẫu đóng góp một ma trận hạng một: cột đầu vào $\tilde{a}_i^T$ nhân hàng “lời phàn nàn” d_i.

Cập nhật = trung bình lực kéo

Mỗi mẫu kéo W về phía làm giảm mất mát của riêng nó.

Câu hỏi: lực kéo của một mẫu **sai** khác lực kéo của một mẫu **đúng** thế nào?

4.1 · LỚP RA

Mẫu sai kéo mạnh, mẫu đúng gần như không kéo

Mẫu đúng và tự tin

p = (0,98; 0,01; 0,01), y = (1, 0, 0)

d = p − y = (−0,02; 0,01; 0,01)

‖d‖ ≈ 0,024

Mẫu bị đoán sai

p = (0,70; 0,20; 0,10), y = (0, 1, 0)

d = p − y = (0,70; −0,80; 0,10)

‖d‖ ≈ 1,068

≈ 44 lần Mẫu sai kéo trọng số mạnh gấp khoảng 44 lần mẫu đúng và tự tin.

Dấu dương (0,70) ở lớp bị đoán nhầm: điểm số lớp này phải **giảm**.

Dấu âm (−0,80) ở lớp thật: điểm số lớp này phải **tăng**.

4.1 · VÍ DỤ MẠNG 2–3–3

Lỗi lan ngược qua mạng: ba bước

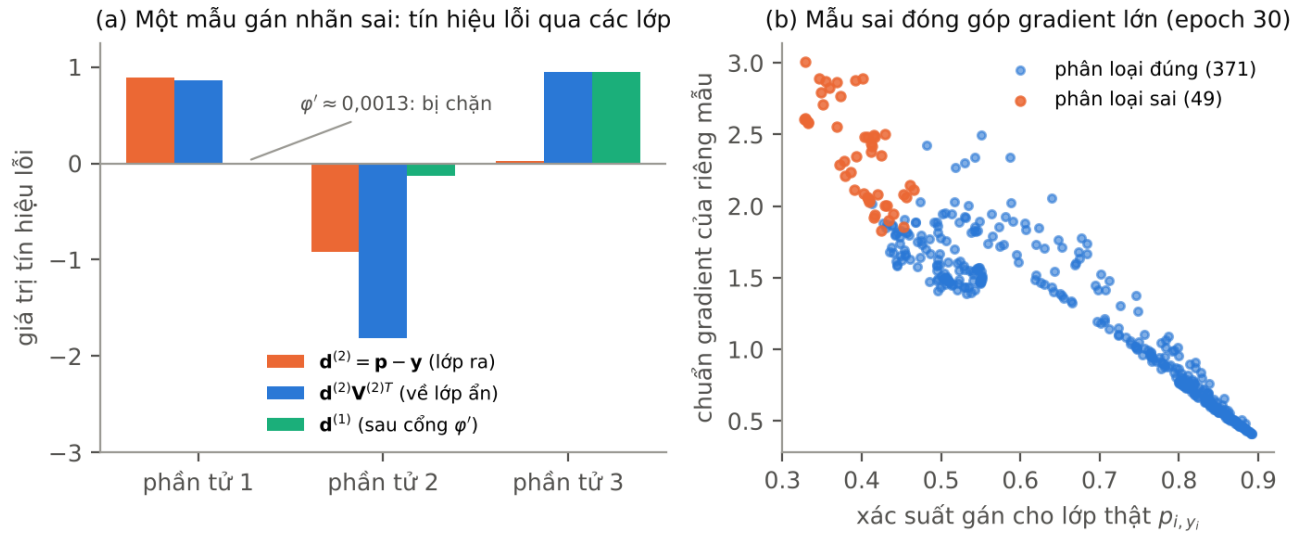
Mẫu thứ hai của mục 3.3: mạng đoán lớp 0 với xác suất 0,8932, nhưng giả sử nhãn thật là lớp 1. Ta có $p_2 = (0{,}8932;\ 0{,}0788;\ 0{,}0280)$, $a_2^{(1)} = (0{,}9993;\ -0{,}9640;\ 0)$.

1 · Lớp ra	$d_2^{(2)} = p_2 - y_2 = [0{,}8932\ -0{,}9212\ 0{,}0280]$
2 · Về lớp ẩn	$d_2^{(2)} V^{(2)T} = [0{,}8652\ -1{,}8143\ 0{,}9492]$
3 · Qua cổng φ'	$\varphi' = 1 - a \odot a = (0{,}0013;\ 0{,}0707;\ 1{,}0000)$
Kết quả	$d_2^{(1)} = [0{,}0012\ \mathbf{-0{,}1282\ 0{,}9492}]$

$V^{(2)} = [[1, 0, -1], [-1, 1, 0], [0, -1, 1]]$ (mục 3.3). Chỉ nơ-ron thứ ba (tanh = 0, vùng nhạy) nhận trọn tín hiệu.

4.1 · HÌNH 15

Nhìn lỗi lan ngược và gradient theo mẫu



Phân bổ theo trọng số nối. Nơ-ron ẩn thứ hai: 0,8932·(−1) + (−0,9212)·1 = −1,8143, hai đóng góp cùng dấu nên cộng dồn.

Cổng φ' chặn bão hòa. 0,8652 → 0,0012 (tanh = 0,9993); −1,8143 → −0,1282 (tanh = −0,9640).

Dấu đổi qua mỗi lớp, tùy nơ-ron nối tới lớp ra bằng trọng số dương hay âm.

Hình 15. (a) Tín hiệu lỗi của mẫu thứ hai ở mạng 2–3–3: tại lớp ra, sau khi truyền về lớp ẩn, và sau cổng φ'. (b) Mạng 2–8–3 sau 30 epoch Adam: mỗi chấm là một mẫu, trục tung là chuẩn gradient riêng của mẫu.

4.1 · VÍ DỤ MẠNG 2–3–3

Gradient riêng của mẫu sai và một bước cập nhật

ãᵀ d (theo W⁽²⁾), cột = lớp 0, 1, 2

0,8932	**−0,9212**	0,0280
0,8926	**−0,9205**	0,0280
−0,8610	**0,8880**	−0,0270
0	0	0

Hàng cuối bằng 0: nơ-ron ẩn thứ ba có đầu ra 0 nên không góp vào điểm số nào.

Chuẩn gradient	Lớp 2	Lớp 1
Mẫu 2 (sai)	2,196	3,177
Mẫu 1 (đúng)	0,559	0,699

Chỉ dùng mẫu này cập nhật một bước (η = 0,5): xác suất lớp thật tăng **0,0788 → 0,7863**; mất mát mẫu giảm **2,540 → 0,240**.

Cột lớp 0 (đang đoán quá cao) có gradient dương: bước W ← W − η∇ **hạ** điểm số lớp 0; cột lớp thật bị đẩy **ngược lại**.

4.1 · TRÊN DỮ LIỆU THẬT

Mẫu sai chi phối gradient

Mạng 2–8–3, dữ liệu ba vòng tròn (420 mẫu), huấn luyện bằng Adam:

Epoch 30

49 / 420

mẫu bị sai (11,7%)

Chuẩn gradient riêng

2,36 so với 1,15

mẫu sai gấp đôi mẫu đúng

Phần của nhóm sai

21%

tổng chuẩn gradient, gần gấp đôi tỉ lệ số mẫu

Epoch 2

238 / 420

mẫu sai, chiếm 63% tổng

Hệ quả 1 · Nhiễu kéo rất mạnh

Mạng không biết nhãn sai hay đúng nên cũng chỉnh trọng số để khớp mẫu nhiễu. Mạng càng lớn càng làm được: gốc của quá khớp (mục 4.4).

Hệ quả 2 · Cross-entropy có lý do

So với mất mát bình phương, cross-entropy giữ gradient lớn khi mạng sai nặng (slide kế tiếp).

4.1 · VÌ SAO CROSS-ENTROPY

Bình phương sai số sửa chậm đúng lúc cần nhanh

Nơ-ron ra sigmoid, nhãn y = 1, mạng đoán sai nặng: a = 0,01.

Hàm mất mát	∂l/∂s	Nhận xét
Cross-entropy	a − y = −0,99	Lỗi nặng, kéo mạnh
Bình phương	(a − y)·a(1 − a) = −0,0098	Nhỏ hơn 101 lần: nơ-ron bão hòa ở sai chỗ nên gần như "đứng yên"

Cross-entropy: thừa số a(1 − a) của sigmoid triệt tiêu mẫu số của logarit, chỉ còn **a − y**.

Bình phương: còn nguyên thừa số a(1 − a), rất nhỏ khi a gần 0 hoặc 1.

✓ Phạt đúng nhưng sửa chậm chính khi lỗi lớn nhất. Đó là một lý do bài toán phân loại dùng **cross-entropy** (mục 2.4).

Chứng minh J của mạng không lồi

Softmax: J lồi

Mọi điểm dừng là cực tiểu toàn cục; kết quả không phụ thuộc điểm xuất phát.

Hàm lồi thỏa mãn

$J(\frac{1}{2}\theta A + \frac{1}{2}\theta B) \le \frac{1}{2}J(\theta A) + \frac{1}{2}J(\theta B)$ với mọi cặp θA, θB.

Ý tưởng: tạo hai bộ tham số có **cùng** mất mát nhờ **đối xứng hoán vị**: đổi chỗ hai nơ-ron ẩn (hai cột của $W^{(1)}$, hai hàng của $V^{(2)}$) thì $\sum_j \varphi(z_j)v_j$ không đổi, hàm của mạng y hệt.

θA (mạng 2–8–3 đã học)

J = 0,0582

[icon: Arrow-right]

Điểm giữa ½θA + ½θB

J = 6,85

[icon: Arrow-right]

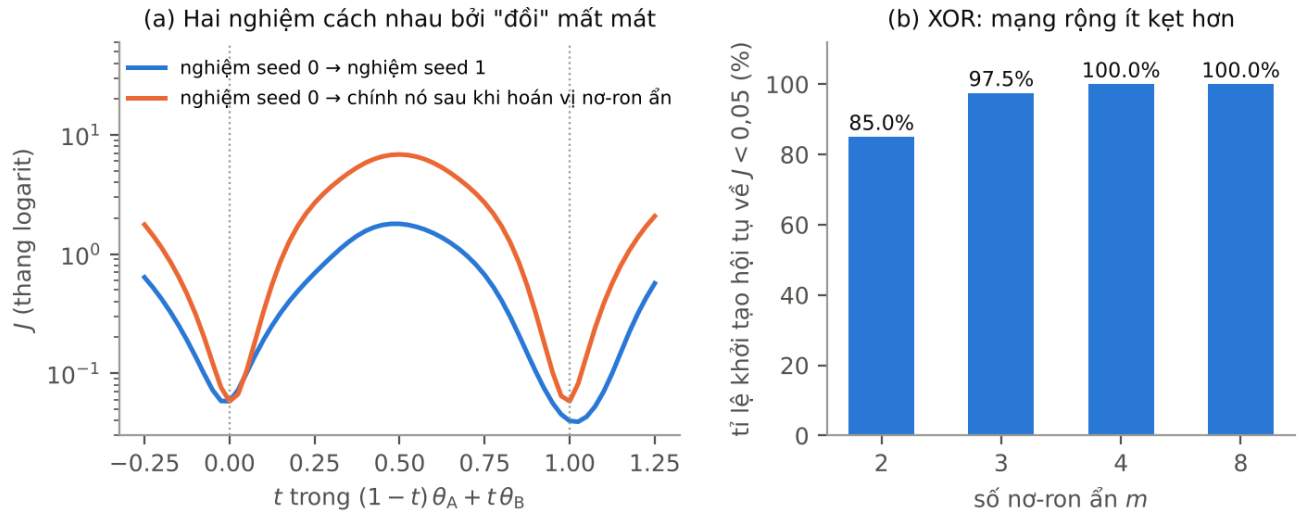
θB (hoán vị 8 nơ-ron ẩn)

J = 0,0582

⚠ Hàm lồi buộc J(điểm giữa) ≤ 0,0582, nhưng thực tế là 6,85. Vậy **J của mạng nơ-ron không lồi**.

4.2 · HÌNH 16

Nhiều nghiệm “giống nhau” và nghiệm khác nhau



Đếm bộ tham số tương đương. Một lớp ẩn m nơ-ron có m! hoán vị; với tanh nhân thêm 2^m cách đổi dấu (tanh là hàm lẻ).

Với m = 8: 8! = 40 320, hoặc $2^8 \cdot 8!$ = 10 321 920 bộ tham số cho cùng một hàm.

Không đáng lo: các cực tiểu “giống nhau” cho cùng chất lượng. Đáng lo là các cực tiểu **thật sự khác nhau**.

Hình 16. (a) J dọc theo đoạn thẳng nối hai nghiệm: giữa hai khởi tạo khác nhau (xanh) và giữa một nghiệm với chính nó sau khi hoán vị (cam), J đều tăng cao ở giữa. (b) Tỉ lệ 40 khởi tạo hội tụ về J < 0,05 trên XOR theo số nơ-ron ẩn (η = 1, 5000 epoch).

Cực tiểu, cực đại và điểm yên ngựa

$J(\theta + \delta) \approx J(\theta) + \frac{1}{2}\,\delta^T H_J\,\delta$ (tại $\nabla J \approx 0$)

Theo hướng riêng thứ k của H_J: $\delta^T H_J\,\delta = \mu_k \|\delta\|^2$, với μ_k là giá trị riêng.

Giá trị riêng của H_J	Điểm dừng	Gradient Descent
Tất cả > 0	Cực tiểu địa phương	Bị giữ lại
Cả dương lẫn âm	Điểm yên ngựa	Đi ra theo hướng $\mu_k < 0$, nhưng chậm nếu $\|\mu_k\|$ nhỏ
Tất cả < 0	Cực đại địa phương	Rời đi ngay
Có giá trị ≈ 0	Vùng phẳng	Di chuyển rất chậm

Số tham số P rất lớn: cực tiểu cần **cả P** giá trị riêng dương, nên điểm dừng "ngẫu nhiên" hầu hết là **điểm yên ngựa** (Dauphin và cộng sự 2014).

4.2 · QUAN SÁT TRÊN XOR

Mạng càng rộng càng ít bị kẹt

40 khởi tạo ngẫu nhiên (Xavier, sigmoid, $\eta = 1$, 5000 epoch), số nơ-ron ẩn m thay đổi:

Số nơ-ron ẩn m	2	3	4	8
Số lần hội tụ về $J < 0{,}05$ (trên 40)	34	39	40	40

m = 2: 6 lần kẹt, cả sáu ở J ≈ 0,348

Một lần kẹt (seed 6): p = (0,002; 0,499; 0,999; 0,500). Đúng mẫu 1 và 3; gán ≈ 0,5 cho mẫu 2 và 4.

$\|\nabla J\|$ chỉ còn $5{,}7\times10^{-4}$.

Hessian 9×9 tại điểm kẹt

8 giá trị riêng trong [0; 0,003] và một giá trị 0,25; **không có giá trị riêng âm đáng kể**.

Thung lũng gần như phẳng: gần một cực tiểu địa phương "xấu"; sigmoid ở đó đã bão hòa.

⚠ Kết quả huấn luyện **phụ thuộc điểm xuất phát**; mạng rộng hơn ít kẹt hơn (mục 4.5).

4.2 · QUAY LẠI HÌNH 14

Cao nguyên là vùng điểm yên ngựa

Hessian 9×9 của mạng XOR 2–2–1, từ đúng điểm khởi tạo ở mục 3.6 (J = 0,7011):

Epoch	J	$\|\nabla J\|$	Giá trị riêng nhỏ nhất	Số giá trị riêng < -10^{-4}
0	0,7011	0,0728	−0,0203	4
300	0,6930	0,0015	−0,0051	3
600	0,6850	0,0133	−0,0128	3

Gradient nhỏ (0,0015 ở epoch 300) nhưng Hessian **luôn có giá trị riêng âm**: đây là yên ngựa, có hướng thoát.

Độ cong theo hướng thoát rất nhỏ (−0,005 đến −0,02), nên GD đi chậm, rồi tăng tốc và giảm đột ngột.

Giải thích vì sao ở mục 3.6, η = 0,5 cần khoảng **gấp đôi** số epoch của η = 1 để rơi khỏi cao nguyên.

4.2 · VAI TRÒ CỦA KHỞI TẠO

Hai nghiệm tốt vẫn khác nhau

Khởi tạo seed 0

$J = 0{,}0582$

[icon: Arrow-right]

Điểm giữa hai nghiệm

$J = 1{,}79$

[icon: Arrow-right]

Khởi tạo seed 1

$J = 0{,}0398$

Mạng 2–8–3, 300 epoch Adam. Độ chính xác kiểm tra cùng cỡ 96% đến 98%, nhưng hai nghiệm nằm ở **hai thung lũng khác nhau**, ngăn cách bởi một sườn đồi mất mát (đường xanh ở Hình 16a).

1 · Khởi tạo có kiểm soát

Xavier hoặc He (mục 3.5) để bắt đầu ở vùng nhạy, không bão hòa.

2 · Chạy nhiều khởi tạo

Báo cáo trung bình và độ lệch chuẩn, như ở mục 3.5 và 3.7.

3 · Mạng đủ rộng

Cộng thuật toán có quán tính (momentum, Adam) và nhiễu mini-batch để vượt yên ngựa, vùng phẳng.

4.3 · PHÁ ĐỐI XỨNG

Đối xứng là điểm yên ngựa được bảo toàn

Nếu hai nơ-ron j, k giống nhau

Cột j, k của $W^{(1)}$ bằng nhau và hàng j, k của $V^{(2)}$ bằng nhau ⇒ $h_j = h_k$ với mọi mẫu.

Tín hiệu lỗi cũng bằng nhau

$d_j^{(1)} = d_k^{(1)}$ ⇒ hai cột của $\nabla W^{(1)}$ bằng nhau ⇒ sau cập nhật **vẫn bằng nhau**, mãi mãi.

Đó là yên ngựa: tại điểm kẹt ở mục 3.5 (J = 0,4832, mạng 2–3–1, mọi trọng số 0,3) Hessian có **hai giá trị riêng âm** (≈ -2×10^{-4}), ứng với hướng làm hai nơ-ron lệch nhau.

Thêm nhiễu Gauss độ lệch chuẩn ε vào trọng số 0,3; số epoch để J < 0,1 (trung vị 5 lần chạy, η = 1):

ε	0	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-6}
Số epoch	không thoát	780	1020	1180	3040 đến 4290 (thoát 3/5 lần)	không thoát sau 40 000

Nhiễu càng nhỏ, thoát càng chậm. Vì vậy chọn khởi tạo ngẫu nhiên cỡ **Xavier hoặc He**, không phải ε rất nhỏ.

4.3 · BÃO HÒA

Hàm kích hoạt “tắt”: khi φ' ≈ 0

Công thức $D^{(l-1)} = (D^{(l)}V^{(l)T}) \odot \varphi'$ có thừa số φ' ở mỗi lớp ẩn. Khi |z| lớn, sigmoid và tanh bão hòa:

z	0	2	5	10
σ'(z)	0,25	0,105	0,0066	$4{,}5\times10^{-5}$
tanh'(z)	1	0,0707	$1{,}8\times10^{-4}$	$8{,}2\times10^{-9}$

Sigmoid, tanh

Nơ-ron bão hòa không nhận tín hiệu lỗi (0,8652 thành 0,0012 ở mục 4.1): trọng số đi vào gần như không học nữa. Khởi tạo quá lớn làm tanh bám sát ±1 ở mọi lớp.

ReLU: φ' ∈ {0, 1}

Không bão hòa ở phía dương. Nhưng nơ-ron có z < 0 với **mọi** mẫu thì gradient luôn 0: nơ-ron **“chết”**, không hồi sinh. η quá lớn hoặc khởi tạo xấu làm việc này hay xảy ra.

Leaky ReLU $\varphi(z) = \max(\alpha z, z)$, α nhỏ như 0,01: giữ lại chút gradient ở phía âm để tránh nơ-ron chết.

Tín hiệu lỗi bị nhân qua L – 1 lớp

$$\|d^{(1)}\| \le \left(\gamma \prod \|V^{(l)}\|_2 \right) \|d^{(L)}\|, \gamma = \max |\varphi'|$$

Mỗi lớp nhân với một thừa số cỡ $\gamma\|V\|_2$ ($\|\cdot\|_2$ là giá trị kỳ dị lớn nhất). Đây là **cận trên**, nhưng cho thấy cơ chế.

Thừa số < 1: biến mất

Gradient ở các lớp đầu co theo cấp số nhân.

Thừa số > 1: bùng nổ

Gradient có thể lớn lên theo cấp số nhân.

Ví dụ sigmoid

$\gamma = 0{,}25$; $\|V\|_2 \approx 2 \Rightarrow$ thừa số 0,5. Qua 10 lớp: $0{,}5^{10} \approx \mathbf{10^{-3}}$.

Với Xavier, $m_{(l-1)} = m_l = m$

Mỗi lớp nhân phương sai lỗi với $E[\varphi'^2]$. tanh gần $z = 0$: ≈ 1, giữ nguyên. Sigmoid: $\le 1/16$, độ lệch chuẩn co $\ge$ **4 lần** mỗi lớp.

4.3 · ĐO BẰNG THỰC NGHIỆM

Chuẩn gradient theo lớp tại khởi tạo

Mạng 10 lớp ẩn × 50 nơ-ron, dữ liệu ba vòng tròn, đo $\|\nabla V^{(l)} J\|_F$ ngay tại khởi tạo (seed 0). Lớp 1 gần đầu vào nhất, lớp 11 là lớp ra.

Hàm kích hoạt	Lớp 1 (gần vào)	Lớp 6	Lớp 11 (ra)
sigmoid (Xavier)	$2{,}3\times10^{-8}$	$6{,}7\times10^{-4}$	0,61
tanh (Xavier)	0,10	0,12	0,093
ReLU (He)	0,18	1,3	1,4

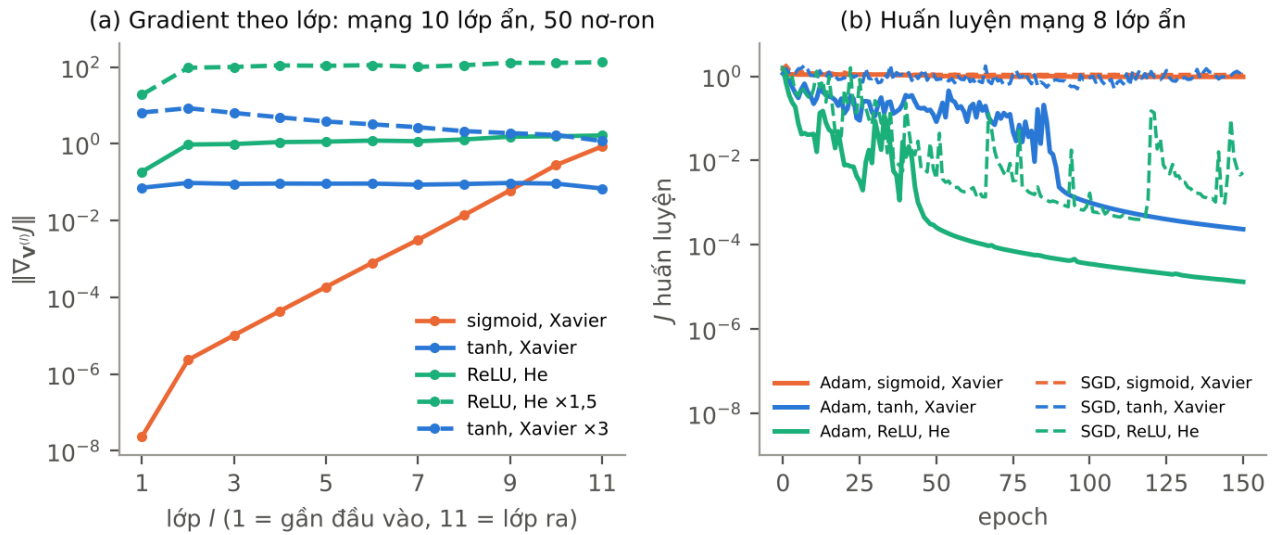
Sigmoid: giảm đều từ 0,61 xuống $2{,}3\times10^{-8}$, nhỏ hơn khoảng $2{,}7\times10^{7}$ lần; mỗi lớp lùi co khoảng 0,18 lần, cùng cỡ ước lượng lý thuyết.

tanh, ReLU: gần đồng đều giữa các lớp nhờ Xavier và He giữ độ lớn tín hiệu.

⚠ **Bùng nổ:** ReLU với độ lệch chuẩn gấp 1,5 lần He: gradient mọi lớp cỡ 100 (gấp 80 đến 100 lần), điểm số ra từ 0,63 lên 54,7, mất mát đầu 15,3 thay vì 1,39.

4.3 · HÌNH 17

Gradient theo lớp và đường cong mất mát



(a) Sigmoid: gradient co theo cấp số nhân khi lùi về các lớp đầu.

(a) tanh (Xavier), ReLU (He): gần đều. Khởi tạo quá lớn (nét đứt): gradient lớn lên.

(b) Mạng 8 lớp ẩn × 30 nơ-ron: nét liền là Adam, nét đứt là SGD; trung bình 3 khởi tạo.

Hình 17. (a) Chuẩn gradient theo lớp tại khởi tạo, mạng 10 lớp ẩn × 50 nơ-ron, trung bình hình học 5 khởi tạo (thang logarit). (b) Mất mát huấn luyện của mạng 8 lớp ẩn × 30 nơ-ron trên dữ liệu vòng tròn, trung bình 3 khởi tạo.

4.3 · HẬU QUẢ KHI HUẤN LUYỆN

Mạng sâu sigmoid gần như không học được

Mạng 2–30–…–30–3 (8 lớp ẩn), mini-batch 32, 150 epoch, 3 khởi tạo, dữ liệu vòng tròn:

Kích hoạt, khởi tạo	Thuật toán, η	J huấn luyện cuối	Độ chính xác kiểm tra (3 lần)
sigmoid, Xavier	Adam, 0,01	0,969	43,3%; 42,8%; 43,9%
sigmoid, Xavier	SGD, 0,5	1,107	33,3% cả ba lần (đoán mò)
tanh, Xavier	Adam, 0,01	2×10^{-4}	97,8%; 99,4%; 98,3%
tanh, Xavier	SGD, 0,5	0,950	66,1%; 43,9%; 33,3%
ReLU, He	Adam, 0,01	1×10^{-5}	99,4%; 100%; 99,4%
ReLU, He	SGD, 0,1	0,005	99,4%; 99,4%; 98,9%

SGD + sigmoid: J = 1,107 ≈ ln 3 ≈ 1,099 (chưa biết gì) kể cả sau 150 epoch.

Adam chuẩn hóa bước theo từng tham số nên gradient nhỏ vẫn đi được cỡ η. Kết quả chỉ đúng với bài toán này.

4.3 · CÁCH KHẮC PHỤC

Mỗi vấn đề một cách chữa

Vấn đề	Cách khắc phục thường dùng
Đối xứng	Khởi tạo ngẫu nhiên; không bao giờ khởi tạo mọi trọng số bằng nhau
Bão hòa tanh, sigmoid	Khởi tạo Xavier; chuẩn hóa đầu vào; hạn chế sigmoid ở lớp ẩn
Gradient biến mất	ReLU và Leaky ReLU, khởi tạo He, Adam; ngoài chương: chuẩn hóa theo lô, kết nối tắt
Gradient bùng nổ	Khởi tạo đúng độ lớn, giảm η, cắt gradient
ReLU chết	Leaky ReLU, giảm η, khởi tạo He

Cắt gradient (gradient clipping)

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|)$$

Co gradient về độ dài tối đa c, giữ nguyên hướng.

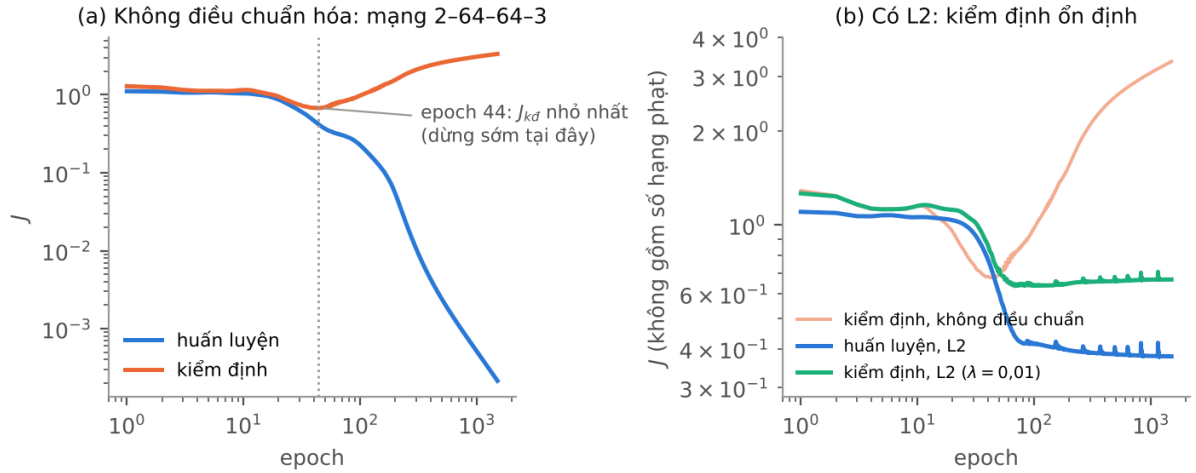
Ví dụ

$\mathbf{g} = (3, 4)$, $\|\mathbf{g}\| = 5$, $c = 1 \Rightarrow \mathbf{g} \leftarrow (0{,}6; 0{,}8)$: hướng giữ nguyên, độ dài còn 1.

Thói quen tốt: in chuẩn gradient theo lớp ở vài epoch đầu; lớp đầu nhỏ hơn lớp cuối vài bậc là biến mất, lớn đột biến là bùng nổ.

4.4 · QUÁ KHỚP

Khi mạng học thuộc lòng



Hình 18. Đường cong học trên 60 mẫu huấn luyện (thang logarit cả hai trục). (a) Không điều chuẩn hóa: J kiểm định nhỏ nhất ở epoch 44 rồi tăng. (b) Có L2: J kiểm định ổn định.

Thiết lập. Ba vòng tròn nhòe σ = 0,8 (lớp lấn nhau). 60 mẫu huấn luyện, 300 kiểm định, 900 kiểm tra. Mạng 2–64–64–3, tanh, **4547 tham số** (gấp 76 lần số mẫu). Mức tham chiếu với 6000 mẫu: 75,3%.

Kết quả. J huấn luyện giảm đến 2×10^{-4}, độ chính xác 100%. J kiểm định nhỏ nhất 0,677 ở epoch 44 rồi **tăng** lên 3,35. Độ chính xác kiểm tra chỉ **65,2%**, kém tham chiếu 10 điểm.

⚠ **Quá khớp:** J huấn luyện tiếp tục giảm nhưng J trên dữ liệu chưa từng thấy bắt đầu tăng.

4.4 · ĐIỀU CHUẨN HÓA L2

Phạt trọng số lớn

Mạng khớp nhiễu bằng trọng số rất lớn để tạo biên cong gắt (‖V‖ lên đến 36,1 trong thí nghiệm). L2 thêm số hạng phạt (không phạt hệ số chệch):

$$J_\lambda = J + (\lambda/2) \sum_l \|V^{(l)}\|^2_F$$

$$\nabla V^{(l)} J_\lambda = \nabla V^{(l)} J + \lambda V^{(l)}$$

Suy giảm trọng số

SGD: $V \leftarrow (1 - \eta\lambda)V - \eta\nabla J$. Mỗi bước, V co lại một chút trước khi cập nhật theo gradient.

Ví dụ số

$V = 2{,}0$; $\eta = 0{,}1$; $\lambda = 0{,}5$; $\nabla J = 0{,}3 \Rightarrow V \leftarrow 0{,}95 \cdot 2{,}0 - 0{,}1 \cdot 0{,}3 =$ **1,87**.

Trực giác: trọng số nhỏ làm hàm của mạng "trơn" hơn; đạo hàm đầu ra theo đầu vào bị chặn bởi tích $\gamma\|V\|_2$ qua các lớp (cận mục 4.3).

Góc nhìn xác suất: L2 ứng với giả định trọng số có phân phối Gauss quanh 0. Với Adam, L2 và suy giảm trọng số không tương đương (AdamW tách riêng).

4.4 · CHỌN Λ

Chọn λ trên tập kiểm định

Mạng 2–64–64–3 trên 60 mẫu, 5 khởi tạo cho mỗi giá trị λ:

λ	10^{-4}	10^{-3}	3×10^{-3}	10^{-2}	3×10^{-2}	10^{-1}
Chính xác kiểm định	65,3%	67,2%	68,9%	72,4%	71,5%	33,3%
Chính xác kiểm tra	65,9%	65,0%	66,8%	72,1%	71,3%	33,3%

λ quá nhỏ (10^{-4}): gần như không có tác dụng, kết quả sát không điều chuẩn hóa.

λ quá lớn (0,1): ép mọi trọng số về 0, mạng đoán mò: **thiếu khớp** (underfitting).

λ = 0,01 tốt nhất theo kiểm định: J kiểm định không tăng lại (Hình 18b), biên trơn (Hình 19c).

Chọn λ trên tập **kiểm định**, không phải tập kiểm tra (mục 5.2): tập kiểm tra chỉ để báo cáo.

4.4 · DỪNG SỚM

Ngừng khi J kiểm định bắt đầu tăng

1 · Đo

Sau mỗi epoch, tính J trên tập kiểm định.

2 · Lưu

Lưu bộ trọng số có J kiểm định nhỏ nhất từ trước đến giờ.

3 · Dừng

Sau k epoch (“kiên nhẫn”) không cải thiện thì dừng, trả về bộ trọng số đã lưu.

Vì sao cùng tác dụng với L2

Mạng khởi tạo trọng số nhỏ và chỉ lớn dần theo huấn luyện; dừng sớm cũng **giữ trọng số nhỏ**.

Kết quả (k = 30, 10 khởi tạo)

Trọng số tốt nhất ở epoch **46**; ‖V‖ còn **12,5**; độ chính xác kiểm tra **69,7%** (Hình 19b).

Rẻ và đơn giản: không cần đổi hàm mất mát, chỉ cần theo dõi J kiểm định mỗi epoch.

4.4 · DROPOUT

Ngẫu nhiên tắt nơ-ron ẩn khi huấn luyện

$$\hat{A}^{(l)} = \varphi_1(Z^{(l)}) \odot M^{(l)}$$

Mỗi phần tử của M bằng 0 với xác suất p và bằng 1/(1 − p) với xác suất 1 − p. Kỳ vọng E[m] = 0·p + (1/(1 − p))(1 − p) = 1.

Ví dụ p = 0,5: a = 0,8 thành 0 hoặc 1,6; kỳ vọng vẫn 0,8. Khi dự đoán dùng mạng bình thường, không mặt nạ.

Hiệu quả: huấn luyện đồng thời rất nhiều mạng con dùng chung trọng số; dự đoán là xấp xỉ trung bình của chúng.

Lan truyền ngược chỉ thêm mặt nạ

$$D^{(l-1)} = \left(D^{(l)} V^{(l)T} \odot M^{(l-1)} \right) \odot \varphi'(Z^{(l-1)})$$

Nơ-ron bị tắt không nhận lỗi; nơ-ron giữ nhận lỗi nhân 1/(1 − p). φ' tính từ Z, không từ đầu ra đã nhân mặt nạ.

✓ Kiểm tra gradient phải **cố định mặt nạ**, nếu không J(w ± ε) dùng hai mặt nạ khác nhau. Với mặt nạ cố định: sai số tương đối $1{,}6 \times 10^{-7}$ (mạng 3–5–4–3, p = 0,3, λ = 0,1).

4.4 · CÀI ĐẶT

Lan truyền ngược có L2 và dropout

```
def nguoc_dropout(As, Zs, Ms, Y, Ws, kich_hoat, lam=0.0):
  phi, dphi = KICH_HOAT[kich_hoat]
  n, L = len(Y), len(Ws)
  grads = [None] * L
  D = (As[-1] - Y) / n
  for l in range(L - 1, -1, -1):
    grads[l] = them_bias(As[l]).T @ D
    if lam > 0:
      grads[l][1:] += lam * Ws[l][1:] # + lambda V
    if l > 0:
      dA = D @ Ws[l][1:].T
      if Ms:
        dA = dA * Ms[l - 1] # mặt nạ dropout
      D = dA * dphi(phi(Zs[l - 1]), Zs[l - 1])
  return grads
```

L2: chỉ cộng λV vào gradient, bỏ hàng 0 (hệ số chệch).

Dropout: nhân dA với mặt nạ của lớp trước.

φ': tính từ Z (dòng cuối vòng lặp), không từ đầu ra đã nhân mặt nạ.

4.4 · SO SÁNH CÁC CÁCH

Điều chuẩn hóa: 10 khởi tạo, 900 mẫu kiểm tra

Cách	Chính xác kiểm tra	J kiểm tra	‖V‖ cuối
Không điều chuẩn hóa	66,3% ± 0,8%	2,826	36,1
Dừng sớm (k = 30)	69,7% ± 0,5%	0,645	12,5
L2 (λ = 0,01)	72,1% ± 0,0%	0,654	5,7
Dropout (p = 0,1)	66,8% ± 0,8%	1,372	41,4
L2 + dropout	71,6% ± 0,6%	0,619	6,0

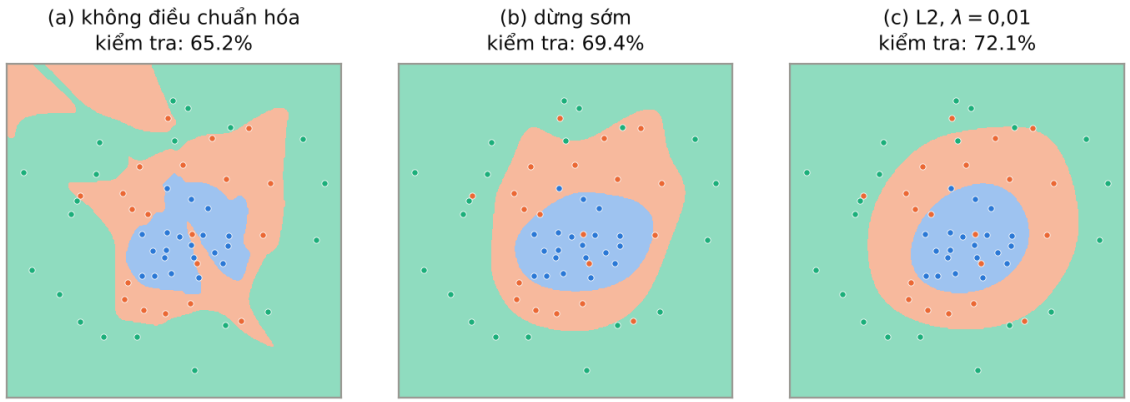
Mức tham chiếu (6000 mẫu): 75,3%. L2 gần tới mức này.

Chuẩn ‖V‖ giảm từ 36,1 xuống 5,7 với L2: trọng số nhỏ ⇒ biên trơn hơn.

Quy trình luôn giống nhau: **chọn trên tập kiểm định, báo cáo trên tập kiểm tra** (mục 5.2).

4.4 · HÌNH 19

Vùng quyết định và cách đọc bảng



L2 và dừng sớm cải thiện rõ. Hình (a): biên chằng chịt, khớp cả nhiễu. Hình (c): biên trơn, gần ba vành tròn.

Dropout p = 0,1 không giúp ở đây. Thử p = 0,3 và 0,5: 66,4% và 66,5%; mạng 2–256–256–3: 64,8% và 65,6% so với 66,5% không dropout. Thiết kế cho mạng lớn trên dữ liệu lớn.

Đọc cẩn thận. Độ lệch chuẩn chỉ phản ánh khác biệt do khởi tạo, không do dữ liệu; 1% trên 900 mẫu chỉ là 9 mẫu.

4.5 · RỘNG HAY SÂU

Biểu diễn được và học được là hai chuyện

Cần một mạng mạnh hơn: thêm nơ-ron vào mỗi lớp (**rộng**) hay thêm lớp (**sâu**)? Hai khái niệm cần tách bạch:

Khả năng biểu diễn

Tập các hàm mạng **có thể** biểu diễn nếu chọn trọng số phù hợp.

Khả năng học

Thuật toán huấn luyện (mục 3) có **tìm được** trọng số đó từ dữ liệu, trong thời gian chấp nhận được hay không.

Mạng nông, một biến

ReLU, m nơ-ron ẩn: tuyến tính từng khúc, tối đa **m + 1** đoạn (mỗi nơ-ron thêm tối đa một điểm gãy).

Mạng nông, hai chiều vào

Mỗi nơ-ron chia mặt phẳng bằng một đường thẳng; m nơ-ron tạo tối đa **1 + m + C(m, 2)** vùng.

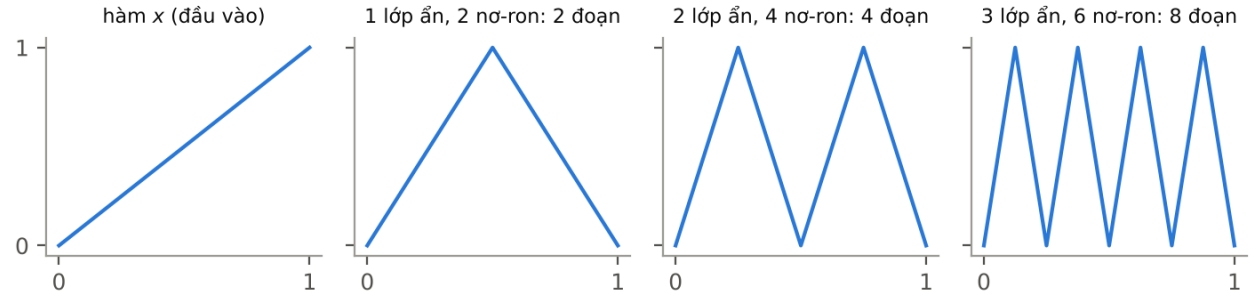
Định lý xấp xỉ phổ quát (mục 2.5): một lớp ẩn **đủ rộng** là đủ về lý thuyết. Nhưng "đủ rộng" có thể là **rất rộng**.

4.5 · HÌNH 20

Hàm răng cưa: sâu gấp đôi số đoạn mỗi lớp

$h(x) = 2\,\mathrm{ReLU}(x) - 4\,\mathrm{ReLU}(x - ½) = 2x$ nếu $x \le ½$; $2 - 2x$ nếu $x \ge ½$

Một lớp ẩn 2 nơ-ron vẽ một "tam giác" trên [0, 1]. Xếp k lớp: h ∘ h ∘ … ∘ h.



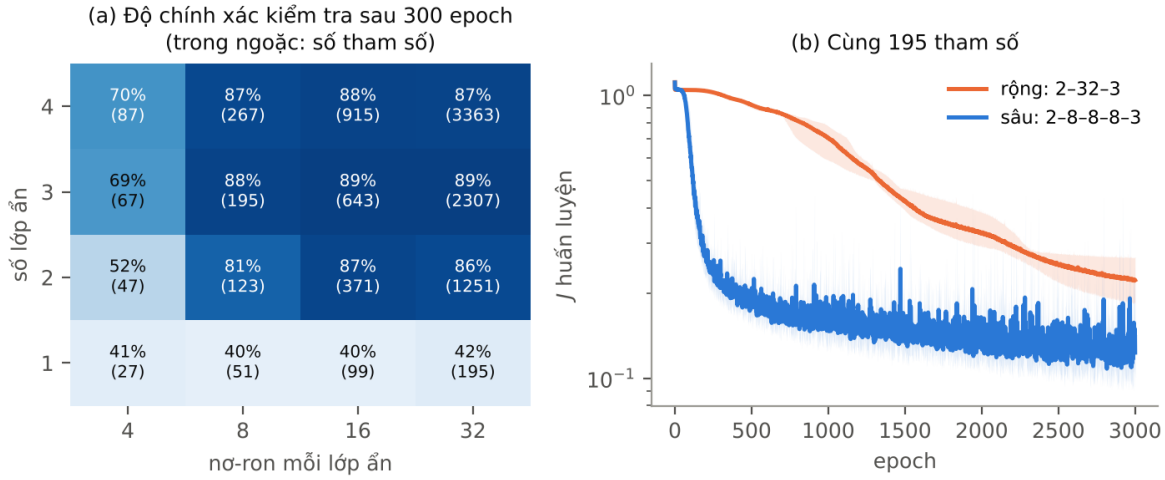
Mạng sâu: 2^k đoạn chỉ với **2k nơ-ron**. k = 10: 20 nơ-ron.

Mạng nông: muốn 2^k đoạn cần ít nhất $2^k - 1$ nơ-ron. k = 10: **1023** nơ-ron.

Ý nghĩa: sâu biểu diễn hiệu quả các hàm có cấu trúc lặp (đặc trưng được dùng lại).

4.5 · HÌNH 21

Khả năng học trên dữ liệu xoắn ốc



Hình 21. (a) Độ chính xác kiểm tra sau 300 epoch theo số lớp ẩn (1 đến 4) và số nơ-ron mỗi lớp (4 đến 32). (b) Cùng 195 tham số: mạng rộng 2–32–3 và mạng sâu 2–8–8–8–3.

1 · Một lớp ẩn gần như không học được trong 300 epoch: 40 đến 42% với m = 4, 8, 16, 32. 2–256–3 và 2–1024–3 cũng chỉ 41,7% và 42,7%. 2–32–3 cần 2000 epoch mới đạt 82,7%.

2 · Từ hai lớp ẩn trở lên học nhanh: với m ≥ 8, độ chính xác 81% đến 89%.

3 · Thêm lớp không phải lúc nào cũng tốt hơn: 4 lớp không hơn 3 lớp (87 đến 88% so với 88 đến 89%). m = 4 thì thiếu sức biểu diễn.

4.5 · SO SÁNH CÔNG BẰNG

Cùng 195 tham số: rộng hay sâu?

Mạng rộng 2–32–3 và mạng sâu 2–8–8–8–3 đều có đúng 195 tham số. Huấn luyện 3000 epoch, 5 khởi tạo (Hình 21b):

Mạng	J ở epoch 300	J ở epoch 1000	J ở epoch 3000	Chính xác kiểm tra cuối
Rộng 2–32–3	1,004	0,705	0,222	85,9% ± 0,8%
Sâu 2–8–8–8–3	0,209	0,175	0,123	87,1% ± 0,8%

Mạng sâu giảm mất mát nhanh hơn nhiều

Ở epoch 300: 0,209 so với 1,004.

Chất lượng cuối chỉ nhỉnh hơn một chút

Hơn 1,2 điểm phần trăm, cỡ một đến hai độ lệch chuẩn: chưa kết luận chắc.

Khác biệt rõ nhất là **tốc độ học**, không phải khả năng biểu diễn cuối: mạng rộng nông cũng biểu diễn được hàm này, nhưng thuật toán mất nhiều thời gian hơn để tìm ra.

4.5 · HƯỚNG DẪN THỰC HÀNH

Bốn nguyên tắc thiết kế mạng

1 · Bắt đầu nhỏ

Một đến hai lớp ẩn, vài chục nơ-ron mỗi lớp; tăng dần nếu mạng thiếu khớp (J huấn luyện còn cao).

2 · Tăng độ rộng

Ít kẹt hơn (Hình 16b) và tăng sức biểu diễn, nhưng tham số tăng theo m^2 mỗi lớp và dễ quá khớp. Số tham số: $\sum (m_{(l-1)} + 1)\, m_l$.

3 · Tăng độ sâu

Hiệu quả khi hàm có cấu trúc phân cấp; cùng số tham số thường học nhanh hơn, nhưng khó huấn luyện: cần ReLU, He, Adam và theo dõi gradient từng lớp.

4 · Luôn đối chiếu J huấn luyện và J kiểm định

Huấn luyện cao: thiếu khớp (mạng lớn hơn, huấn luyện lâu hơn). Huấn luyện thấp, kiểm định cao: quá khớp (điều chuẩn hóa hoặc mạng nhỏ hơn).

⚠ Các nguyên tắc rút ra từ **bài toán nhỏ**; hãy kiểm tra lại trên bài toán của bạn. Mục 5 sẽ làm việc đối chiếu này bài bản.

TỔNG KẾT · LIÊN HỆ

Softmax Regression và mạng nơ-ron

Khía cạnh	Softmax Regression	Mạng nơ-ron
Hình dạng J	Lồi: một cực tiểu toàn cục	Không lồi: nhiều cực tiểu, yên ngựa, vùng phẳng
Phụ thuộc khởi tạo	Không	Có (kết quả khác nhau giữa các lần chạy)
Độ sâu	Một lớp	Gradient có thể biến mất hoặc bùng nổ khi sâu
Quá khớp	Điều chuẩn hóa L2	L2, dừng sớm, dropout
Cấu trúc	Chỉ chọn η, số epoch	Còn chọn số lớp, số nơ-ron, hàm kích hoạt, khởi tạo

Mục 3 cho thuật toán huấn luyện; mục 4 giải thích **khi nào và vì sao** nó hoạt động hoặc thất bại.

Mục 5: “mạng đã đủ tốt chưa”, “lỗi nằm ở đâu”, “chọn siêu tham số thế nào” cần công cụ đánh giá có hệ thống: đường cong học, chia tập, các chỉ số cho bài toán nhiều lớp.

TÓM TẮT MỤC 4 · PHẦN 1/2

Mẫu sai, không lồi và đối xứng

1. Mẫu sai

Gradient là trung bình các lực kéo $\tilde{a}_i^T d_i$. Mẫu sai có $\|p - y\|$ lớn nên kéo mạnh (gấp khoảng 44 lần mẫu đúng); lỗi phân bổ về lớp ẩn theo trọng số nối và bị cổng φ' chặn ở nơ-ron bão hòa. Cross-entropy giữ gradient lớn khi sai nặng, bình phương sai số thì không.

2. Không lồi

Hoán vị nơ-ron ẩn cho cùng hàm nhưng J ở điểm giữa tăng cao, nên J không lồi. Điểm dừng phân loại qua giá trị riêng của Hessian. Trên XOR có cực tiểu địa phương gần phẳng ($J \approx 0{,}348$) và cao nguyên là vùng yên ngựa. Kết quả phụ thuộc khởi tạo; mạng rộng ít kẹt hơn.

3. Đối xứng, bão hòa, biến mất/bùng nổ

Trạng thái đối xứng là yên ngựa được bảo toàn; nhiễu càng nhỏ thoát càng chậm. Tín hiệu lỗi qua L lớp bị nhân với tích các thừa số cỡ $\gamma\|V\|_2$, nên sigmoid ($\gamma = 0{,}25$) làm gradient co theo cấp số nhân. Chữa: ReLU, He hoặc Xavier, Adam, cắt gradient; theo dõi chuẩn gradient theo lớp.

TÓM TẮT MỤC 4 · PHẦN 2/2

Quá khớp, rộng và sâu

4. Quá khớp và điều chuẩn hóa

Mạng nhiều tham số khớp cả nhiễu: J huấn luyện giảm, J kiểm định tăng lại. L2 thêm λV vào gradient (suy giảm trọng số); dừng sớm giữ trọng số tốt nhất theo kiểm định; dropout tắt ngẫu nhiên nơ-ron ẩn và nhân $1/(1 - p)$. Mọi siêu tham số chọn trên tập kiểm định. Thí nghiệm: L2 tốt nhất (72,1%), dropout không giúp với 60 mẫu.

5. Rộng và sâu

Một lớp ẩn đủ về lý thuyết, nhưng mạng sâu biểu diễn hiệu quả hơn hàm có cấu trúc lặp (2^k đoạn với 2k nơ-ron, so với tối thiểu $2^k - 1$ nơ-ron nếu nông). Biểu diễn được chưa có nghĩa là học được: trên dữ liệu xoắn ốc, mạng một lớp ẩn học chậm hơn nhiều mạng sâu cùng số tham số; nhưng thêm lớp không phải lúc nào cũng tốt hơn.

[icon: Arrow-right] **Mục 5:** đánh giá và trực quan hóa mô hình: đường cong học, chia tập, các chỉ số cho bài toán nhiều lớp.

KIỂM TRA NHANH · CÂU 1 ĐẾN 6

Bạn đã nắm được chưa?

1. Mẫu A: $p = (0{,}98; 0{,}01; 0{,}01)$, $y = (1, 0, 0)$. Mẫu B: $p = (0{,}7; 0{,}2; 0{,}1)$, $y = (0, 1, 0)$. Tính $\|p - y\|$ của từng mẫu và tỉ số.

2. Nơ-ron ra sigmoid, $y = 1$, $a = 0{,}01$. Tính $\partial l/\partial s$ với cross-entropy và với $l = \frac{1}{2}(a - y)^2$. Hàm nào sửa sai nhanh hơn?

3. $d^{(2)} = [0{,}5\ -0{,}5]$, $V^{(2)} = [[1, 2], [-1, 0{,}5]]$, $a^{(1)} = [0{,}9\ 0]$ (tanh). Tính $d^{(1)}$.

4. Lớp ẩn tanh có $m = 5$ nơ-ron. Có bao nhiêu bộ trọng số cho cùng một hàm do hoán vị và đổi dấu?

5. $J(\theta A) = J(\theta B) = 0{,}1$, còn điểm giữa $J = 2{,}0$. Vì sao điều này chứng tỏ J không lồi?

6. Hessian tại điểm dừng có giá trị riêng $\{2; 0{,}5; -0{,}1; 0\}$. Đó là loại điểm dừng nào? GD đi tiếp thế nào?

KIỂM TRA NHANH · CÂU 7 ĐẾN 12

Biến mất, điều chuẩn hóa, rộng và sâu

7 Mạng 10 lớp ẩn, mỗi lớp $\|V\|_2 = 2$. Tính cận trên hệ số nhân gradient với sigmoid ($\gamma = 0{,}25$) và tanh ($\gamma = 1$). Cận tanh có chứng tỏ bùng nổ không?

8 Gradient g có độ dài 50, ngưỡng cắt c = 5. Độ dài sau cắt và hệ số nhân là bao nhiêu?

9 SGD có L2: V = 2,0; $\eta = 0{,}1$; $\lambda = 0{,}5$; $\nabla J = 0{,}3$. Tính V mới và phần đóng góp của suy giảm trọng số.

10 Dropout p = 0,5, đầu ra a = 0,8. Các giá trị có thể có của â và kỳ vọng của nó?

11 Mạng ReLU nông một biến có 6 nơ-ron ẩn biểu diễn tối đa bao nhiêu đoạn? Mạng sâu 3 lớp ẩn, mỗi lớp 2 nơ-ron (hàm răng cưa h) có bao nhiêu đoạn?

12 Mạng 2–64–64–3 có bao nhiêu tham số? Với 60 mẫu vì sao dễ quá khớp? Vì sao chọn λ trên tập kiểm định mà không phải tập kiểm tra?

ĐÁP ÁN · CÂU 1 ĐẾN 4

Lời giải ngắn gọn

1 A: $p - y = (-0{,}02; 0{,}01; 0{,}01)$, độ dài $\sqrt{0{,}0006} \approx$ **0,0245**. B: $(0{,}7; -0{,}8; 0{,}1)$, độ dài $\sqrt{1{,}14} \approx$ **1,068**. Tỉ số ≈ **43,6**: mẫu sai kéo mạnh gấp khoảng 44 lần.

2 Cross-entropy: $a - y =$ **−0,99**. Bình phương: $(a - y)a(1 - a) = -0{,}99 \cdot 0{,}01 \cdot 0{,}99 =$ **−0,0098**. Cross-entropy sửa nhanh hơn cỡ 101 lần vì thừa số $a(1 - a)$ làm bình phương nhỏ đi khi nơ-ron bão hòa ở sai chỗ.

3 $d^{(2)}V^{(2)T} = [0{,}5 \cdot 1 - 0{,}5 \cdot 2,\ 0{,}5 \cdot (-1) - 0{,}5 \cdot 0{,}5] = [-0{,}5\ -0{,}75]$; $\varphi' = 1 - a \odot a = [0{,}19\ 1]$; vậy $d^{(1)} =$ **[−0,095 −0,75]**.

4 Hoán vị $5! = 120$; đổi dấu $2^5 = 32$; tổng $2^5 \cdot 5! =$ **3840** bộ trọng số.

ĐÁP ÁN · CÂU 5 ĐẾN 8

Lời giải ngắn gọn

5 Nếu J lồi thì J(điểm giữa) ≤ ½(0,1 + 0,1) = **0,1**. Thực tế là 2,0 > 0,1, trái với tính lồi.

6 Có cả dương và âm: **điểm yên ngựa** (giá trị riêng 0 cho thêm hướng phẳng). GD đi ra theo hướng ứng với −0,1, nhưng chậm vì độ cong theo hướng đó nhỏ.

7 Sigmoid: $(0{,}25 \cdot 2)^{10} = 0{,}5^{10} \approx \mathbf{9{,}8 \times 10^{-4}}$. tanh: $(1 \cdot 2)^{10} = \mathbf{1024}$. Chỉ là cận trên, **không** chứng tỏ bùng nổ: thực tế có thể nhỏ hơn nhiều, tùy hướng của d so với các hướng riêng của V.

8 Hệ số nhân min(1, 5/50) = **0,1**; độ dài sau cắt là **5**, hướng giữ nguyên.

ĐÁP ÁN · CÂU 9 ĐẾN 12

Lời giải ngắn gọn

9 $V \leftarrow (1 - 0{,}1 \cdot 0{,}5) \cdot 2{,}0 - 0{,}1 \cdot 0{,}3 = 1{,}9 - 0{,}03 =$ **1,87**. Suy giảm trọng số làm V giảm **0,1** (2,0 → 1,9), gradient giảm thêm 0,03.

10 $\hat{a} \in \{0; 1{,}6\}$ mỗi giá trị xác suất 0,5; kỳ vọng $0{,}5 \cdot 0 + 0{,}5 \cdot 1{,}6 =$ **0,8 = a**.

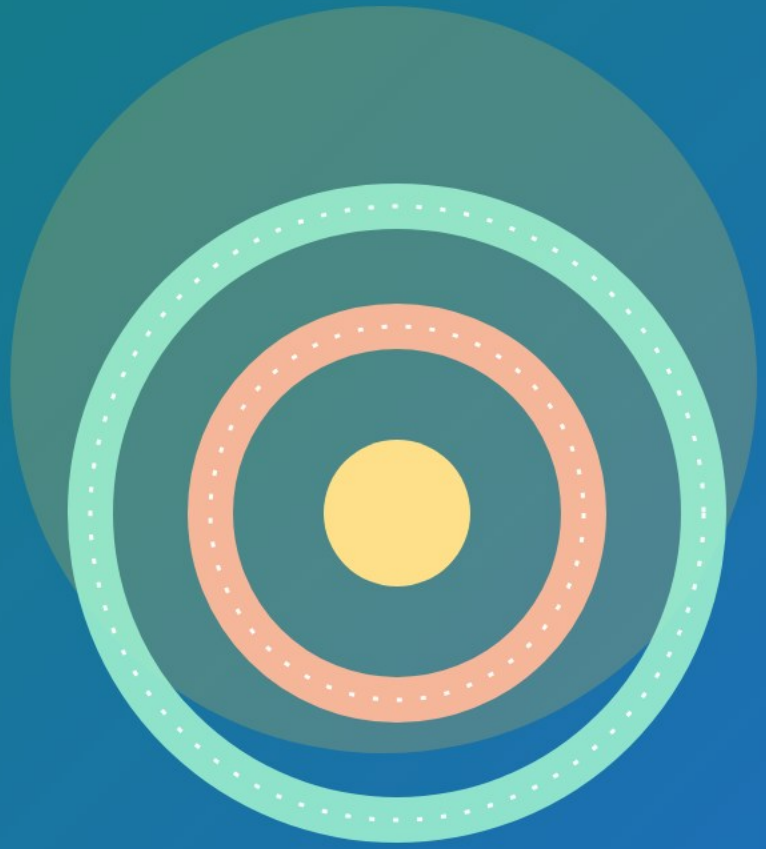
11 Nông: tối đa 6 + 1 = **7 đoạn**. Sâu: $2^3 =$ **8 đoạn** với cũng 6 nơ-ron, nhiều hơn mạng nông.

12 Tham số: $3 \cdot 64 + 65 \cdot 64 + 65 \cdot 3 = 192 + 4160 + 195 =$ **4547**, cho 60 mẫu nên có thể khớp từng mẫu, kể cả nhiễu. Chọn λ trên tập kiểm tra thì độ chính xác báo cáo trên chính tập đó bị lạc quan (đã dùng nó để chọn), không còn là ước lượng khách quan cho dữ liệu mới.

CHƯƠNG: MẠNG NƠ-RON NHÂN TẠO · MỤC 5

Đánh giá và trực quan hóa mô hình

Đo đúng để chẩn đoán đúng: từ đường cong học đến ma trận nhầm lẫn



KẾ HOẠCH

Hôm nay ta sẽ đo gì, và đo thế nào?

5.1	**Quan sát quá trình học**	Đường cong học, vùng quyết định theo epoch, biểu diễn của lớp ẩn
5.2	**Chia dữ liệu và chọn siêu tham số**	Huấn luyện / kiểm định / kiểm tra, sai số chuẩn, kiểm định chéo
5.3	**Chỉ số cho bài toán nhiều lớp**	Ma trận nhầm lẫn, precision, recall, F1, log-loss
5.4	**Chẩn đoán lỗi**	Đường cong học theo số mẫu, thiên lệch và phương sai, xem mẫu sai
5.5	**So sánh với thư viện**	MLPClassifier và Softmax Regression; khi nào mạng thật sự có lợi

NỐI VỚI MỤC 4

Từ cơ chế học đến phép đo

Mục 4 đã nói rằng…

Hàm mất mát **không lồi**: kết quả phụ thuộc khởi tạo.
Gradient có thể **biến mất** hoặc bùng nổ.
Mạng quá lớn sẽ **quá khớp**.

…nhưng dựa vào đâu?

Mọi kết luận đó dựa trên các **phép đo**:

- J trên tập huấn luyện và kiểm định
- độ chính xác trên tập kiểm tra
- đồ thị vùng quyết định

Câu hỏi của mục 5: làm sao biết **mô hình của mình** đang học tốt, thiếu khớp, quá khớp hay kẹt ở cao nguyên — và con số ta báo cáo có đáng tin không?

KÝ HIỆU BỔ SUNG

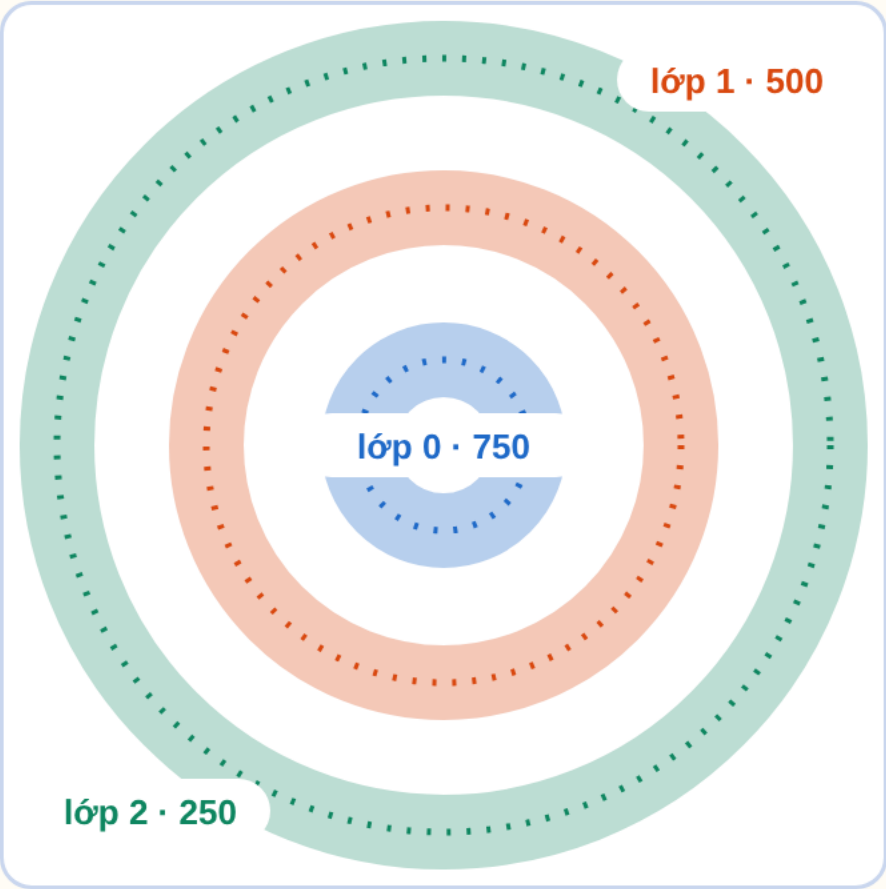
Bảng ký hiệu của mục 5

Đối tượng	Ký hiệu	Ý nghĩa
Nhãn dự đoán	$\hat{y}_i = \text{argmax}_k\ p_{ik}$	Lớp có xác suất lớn nhất của mẫu i
Ma trận nhãn dự đoán	$\hat{\mathbf{Y}}$ (n × K)	One-hot của $\hat{y}_i$
Ma trận nhầm lẫn	$\mathbf{C}$ (K × K)	c_{kj}: số mẫu thật thuộc lớp k bị dự đoán là lớp j
Chỉ số theo lớp	Prec_k, Rec_k, F1_k	Precision, recall, F1 của lớp k
Kích thước các tập	n(tr), n(kđ), n(kt)	Số mẫu huấn luyện, kiểm định, kiểm tra
Sai số chuẩn	SE	Độ lệch chuẩn của một số ước lượng (ví dụ độ chính xác)

Quy ước: ma trận ký hiệu bằng chữ **hoa in đậm** (**C**, **Ŷ**, **W**), vectơ bằng chữ **thường in đậm** (**h**, **s**). Các ký hiệu p, J, η, λ giữ nguyên nghĩa như các mục trước.

5.1 · DỮ LIỆU XUYÊN SUỐT

Ba vòng tròn nhòe, lớp không cân bằng



Dữ liệu 'khó' hơn mục 3.7

Ba vòng tròn đồng tâm, bán kính **1**; **2,8**; **4,6**, độ nhòe (độ lệch chuẩn bán kính) **0,7** nên các vòng chồng lấn.

Không cân bằng + chia phân tầng

750 / 500 / 250 mẫu (tổng 1500). Chia 60/20/20 phân tầng: huấn luyện 900, kiểm định 300, kiểm tra 300. Chuẩn hóa bằng thống kê của riêng tập huấn luyện.

Hai mốc để so sánh

Mốc tầm thường: luôn đoán lớp 0 → đúng **50%**. **Mốc tham chiếu:** mạng 2–16–16–3 trên 15 000 mẫu: **86,9%**, J = 0,322; lỗi ≈ **13,1%** gần như không thể giảm.

5.1 · ĐƯỜNG CONG HỌC THEO EPOCH

Ghi nhật ký trong lúc huấn luyện

```
def huan_luyen_co_ghi(X, Y, Xv, Yv, kich_thuoc, kich_hoat='tanh',
            eta=0.01, so_epoch=300, b=32, seed=0):
  rng = np.random.default_rng(seed)
  Ws = khoi_tao(kich_thuoc, kich_hoat, seed)
  tt = tao_trang_thai(Ws)
  nhat_ky = {'J_tr': [], 'J_kd': [], 'acc_tr': [], 'acc_kd': []}
  def ghi():
    for ten, (A, B) in (('tr', (X, Y)), ('kd', (Xv, Yv))):
      P = lan_truyen_thuan(A, Ws, kich_hoat)[0][-1]
      nhat_ky['J_' + ten].append(mat_mat(P, B))
      nhat_ky['acc_' + ten].append(np.mean(P.argmax(1) == B.argmax(1)))
  ghi()
  for epoch in range(so_epoch):
    idx = rng.permutation(len(X))
    for k in range(0, len(X), b):
      lo = idx[k:k + b]
      As, Zs = lan_truyen_thuan(X[lo], Ws, kich_hoat)
      cap_nhat(Ws, lan_truyen_nguoc(As, Zs, Y[lo], Ws, kich_hoat), tt, 'adam', eta)
    ghi()
  return Ws, nhat_ky
```

Cấu hình

Mạng 2–8–3, tanh, Adam, η = 0,01, mini-batch 32, 300 epoch.

Ghi lại

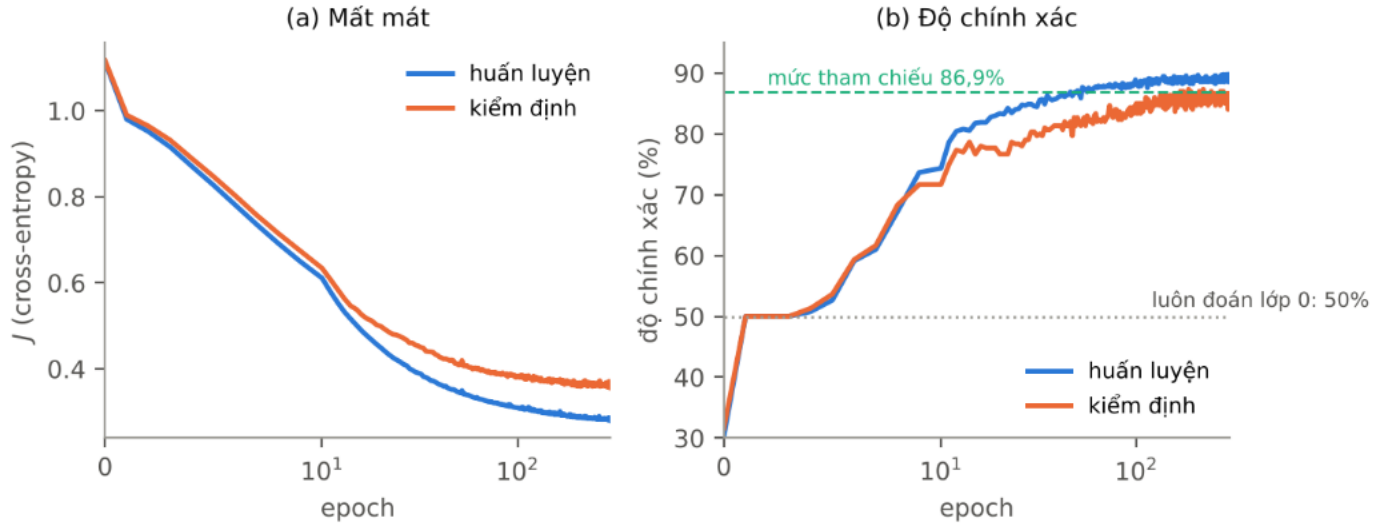
J và độ chính xác trên tập **huấn luyện** và **kiểm định**, sau **mỗi epoch**.

Điểm cần nhớ

Đây là hàm **huan_luyen** của mục 3.7 cộng phần ghi nhật ký (các dòng vàng): không đổi thuật toán học.

5.1 · HÌNH 22

Đường cong học theo epoch



J cùng giảm: J huấn luyện 0,281, kiểm định 0,358; khoảng cách nhỏ, J kiểm định chưa tăng lại → chưa quá khớp.

Cao nguyên 50%: vài epoch đầu chỉ đoán lớp đông nhất: mạng học tỉ lệ lớp (hệ số chệch) trước rồi mới học hình dạng.

Cuối cùng: 89,8% (huấn luyện) và 86,3% (kiểm định), gần mốc tham chiếu 86,9%.

Lưu ý: trục hoành chia theo thang symlog để thấy rõ các epoch đầu.

Hình 22. Đường cong học theo epoch của mạng 2–8–3 trên ba vòng tròn nhòe. (a) J huấn luyện và kiểm định. (b) Độ chính xác.

5.1 · BỐN DẠNG ĐƯỜNG CONG

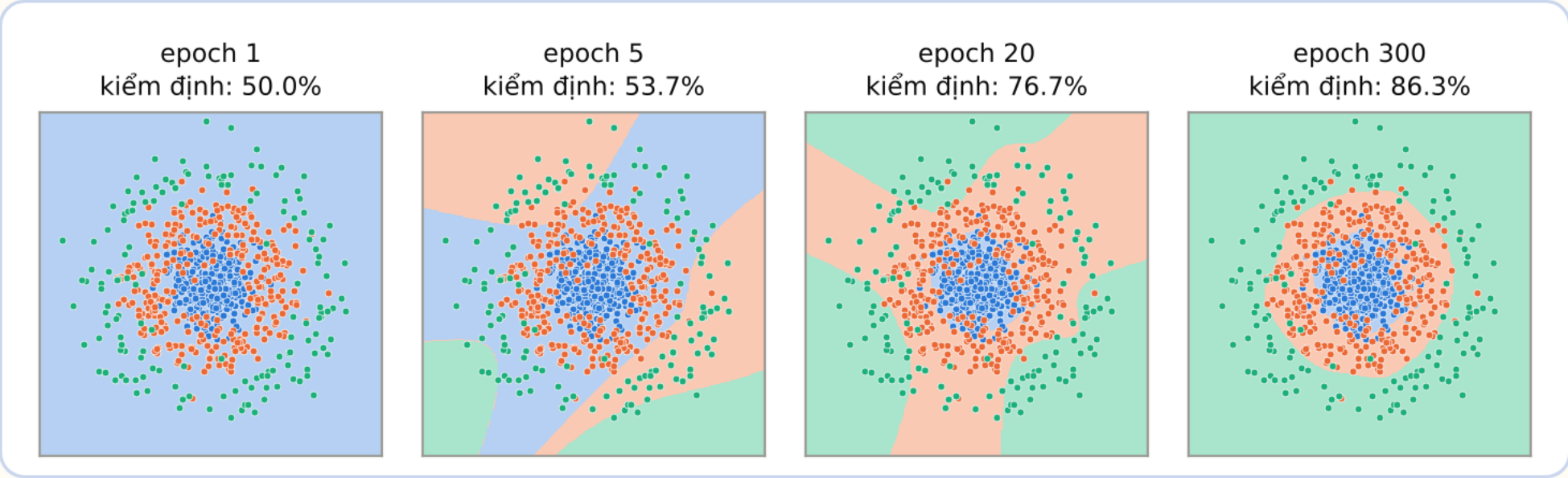
Đường cong nói gì về mô hình?

Dạng đường cong	Chẩn đoán	Việc cần làm
Cả hai giảm và hội tụ gần nhau	**Học tốt**	Giữ nguyên; huấn luyện thêm nếu còn giảm
J huấn luyện tiếp tục giảm, J kiểm định tăng trở lại	**Quá khớp (mục 4.4)**	Điều chuẩn hóa, dừng sớm, mạng nhỏ hơn, thêm dữ liệu
Cả hai cao và gần như phẳng	**Thiếu khớp, η quá nhỏ, hoặc kẹt cao nguyên (4.2, 4.3)**	Mạng lớn hơn, tăng η, kiểm tra gradient từng lớp
J dao động mạnh hoặc thành nan	**η quá lớn, gradient bùng nổ (4.3)**	Giảm η, cắt gradient, kiểm tra khởi tạo

Mẹo: luôn đọc **cả hai** đường (huấn luyện và kiểm định) **và** độ chính xác; một đường đơn lẻ có thể đánh lừa.

5.1 · HÌNH 23

Vùng quyết định theo epoch



Epoch 1: cả mặt phẳng là lớp 0 → 50,0%, đúng mốc tầm thường.

Epoch 5: xuất hiện các dải thẳng, chưa có cấu trúc vòng → 53,7%.

Epoch 20: biên cong dần quanh tâm → 76,7%.

Epoch 300: ba vành gần tròn đồng tâm → 86,3%.

Hình 23. Vùng quyết định của mạng 2–8–3 tại epoch 1, 5, 20 và 300 (chấm: mẫu huấn luyện). Độ chính xác kiểm định tăng từ 50,0% lên 86,3%.

5.1 · BIỂU DIỄN CỦA LỚP ẨN

Mạng tự học đặc trưng

$$s = [1, h]\, W^{(L)}$$

tuyến tính theo h ⇒ biên giữa các lớp là đường thẳng

Ý tưởng thí nghiệm

Mạng **thắt cổ chai** 2–8–2–3: lớp ẩn cuối chỉ có 2 nơ-ron tanh, nên **h** = (h_1, h_2) vẽ được trên mặt phẳng.

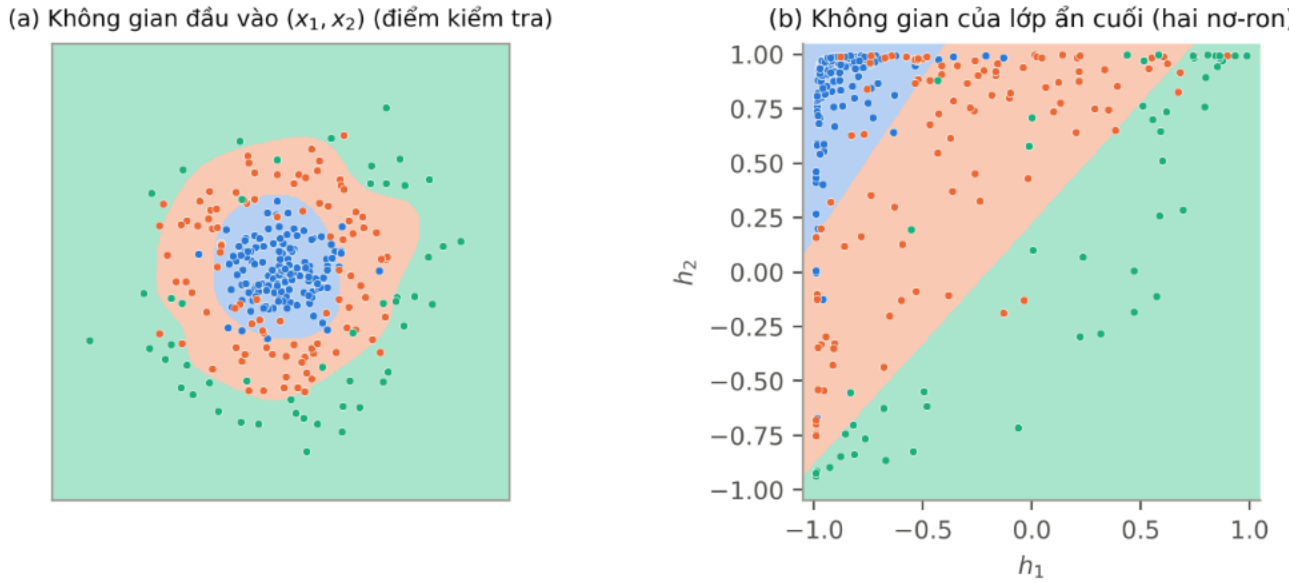
Hệ quả

Trong không gian (h_1, h_2), vùng thắng của mỗi lớp là **đa giác lồi** bị chặn bởi các đường thẳng: đó là **Softmax Regression** chạy trên đặc trưng **h** do mạng tự học.

Đây là hình ảnh cụ thể của ý tưởng ở mục 1.2: thay vì **ta đoán** đặc trưng đa thức, mạng **tự học đặc trưng** rồi đưa cho một mô hình Softmax.

5.1 · HÌNH 24

Hai không gian, hai hình dạng biên



(a) Đầu vào: ba lớp không tách được bằng đường thẳng; vùng quyết định cong.

(b) Lớp ẩn cuối: lớp 0 (xanh dương) dồn trên trái, lớp 2 (xanh lá) dưới phải, lớp 1 (cam) ở giữa; hai **đường thẳng** ngăn cách.

Điểm còn lẫn: là mẫu ở vùng lấn thật của dữ liệu, không phải lỗi mô hình.

Kết quả: chỉ 2 nơ-ron ở lớp cuối vẫn đạt 85,3% (mạng 2–8–3: 84,7%).

Hình 24. Mạng 2–8–2–3 (độ chính xác kiểm tra 85,3%). (a) Không gian đầu vào, vùng quyết định cong. (b) Cùng các điểm kiểm tra ở không gian lớp ẩn cuối (h_1, h_2): biên là đường thẳng.

Ba tập, ba vai trò

Tập	Dùng để	Được nhìn bao nhiêu lần
Huấn luyện	Tính gradient và cập nhật trọng số $\mathbf{W}^{(l)}$	Mọi epoch
Kiểm định	Chọn siêu tham số (số nơ-ron, λ, η, số epoch dừng sớm…)	Nhiều lần, mỗi cấu hình thử
Kiểm tra	Ước lượng chất lượng cuối cùng trên dữ liệu mới	**Một lần**, ở cuối cùng

Nguyên tắc cốt lõi: mọi quyết định thiết kế dựa trên tập **kiểm định**; tập **kiểm tra** chỉ để báo cáo.

Cách chia

Tỉ lệ **60/20/20** hoặc **70/15/15**. Dữ liệu ít → khó đủ lớn → dùng kiểm định chéo.

Chia phân tầng

Mỗi tập giữ nguyên tỉ lệ các lớp (ở đây 3 : 2 : 1), kẻo một tập thiếu hẳn lớp nhỏ.

Chia phân tầng, chuẩn hóa không rò rỉ

```
def chia_3_tap(X, y, ti_le=(0.6, 0.2, 0.2), seed=0):
  rng = np.random.default_rng(seed)
  tr, kd, kt = [], [], []
  for k in np.unique(y):
    idx = rng.permutation(np.where(y == k)[0])
    a = int(ti_le[0] * len(idx))
    b = int((ti_le[0] + ti_le[1]) * len(idx))
    tr += list(idx[:a]); kd += list(idx[a:b]); kt += list(idx[b:])
  return [rng.permutation(i) for i in (tr, kd, kt)]
itr, ikd, ikt = chia_3_tap(X, y)
mu, sd = X[itr].mean(axis=0), X[itr].std(axis=0)
Xtr, Xkd, Xkt = [(X[i] - mu) / sd for i in (itr, ikd, ikt)]
```

Chia phân tầng

Cắt **từng lớp** theo tỉ lệ rồi gộp lại → mỗi tập đúng tỉ lệ 3 : 2 : 1.

Chuẩn hóa

μ và σ chỉ tính trên **tập huấn luyện** (hai dòng vàng), rồi dùng chung cho cả ba tập.

Vì sao?

Tính trên toàn bộ dữ liệu → thông tin tập kiểm tra “rò” vào huấn luyện: **rò rỉ dữ liệu** (data leakage).

Mọi bước tiền xử lý có “học” từ dữ liệu (chuẩn hóa, giảm chiều, chọn đặc trưng) đều phải theo quy tắc này.

Tìm kiếm lưới trên tập kiểm định

```
ket_qua = {}
for m in (2, 4, 8, 16, 32, 64):
  for lam in (0.0, 1e-3, 1e-2, 3e-2):
    acc_kd = []
    for seed in range(3):
      Ws, _, _, _ = huan_luyen_dc(Xtr, Ytr, [2, m, 3],
              so_epoch=200, b=32, seed=seed, lam=lam)
      acc_kd.append(np.mean(du_doan(Xkd, Ws) == ykd))
    ket_qua[(m, lam)] = np.mean(acc_kd)
m_tot, lam_tot = max(ket_qua, key=ket_qua.get)
```

Tham số

Các $\mathbf{W}^{(l)}$, được học bằng gradient.

Siêu tham số

Số lớp ẩn, số nơ-ron, hàm kích hoạt, η, mini-batch, số epoch, λ, dropout p, cách khởi tạo: ta đặt **trước** khi huấn luyện.

Tìm kiếm lưới

6 giá trị m × 4 giá trị λ = **24 cấu hình**, mỗi cấu hình 3 khởi tạo, 200 epoch.

Không có công thức cho siêu tham số: **thử và đo trên tập kiểm định**. Lấy trung bình 3 khởi tạo để bớt nhiễu của hàm mất mát không lồi.

5.2 · KẾT QUẢ TÌM KIẾM LƯỚI

Bảng độ chính xác kiểm định

m \ λ	0	10^{-3}	10^{-2}	3×10^{-2}
m = 2	70,6	70,0	67,1	62,3
m = 4	85,6	84,7	76,1	70,6
m = 8	85,6	85,1	81,4	72,3
m = 16	**86,0**	85,9	83,3	73,4
m = 32	85,9	85,2	81,3	70,6
m = 64	85,3	84,9	80,3	71,0

Thiếu khớp: m = 2 chỉ ≈ 70%: hai nơ-ron không đủ tạo biên cong cho ba vành.

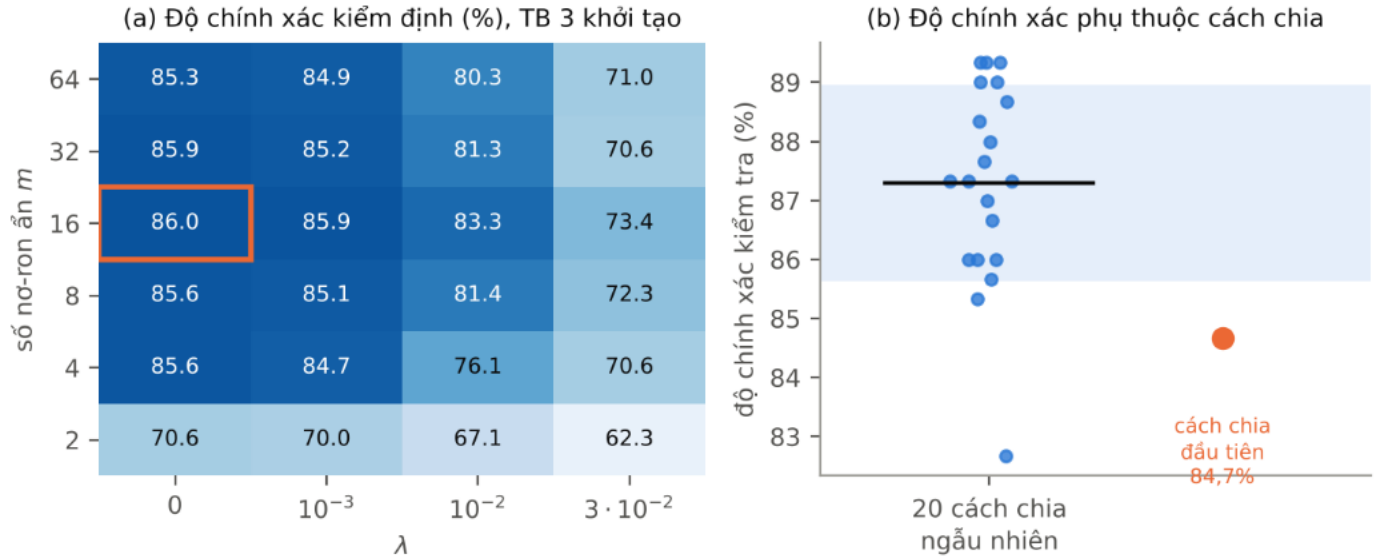
λ lớn: cột 3×10^{-2} chỉ còn 62% đến 73%. Dữ liệu đủ lớn nên λ = 0 đã tốt.

Nhiều ô ngang nhau: từ m = 4 với $\lambda \le 10^{-3}$: 84,7% đến 86,0%.

Độ chính xác kiểm định (%). Ô cam: tốt nhất (86,0). Ô xanh: ≥ 84,0, trong phạm vi 1 sai số chuẩn của ô tốt nhất.

5.2 · HÌNH 25

Kết quả phụ thuộc cách chia



(a) độ chính xác kiểm định theo m và λ; ô viền cam là tốt nhất (chính là bảng ở slide trước).

(b) cùng một cấu hình (m = 16, λ = 10^{-3}) trên **20 cách chia** ngẫu nhiên: trung bình 87,3%, độ lệch chuẩn 1,7 điểm; nhỏ nhất 82,7%, lớn nhất 89,3%.

Chấm cam: cách chia ta dùng (84,7%) nằm ở phía thấp của khoảng đó.

Thông điệp: một con số trên một tập kiểm tra nhỏ có sai số cỡ vài điểm phần trăm.

Hình 25. (a) Độ chính xác kiểm định (%) theo m và λ. (b) Độ chính xác kiểm tra của cùng một cấu hình qua 20 cách chia ngẫu nhiên: đường đen là trung bình, vùng mờ là ±1 độ lệch chuẩn.

5.2 · CHÊNH LỆCH BAO NHIÊU THÌ CÓ NGHĨA?

Sai số chuẩn và quy tắc một SE

SE = √(p(1 − p) / n)

khoảng tin cậy ≈ 95%: acc ± 1,96 · SE

Với tập kiểm định

n(kđ) = 300, p ≈ 0,86 ⇒ **SE = 2,0** điểm phần trăm. Các chênh lệch 0,4 hay 1,0 điểm trong bảng **nằm trong nhiễu**; chỉ chênh lệch lớn như m = 2 so với m ≥ 4 mới đáng tin.

Với tập kiểm tra

Tập kiểm tra 300 mẫu, acc = 0,847:
0,847 ± 1,96·√(0,847·0,153/300) = 0,847 ± **0,041**
→ khoảng **80,6% đến 88,8%**.

Quy tắc một sai số chuẩn

Chọn mô hình **đơn giản nhất** có độ chính xác kiểm định không kém cấu hình tốt nhất quá **một SE**. Ở đây: ngưỡng 86,0 − 2,0 = **84,0%** ⇒ chọn **m = 4, λ = 0** (85,6%). Mạng nhỏ: nhanh hơn, ít quá khớp hơn, dễ diễn giải hơn.

Kiểm định chéo K lần

Lần 1	KĐ	HL	HL	HL	HL
Lần 2	HL	KĐ	HL	HL	HL
Lần 3	HL	HL	KĐ	HL	HL
Lần 4	HL	HL	HL	KĐ	HL
Lần 5	HL	HL	HL	HL	KĐ

HL = huấn luyện, KĐ = kiểm định. Mỗi mẫu được dùng **cả hai** vai trò (ở các lần khác nhau).

$$CV_k = (1/K) \cdot \Sigma\ acc^f\ (f = 1 \ldots K)$$

Cấu hình	CV_5 (TB ± độ lệch chuẩn)
m = 2, λ = 0	69,9% ± 1,9%
m = 4, λ = 0	86,4% ± 2,6%
m = 16, λ = 10^{-3}	**87,2% ± 2,1%**
m = 16, λ = 3×10^{-2}	73,2% ± 0,4%

K = 5 trên 1200 mẫu: mỗi lần huấn luyện 960 mẫu, kiểm định 240 mẫu. Tập kiểm tra vẫn tách riêng, dùng một lần.

5.2 · QUY TRÌNH HOÀN CHỈNH

Bốn bước báo cáo trung thực

1. Chia phân tầng; tính thống kê tiền xử lý **chỉ trên tập huấn luyện**.
2. Thử các cấu hình, chọn theo tập kiểm định (hoặc kiểm định chéo); ưu tiên mô hình đơn giản khi chênh lệch nằm trong nhiễu.
3. **Huấn luyện lại** cấu hình đã chọn trên huấn luyện + kiểm định (1200 mẫu), giữ nguyên siêu tham số.
4. Đánh giá **một lần** trên tập kiểm tra và báo cáo kèm khoảng tin cậy.

Kết quả của ta

Chọn theo log-loss kiểm định: m = 16, $\lambda = 10^{-3}$ (J(kđ) = 0,354; acc kđ 85,9%). Huấn luyện lại trên 1200 mẫu: kiểm tra **84,7%**, **J = 0,338**.

Chọn sai tập thì sao?

Chọn theo accuracy **kiểm tra** sẽ ra m = 4, λ = 0 với 85,7%: cao hơn **1,0 điểm**. Phần chênh là lạc quan vì tập kiểm tra đã được dùng để chọn; nó tăng khi thử nhiều cấu hình hơn.

5.3 · ĐỘNG CƠ

Vì sao độ chính xác chưa đủ?

Che giấu 1

Với lớp không cân bằng, accuracy bị **lớp đông chi phối**: mô hình luôn đoán lớp 0 của ta đã đạt **50%** mà chưa học được gì.

Che giấu 2

Accuracy **không cho biết mô hình sai theo kiểu nào**, mà đó lại là thông tin cần để sửa.

Ví dụ: 1000 mẫu, 950 lớp A và 50 lớp B; mô hình luôn đoán A.

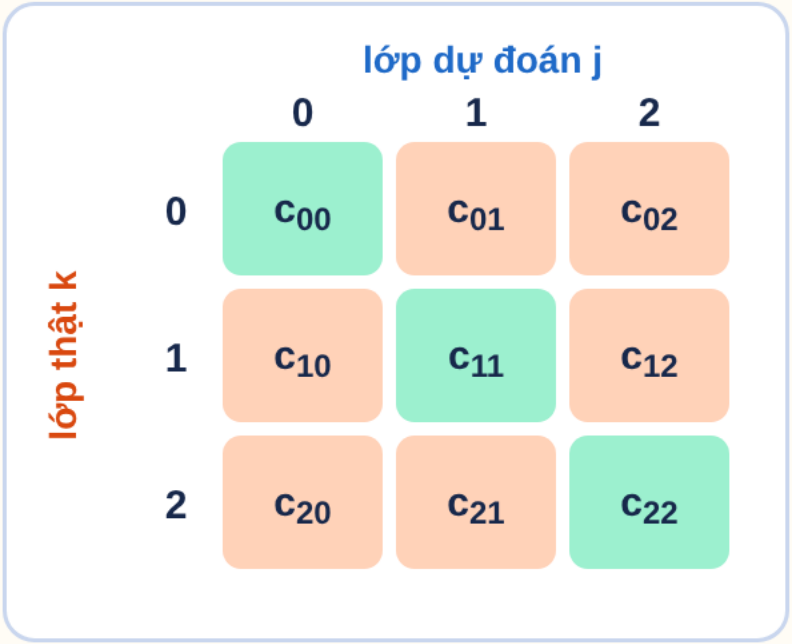
95%

accuracy: trông rất tốt

0,487

F1 macro: lộ ra mô hình vô dụng với lớp B

Sai theo kiểu nào?



$$C = Y^T \hat{Y}$$

$$c_{kj} = \#\{\, i : y_i = k,\ \hat{y}_i = j \,\}$$

Hàng k: các mẫu **thật** thuộc lớp k. **Cột j**: các mẫu được **dự đoán** là lớp j.

Đường chéo (xanh lá) là dự đoán đúng. Tổng mọi phần tử bằng n. Ngoài đường chéo (cam) là các kiểu nhầm.

Ba chỉ số từ một ma trận

$$\text{Accuracy} = \mathrm{tr}(C) / n \quad \text{Prec}_k = c_{kk} / \Sigma_i c_{ik}$$
$$\text{Rec}_k = c_{kk} / \Sigma_j c_{kj} \quad F1_k = 2 \cdot \text{Prec}_k \cdot \text{Rec}_k / (\text{Prec}_k + \text{Rec}_k)$$

Precision (cột)

Trong số mẫu mô hình **gán** vào lớp k (tổng **cột** k), bao nhiêu phần đúng. Thấp ⇒ "báo động nhầm".

Recall (hàng)

Trong số mẫu **thật sự** thuộc lớp k (tổng **hàng** k), tìm lại được bao nhiêu. Thấp ⇒ "bỏ sót".

F1

Trung bình **điều hòa**: bị kéo về số nhỏ hơn, nên chỉ cao khi **cả hai** cùng cao.

Trung bình macro

F1 macro = $(1/K)\ \Sigma\ F1_k$: mọi lớp **quan trọng như nhau**, phạt nặng việc bỏ qua lớp nhỏ.

Có trọng số

F1 có trọng số = $\Sigma\ (n_k/n)\ F1_k$, $n_k = \Sigma_j\ c_{kj}$: gần accuracy hơn. (Micro = accuracy khi mỗi mẫu một nhãn.)

5.3 · VÍ DỤ TÍNH TAY

10 mẫu, 3 lớp

thật \ đoán	0	1	2	tổng hàng
lớp 0	**3**	1	0	4
lớp 1	1	**2**	1	4
lớp 2	0	0	**2**	2
tổng cột	4	3	3	10

C = [[3, 1, 0], [1, 2, 1], [0, 0, 2]], n = 10

Accuracy = (3 + 2 + 2)/10 = **0,7**

Prec = (3/4, 2/3, 2/3); Rec = (3/4, 2/4, 2/2)

$F1_1 = 2\cdot(2/3)\cdot(1/2) / (2/3 + 1/2) =$ **0,571**;
F1 = (0,75; 0,571; 0,8)

F1 macro = **0,707**; F1 có trọng số = (4·0,75 + 4·0,571 + 2·0,8)/10 = **0,689**

Lớp 2: recall = 1 nhưng precision = 2/3: tìm đủ lớp 2 nhưng đã gán nhầm một mẫu lớp 1 vào đó.

5.3 · CÀI ĐẶT BẰNG NUMPY

Ma trận nhầm lẫn trong hai hàm ngắn

```
def ma_tran_nham_lan(y_that, y_du_doan, K):
  # C = Y^T Y_hat: C[k, j] = số mẫu thật thuộc lớp k bị đoán là lớp j
  return np.eye(K)[y_that].T @ np.eye(K)[y_du_doan]
def chi_so_theo_lop(C):
  precision = np.diag(C) / C.sum(axis=0) # chia cho tổng CỘT
  recall = np.diag(C) / C.sum(axis=1) # chia cho tổng HÀNG
  f1 = 2 * precision * recall / (precision + recall)
  return precision, recall, f1
```

Dòng khó 1

np.eye(K)[y] tạo ma trận one-hot (n × K) bằng cách chọn hàng của ma trận đơn vị.

Dòng khó 2

axis=0 cộng theo cột, **axis=1** cộng theo hàng: khớp công thức precision và recall.

Cẩn thận

Lớp không bao giờ được dự đoán ⇒ tổng cột = 0, precision không xác định (thư viện quy ước 0).

Chạy hàm trên ví dụ 10 mẫu ở slide trước cho đúng các số đã tính tay; hàm cũng khớp với **classification_report** của scikit-learn (slide 38).

5.3 · KẾT QUẢ TRÊN TẬP KIỂM TRA

Mô hình cuối cùng của ta

thật \ đoán	0	1	2
lớp 0	**139**	11	0
lớp 1	18	**76**	6
lớp 2	0	11	**39**

Ma trận nhầm lẫn trên 300 mẫu kiểm tra (mô hình cuối: m = 16, $\lambda = 10^{-3}$).

Lớp	Số mẫu	Precision	Recall	F1
0	150	0,885	0,927	0,906
1	100	**0,776**	0,760	**0,768**
2	50	0,867	**0,780**	0,821

Accuracy **0,847**

F1 macro **0,831**

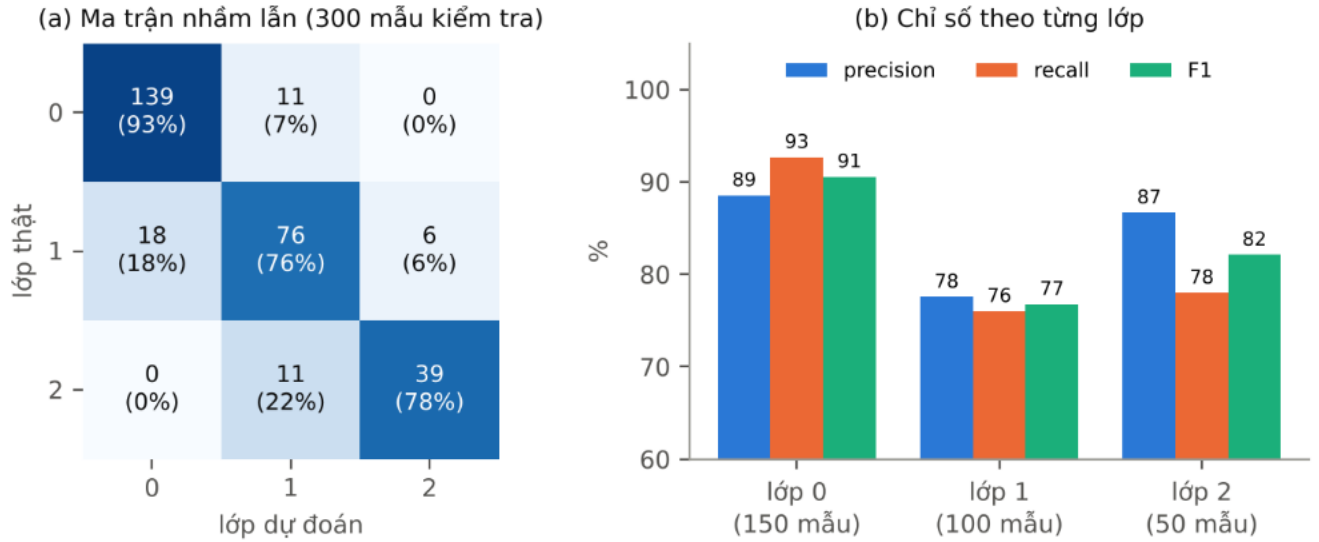
F1 có trọng số **0,846**

Log-loss **0,338**

Kiểm tra lại: accuracy = (139 + 76 + 39)/300 = 254/300 = **0,847**. Lớp 1 (vòng giữa) khó nhất: F1 = 0,768.

5.3 · HÌNH 26

Ma trận nhầm lẫn và chỉ số theo lớp



Chỉ nhầm vòng kề nhau: lớp 0 ↔ 1 (11 + 18 mẫu), lớp 1 ↔ 2 (6 + 11); lớp 0 ↔ 2 bằng **0**. Đúng hình học: hai vòng ngoài cách nhau 3,6 đơn vị.

Lớp 1 khó nhất: F1 = 0,768 vì bị kẹp giữa hai lớp.

Lớp nhỏ nhất: recall 0,78; 11 trong 50 mẫu lớp 2 bị gán sang lớp 1.

Mốc tầm thường: luôn đoán lớp 0 ⇒ accuracy 0,5 nhưng F1 macro chỉ **0,222**.

Hình 26. (a) Ma trận nhầm lẫn trên 300 mẫu kiểm tra (số mẫu và tỉ lệ theo hàng). (b) Precision, recall và F1 theo từng lớp.

5.3 · LOG-LOSS

Đánh giá cả xác suất

Log-loss = $-(1/n)\,\Sigma_i \ln p[i, y_i]$

Phạt tự tin sai

p = 0,9 chỉ tốn $-\ln 0{,}9$ = **0,105**; p = 0,01 tốn **4,605** (gấp ≈ 44 lần): đoán sai mà tự tin bị phạt rất nặng.

Mốc tham chiếu

Chưa biết gì (xác suất đều 1/K): ln 3 = **1,099**. Chỉ biết tỉ lệ lớp (0,5; 0,33; 0,17): entropy = **1,011**.

Mạng (900 mẫu huấn luyện)	Acc huấn luyện	Acc kiểm tra	Log-loss kiểm tra
2–64–64–3, không điều chuẩn hóa	98,6%	82,7%	**1,007**
2–64–64–3, L2 λ = 0,01	88,8%	85,3%	**0,347**

Accuracy chỉ chênh **2,6 điểm**, nhưng log-loss gấp ≈ **3 lần**: mạng quá khớp đoán sai với xác suất rất cao. Log-loss nhạy với quá khớp hơn accuracy.

5.3 · CHỌN CHỈ SỐ

Chọn chỉ số nào?

Tình huống	Chỉ số nên xem
Các lớp cân bằng, mọi lỗi tốn như nhau	Accuracy
Các lớp mất cân bằng	F1 macro, recall từng lớp, ma trận nhầm lẫn
Bỏ sót một lớp rất tốn kém (ví dụ phát hiện bệnh)	Recall của lớp đó
Báo động nhầm rất tốn kém (ví dụ chặn nhầm giao dịch hợp lệ)	Precision của lớp đó
Dùng xác suất đầu ra để ra quyết định	Log-loss

Nhớ: chọn chỉ số theo **cái giá của từng loại sai**, không theo thói quen. Báo cáo cuối cùng nên gồm: ma trận nhầm lẫn, F1 từng lớp, log-loss, kèm khoảng tin cậy.

Có nên thêm dữ liệu không?

Cách làm

Huấn luyện trên $\mathbf{n_s}$ mẫu đầu với n_s = 30, 60, 120, 240, 480, 900; vẽ lỗi huấn luyện và lỗi kiểm định (300 mẫu kiểm định) theo n_s. Mỗi điểm: trung bình 5 lần chạy.

Câu hỏi trả lời

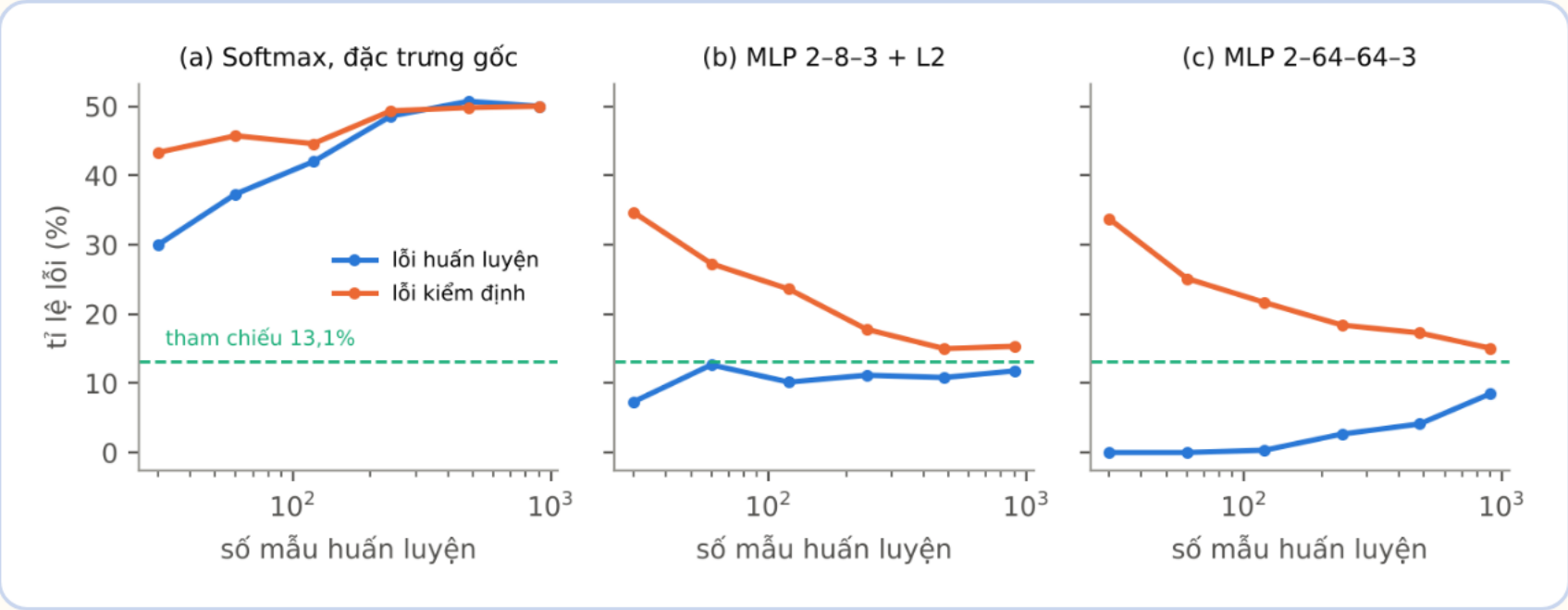
"Nên thu thập thêm dữ liệu hay đổi mô hình?" Ba mô hình: Softmax trên đặc trưng gốc; MLP 2–8–3 + L2 ($\lambda = 10^{-3}$); MLP 2–64–64–3 không điều chuẩn hóa.

Lỗi (%) huấn luyện / kiểm định	n_s = 30	n_s = 240	n_s = 900
Softmax, đặc trưng gốc	30,0 / 43,3	48,6 / 49,3	50,0 / 50,0
MLP 2–8–3 + L2	7,3 / 34,7	11,2 / 17,8	11,8 / 15,3
MLP 2–64–64–3	0,0 / 33,7	2,7 / 18,4	8,5 / 15,1

Lỗi tham chiếu (mức một mô hình tốt đạt được): **13,1%**. Trong thực tế đó có thể là sai số của chuyên gia người hoặc của hệ thống tốt nhất đã biết.

5.4 · HÌNH 27

Ba mô hình, ba dạng đường cong



(a) Softmax: hai đường gặp nhau ở **50%**, rất xa 13,1% ⇒ thiên lệch cao; thêm dữ liệu không giúp.

(b) MLP 2–8–3: huấn luyện 11,8%, kiểm định 15,3%: khá vừa; kiểm định gần phẳng từ n_s = 480.

(c) MLP lớn: khoảng cách 15,7 điểm ở n_s = 240, thu hẹp dần ⇒ phương sai cao; thêm dữ liệu hoặc L2 giúp.

Hình 27. Lỗi huấn luyện (xanh) và lỗi kiểm định (cam) theo số mẫu huấn luyện; đường xanh lá nét đứt là mức tham chiếu 13,1%.

Hai nguồn lỗi

thiên lệch = lỗi huấn luyện − lỗi tham chiếu
phương sai = lỗi kiểm định − lỗi huấn luyện

Thiên lệch lớn

Huấn luyện **cũng sai nhiều**: mô hình quá đơn giản ⇒ **thiếu khớp**.

Phương sai lớn

Tốt trên dữ liệu đã thấy, **kém trên dữ liệu mới ⇒ quá khớp**.

(a) Softmax

50,0 − 13,1 ≈ 37 điểm; kiểm định ≈ huấn luyện ⇒ **thiên lệch cao**.

(b) MLP 2–8–3

Thiên lệch ≈ 0; phương sai 3,5 điểm ⇒ **khá vừa**.

(c) MLP lớn

Phương sai 15,7 điểm (n_s = 240) ⇒ **phương sai cao**.

Đây là ước lượng thực dụng, không phải định nghĩa hình thức. Không nhầm "thiên lệch" này với **hệ số chệch** b của nơ-ron.

5.4 · BẢNG CHẨN ĐOÁN

Từ chẩn đoán đến hành động

Lỗi huấn luyện	Lỗi kiểm định	Chẩn đoán	Cách xử lý
Cao (xa mức tham chiếu)	Cao, gần lỗi huấn luyện	**Thiên lệch cao (thiếu khớp)**	Mạng lớn hơn hoặc sâu hơn, đặc trưng tốt hơn, huấn luyện lâu hơn, giảm λ
Thấp	Cao hơn nhiều	**Phương sai cao (quá khớp)**	Thêm dữ liệu, L2, dropout, dừng sớm, mạng nhỏ hơn
Cao	Cao hơn nhiều	**Cả hai**	Xử lý thiên lệch trước, rồi phương sai
Thấp, gần mức tham chiếu	Gần lỗi huấn luyện	**Tốt**	Dừng, hoặc tinh chỉnh

Câu 5

Lỗi huấn luyện 2%, kiểm định 15%, tham chiếu 3% ⇒ phương sai = 13 điểm ⇒ **quá khớp**.

Câu 6

Lỗi huấn luyện 18%, kiểm định 19%, tham chiếu 4% ⇒ thiên lệch = 14 điểm ⇒ **thiếu khớp**; thêm dữ liệu gần như không giúp.

5.4 · XEM CÁC MẪU BỊ SAI

Lỗi nằm ở đâu?

46/300 mẫu kiểm tra bị sai (mô hình cuối)

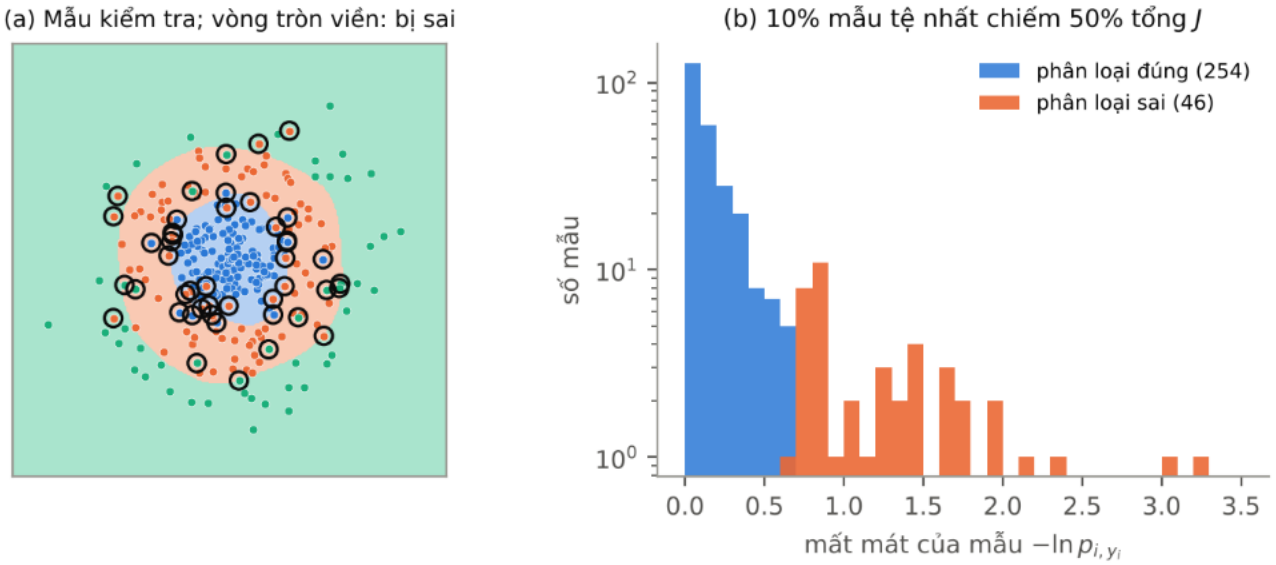
×9 mất mát trung bình nhóm sai 1,383 so với nhóm đúng 0,149

10% → 50% 30 mẫu tệ nhất chiếm một nửa tổng J (Hình 28b)

1. **Sắp mẫu theo mất mát giảm dần**, xem kỹ vài chục mẫu đầu: nhãn sai thật, mẫu mơ hồ, hay thiếu đặc trưng?
2. **Xem lỗi theo từng lớp** (ma trận nhầm lẫn): lỗi dồn vào cặp lớp nào, có hệ thống không?
3. **So với mức tham chiếu**: còn lớn hơn nhiều thì còn chỗ cải thiện; sát rồi thì phần còn lại phần lớn là nhiễu.
4. **Kiểm tra đầu vào**: giá trị thiếu, chưa chuẩn hóa, rò rỉ nhãn.

5.4 · HÌNH 28

Mẫu sai nằm ở biên giữa các vành



(a) mẫu sai (khoanh tròn) nằm gần như toàn bộ **trên biên giữa các vành**, nơi hai phân phối chồng lên nhau (độ nhòe 0,7 so với khoảng cách bán kính 1,8).

(b) phân bố mất mát −ln p của từng mẫu (trục tung log): mẫu sai nằm ở **đuôi phải**.

Kết luận: đây là **nhiễu nội tại**; đừng ép mạng lớn hơn để "sửa" chúng, sẽ dẫn tới quá khớp (mục 4.4).

Hình 28. (a) 300 mẫu kiểm tra trên vùng quyết định của mô hình cuối cùng; mẫu bị sai được khoanh tròn. (b) Phân bố mất mát của từng mẫu.

5.5 · MLPCLASSIFIER

Dùng thư viện làm đối chứng

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (classification_report,
                 confusion_matrix, log_loss)
clf = MLPClassifier(hidden_layer_sizes=(16,),
    activation='tanh', solver='adam',
    alpha=1e-3 * 32, batch_size=32,
    learning_rate_init=0.01, max_iter=200,
    tol=0, n_iter_no_change=1000, random_state=0)
clf.fit(Xtr, ytr)
P = clf.predict_proba(Xkt)
print(confusion_matrix(ykt, P.argmax(axis=1)))
print(classification_report(ykt, P.argmax(axis=1), digits=3))
print('log-loss:', log_loss(ykt, P))
```

Mục đích

Cài đặt từ đầu có sẵn trong scikit-learn: dùng để **kiểm tra cài đặt của mình** và làm đường cơ sở nhanh.

Hai dòng khó

Dòng vàng: **alpha = λ·b** (xem slide sau) và **tắt dừng sớm** (tol = 0, n_iter_no_change lớn) để so công bằng.

Cấu hình

Tương ứng mạng 2–16–3: tanh, Adam, mini-batch 32, 200 epoch, L2 $\lambda = 10^{-3}$.

5.5 · BẢNG ĐỐI CHIẾU THAM SỐ

Tham số nào ứng với tham số nào?

MLPClassifier	Trong mã của ta	Ghi chú
hidden_layer_sizes=(16,)	kich_thuoc = [2, 16, 3]	Lớp vào và ra suy ra từ dữ liệu
activation='tanh'	kich_hoat='tanh'	Còn 'relu', 'logistic'
solver='adam'	thuat_toan='adam'	Còn 'sgd' (kèm momentum)
learning_rate_init	η	
batch_size	b	
max_iter	số epoch	Đếm theo epoch, không theo bước
alpha	**λ**	**Khác thang đo**
tol=0, n_iter_no_change lớn	Không dừng sớm	Mặc định dừng sau 10 epoch không giảm

5.5 · HỆ SỐ ALPHA

alpha không phải λ

Thư viện: thêm $(\alpha / 2b) \Sigma_l \|V^{(l)}\|^2_F$ cho mỗi mini-batch b mẫu

Giáo trình (mục 4.4): $(\lambda / 2) \Sigma_l \|V^{(l)}\|^2_F$

⇒ $\lambda = \alpha / b$, tức alpha = $\lambda \cdot b$

Ví dụ

$\lambda = 10^{-3}$, b = 32 ⇒ **alpha = 0,032** (mã: alpha=1e-3 * 32). Câu 9: λ = 0,01, b = 64 ⇒ alpha = **0,64**.

Lưu ý

Hệ số chệch không bị phạt (cả hai bên). Đây là chỗ **rất dễ nhầm** khi so sánh hai cài đặt.

Khác biệt còn lại: scikit-learn khởi tạo kiểu Xavier phân phối đều và cả hệ số chệch cũng khởi tạo ngẫu nhiên (mã ta: chệch bắt đầu bằng 0); thứ tự xáo trộn mẫu khác. Vì vậy kết quả cùng cỡ nhưng không giống hệt.

5.5 · BÁO CÁO PHÂN LOẠI

Chỉ số của ta khớp với thư viện

```
              precision recall f1-score support
           0 0.885 0.927 0.906 150
           1 0.776 0.760 0.768 100
           2 0.867 0.780 0.821 50
    accuracy 0.847 300
   macro avg 0.843 0.822 0.831 300
weighted avg 0.846 0.847 0.846 300
```

Khớp tuyệt đối

Đây là kết quả của mô hình **của ta** (5.3) đưa qua **classification_report**: số khớp tuyệt đối với bảng ở 5.3.

Ba số khớp

Accuracy 0,847; F1 macro 0,831; log-loss 0,338 cho cả hai cách tính.

Kết luận

Vậy **ma_tran_nham_lan** và **chi_so_theo_lop** của ta đúng.

Cách đọc: **support** = số mẫu thật của lớp; **macro avg** = trung bình đơn giản các lớp; **weighted avg** = trung bình theo support.

5.5 · SO SÁNH

Mạng, thư viện và Softmax

Mô hình (900 mẫu huấn luyện)	Accuracy	F1 macro	Log-loss	Thời gian
Softmax Regression, đặc trưng gốc	**50,0%**	**0,222**	1,003	0,2 s
Softmax, đặc trưng đa thức bậc 2	**86,3%**	0,849	0,345	0,2 s
MLP 2–16–3 của ta	85,1% ± 1,0%	0,832 ± 0,010	0,347 ± 0,004	0,6 s
MLPClassifier	85,5% ± 0,7%	0,836 ± 0,007	0,343 ± 0,004	1,2 s

Cài đặt đúng

Mạng của ta và MLPClassifier cùng cỡ (85,1% và 85,5%, chênh trong độ lệch chuẩn). Softmax của ta khớp hồi quy logistic của scikit-learn: 0,863; 0,849; 0,345.

Softmax gốc thất bại

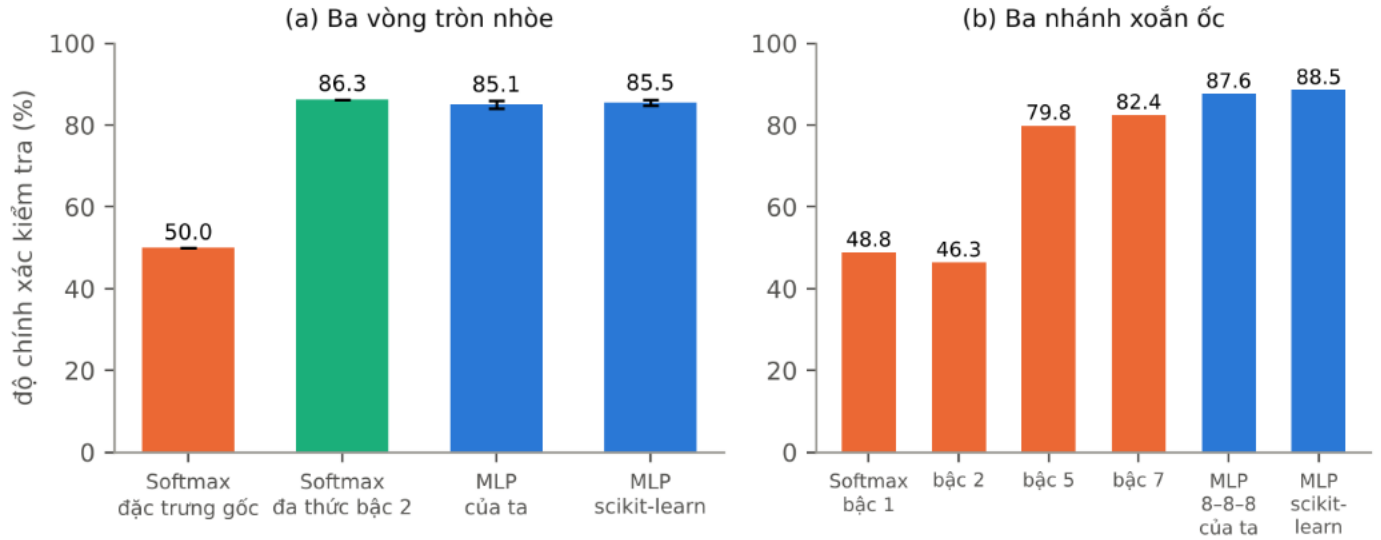
Accuracy 50,0% = mốc tầm thường; F1 macro 0,222 lộ ra điều đó.

Bậc 2 ngang mạng

Ranh giới là đường tròn $x_1^2 + x_2^2 = r^2$, nằm gọn trong lớp đặc trưng bậc hai. Biết đúng đặc trưng ⇒ mô hình tuyến tính ngang mạng, nhanh và ổn định hơn.

5.5 · HÌNH 29

Độ chính xác kiểm tra của các mô hình



(a) Ba vòng tròn: Softmax gốc 50%; Softmax bậc 2 và hai cài đặt mạng cùng ≈ 85–86%; thanh sai số là độ lệch chuẩn qua 5 khởi tạo.

(b) Xoắn ốc: Softmax bậc 1, 2, 5, 7 và hai mạng 8–8–8: bậc thấp thất bại, bậc 7 đạt 82,4%, mạng đạt 87,6–88,5%.

Thông điệp: biết đúng đặc trưng ⇒ Softmax ngang mạng; không biết ⇒ mạng vượt rõ rệt.

Hình 29. Độ chính xác kiểm tra. (a) Ba vòng tròn nhòe (300 mẫu kiểm tra). (b) Ba nhánh xoắn ốc (1200 mẫu kiểm tra).

5.5 · KHI MẠNG THẬT SỰ CÓ LỢI

Ba nhánh xoắn ốc

Mô hình (xoắn ốc: 600 huấn luyện, 1200 kiểm tra)	Số tham số	Accuracy kiểm tra
Softmax, bậc 1 (đặc trưng gốc)	9	48,8%
Softmax, bậc 2	18	46,3%
Softmax, bậc 3	30	**35,6%**
Softmax, bậc 5	63	79,8%
Softmax, bậc 7	108	82,4%
MLP 2–8–8–8–3 của ta	195	**87,6%**
MLPClassifier (8, 8, 8)	195	**88,5%**

Đa thức bậc thấp

Bậc 2 và 3 không bắt được hình xoắn (có khi còn kém đặc trưng gốc). Phải lên bậc 5 mới tốt lên. Số đặc trưng bậc d: C(d+2, 2) − 1.

Mạng

Mạng 195 tham số, Adam η = 0,01, mini-batch 32, 1000 epoch, không điều chuẩn hóa. Nhiều tham số hơn bậc 7 (108), nên ưu thế là **đặc trưng có cấu trúc đúng**, không phải “ít tham số hơn”.

5.5 · KHI NÀO DÙNG CÁI NÀO?

Chọn công cụ theo nhu cầu

Nhu cầu	Gợi ý
Hiểu cơ chế, thử ý tưởng, dạy học	**Cài đặt từ đầu (mục 3.7)**
Đường cơ sở nhanh trên dữ liệu bảng cỡ vừa	**MLPClassifier (và so với LogisticRegression có đặc trưng đa thức)**
Dữ liệu lớn, mạng sâu, GPU, kiến trúc đặc thù (CNN, RNN, Transformer)	**PyTorch hoặc TensorFlow (mục 6.3)**

Khi nào chọn mô hình tuyến tính?

Biết đúng đặc trưng (ví dụ bậc 2 cho các vòng tròn): Softmax đơn giản ngang mạng, nhanh và ổn định hơn (lồi, không phụ thuộc khởi tạo).

Khi nào chọn mạng?

Không biết trước đặc trưng (xoắn ốc, ảnh, văn bản): mạng tự học đặc trưng ⇒ vượt rõ rệt.

5.5 · QUY TRÌNH ĐÁNH GIÁ TÓM TẮT

Sáu bước cho một lần huấn luyện nghiêm túc

1. Chia phân tầng; chuẩn hóa bằng thống kê của **tập huấn luyện**.
2. Huấn luyện **mô hình đơn giản làm đường cơ sở** (mốc tầm thường, Softmax Regression) để biết “tốt” là bao nhiêu.
3. Huấn luyện mạng, theo dõi **đường cong học** và chuẩn gradient (mục 4.3), kiểm tra vùng quyết định nếu vẽ được.
4. Chọn siêu tham số trên tập kiểm định hoặc kiểm định chéo; **ưu tiên mô hình đơn giản**.
5. Chẩn đoán thiên lệch và phương sai bằng đường cong học theo n_s; **xem các mẫu sai**.
6. Báo cáo **một lần** trên tập kiểm tra: ma trận nhầm lẫn, F1 từng lớp, log-loss, kèm khoảng tin cậy.

MỐI LIÊN HỆ

Nối mục trước và mục sau

Mục 4: cơ chế học
Không lồi, cao nguyên, gradient biến mất, quá khớp.

→

Mục 5: đo và chẩn đoán
Đường cong học, chia tập, chỉ số, thiên lệch/phương sai, đối chiếu thư viện.

→

Mục 6: tổng kết
Ưu nhược điểm; so bốn mô hình; hướng tới học sâu; bài tập.

Cao nguyên (4.2) hiện ra ở đường cong accuracy: 50% vài epoch đầu.

Quá khớp (4.4) hiện ra ở khoảng cách J huấn luyện – kiểm định và ở log-loss.

Biểu diễn lớp ẩn là hình ảnh cụ thể của ý tưởng mục 1.2: mạng tự học đặc trưng.

Bảng 5.5 là bước đầu của phần so sánh bốn mô hình ở mục 6.2.

TÓM TẮT MỤC 5 · PHẦN 1/2

Quan sát, chia dữ liệu, chọn siêu tham số

1. Quan sát quá trình học

Đường cong J và accuracy theo epoch cho biết học tốt, quá khớp, thiếu khớp hay không ổn định. Vùng quyết định đi từ “luôn đoán lớp đông nhất” đến ba vành cong. Ở lớp ẩn cuối, các lớp tách bằng đường thẳng: mạng đã tự học đặc trưng cho Softmax.

2. Chia dữ liệu

Chia phân tầng thành huấn luyện / kiểm định / kiểm tra; chọn mọi thứ trên kiểm định, chạm tập kiểm tra một lần. Thống kê chuẩn hóa chỉ từ tập huấn luyện. Kiểm định chéo K lần khi dữ liệu ít.

3. Chọn siêu tham số

Tìm kiếm lưới trên tập kiểm định; SE = √(p(1−p)/n) nên chênh lệch vài điểm trên vài trăm mẫu là nhiễu; quy tắc một SE chọn mô hình đơn giản nhất. 20 cách chia cho 82,7% đến 89,3%.

TÓM TẮT MỤC 5 · PHẦN 2/2

Chỉ số, chẩn đoán, so sánh

4. Chỉ số

Ma trận nhầm lẫn $\mathbf{C} = \mathbf{Y}^{\mathsf{T}}\hat{\mathbf{Y}}$; precision theo cột, recall theo hàng, F1 là trung bình điều hòa; F1 macro cho lớp nhỏ trọng lượng ngang lớp lớn. Log-loss đo cả độ tự tin và nhạy với quá khớp hơn accuracy. Mốc tầm thường: 50%, F1 macro 0,222.

5. Chẩn đoán

Đường cong học theo số mẫu: thiên lệch cao (hai đường gặp nhau ở mức cao ⇒ cần mô hình mạnh hơn) hay phương sai cao (khoảng cách lớn ⇒ cần dữ liệu hoặc điều chuẩn hóa). Một ít mẫu chiếm nửa J; lỗi dồn vào biên chồng lấn là nhiễu nội tại.

6. So sánh

Mạng của ta khớp MLPClassifier (chú ý **alpha = λ·b**); Softmax của ta khớp hồi quy logistic của scikit-learn. Biết đúng đặc trưng ⇒ Softmax ngang mạng; không biết (xoắn ốc) ⇒ mạng vượt rõ rệt.

KIỂM TRA NHANH · CÂU 1 ĐẾN 6

Bạn đã nắm được chưa?

1. 1000 mẫu, 950 lớp A và 50 lớp B. Mô hình luôn đoán A. Tính accuracy và F1 macro.
2. Cho **C** = [[50, 5, 5], [10, 30, 10], [0, 5, 35]]. Tính accuracy, precision, recall, F1 từng lớp và F1 macro.
3. Xác suất gán cho nhãn thật của 4 mẫu: 0,9; 0,8; 0,1; 0,6. Tính log-loss. Nếu mẫu thứ ba là 0,01 thì log-loss thay đổi thế nào?
4. Một mô hình đạt accuracy 80% trên tập kiểm tra 400 mẫu. Tính khoảng tin cậy 95%.
5. Lỗi huấn luyện 2%, kiểm định 15%, tham chiếu 3%. Chẩn đoán và đề xuất ít nhất hai cách xử lý.
6. Lỗi huấn luyện 18%, kiểm định 19%, tham chiếu 4%. Chẩn đoán. Thu thập thêm dữ liệu có giúp không?

KIỂM TRA NHANH · CÂU 7 ĐẾN 12

Bạn đã nắm được chưa?

7 Tốt nhất 86,0% với SE = 2,0 điểm. Theo quy tắc một sai số chuẩn, chọn m nhỏ nhất nào, biết m = 4 đạt 85,6% và m = 2 đạt 70,6%?

8 Kiểm định chéo 5 lần cho 0,85; 0,90; 0,88; 0,84; 0,86. Tính trung bình và độ lệch chuẩn (chia cho N = 5). Với 1200 mẫu, mỗi lần huấn luyện và kiểm định bao nhiêu mẫu?

9 Muốn dùng λ = 0,01 trong MLPClassifier với batch_size = 64, cần đặt alpha bằng bao nhiêu?

10 Tính số tham số của mạng 2–16–3.

11 Một bạn chuẩn hóa bằng trung bình và độ lệch chuẩn của **toàn bộ** 1500 mẫu rồi mới chia ba tập. Có vấn đề gì? Sửa thế nào?

12 Giải thích vì sao ở Hình 24b các lớp tách được bằng đường thẳng dù ở Hình 24a thì không.

ĐÁP ÁN · CÂU 1 ĐẾN 6

Lời giải ngắn gọn

1 Accuracy = 950/1000 = **95%**. Lớp A: Prec = 0,95, Rec = 1 ⇒ F1 = 0,9744. Lớp B: Rec = 0, F1 = 0. F1 macro = **0,487**: accuracy cao nhưng vô dụng với lớp B.

2 Tổng cột (60, 40, 50), tổng hàng (60, 50, 40), n = 150. Accuracy = 115/150 = **0,767**. Prec = (0,833; 0,750; 0,700); Rec = (0,833; 0,600; 0,875); F1 = (0,833; 0,667; 0,778); F1 macro = **0,759**.

3 0,105 + 0,223 + 2,303 + 0,511 = 3,142 ⇒ log-loss = **0,785**. Với 0,01: −ln 0,01 = 4,605 ⇒ (0,105 + 0,223 + 4,605 + 0,511)/4 = **1,361**: một mẫu tự tin sai làm log-loss gần gấp đôi.

4 SE = √(0,8·0,2/400) = 0,02. Khoảng = 0,80 ± 1,96·0,02 = 0,80 ± 0,039 ⇒ **76,1% đến 83,9%**.

5 Thiên lệch = 2 − 3 < 0 (không có); phương sai = 15 − 2 = 13 điểm ⇒ **phương sai cao (quá khớp)**. Xử lý: thêm dữ liệu, L2 hoặc dropout, dừng sớm, mạng nhỏ hơn.

6 Thiên lệch = 18 − 4 = 14 điểm; phương sai = 1 điểm ⇒ **thiên lệch cao (thiếu khớp)**. Thêm dữ liệu gần như không giúp; cần mô hình mạnh hơn (lớn/sâu hơn, đặc trưng tốt hơn, huấn luyện lâu hơn, giảm λ).

ĐÁP ÁN · CÂU 7 ĐẾN 12

Lời giải ngắn gọn

7 Ngưỡng = 86,0 − 2,0 = 84,0%. m = 2 (70,6%) dưới ngưỡng; m = 4 (85,6%) trên ngưỡng ⇒ chọn **m = 4** (λ = 0).

8 Trung bình = 4,33/5 = **0,866**; độ lệch chuẩn = $\sqrt{(1/5)\Sigma(a^f - 0{,}866)^2}$ = **0,0215**. Mỗi lần: huấn luyện **960** mẫu, kiểm định **240** mẫu.

9 alpha = λ·b = 0,01 × 64 = **0,64**.

10 (2 + 1)·16 + (16 + 1)·3 = 48 + 51 = **99** tham số.

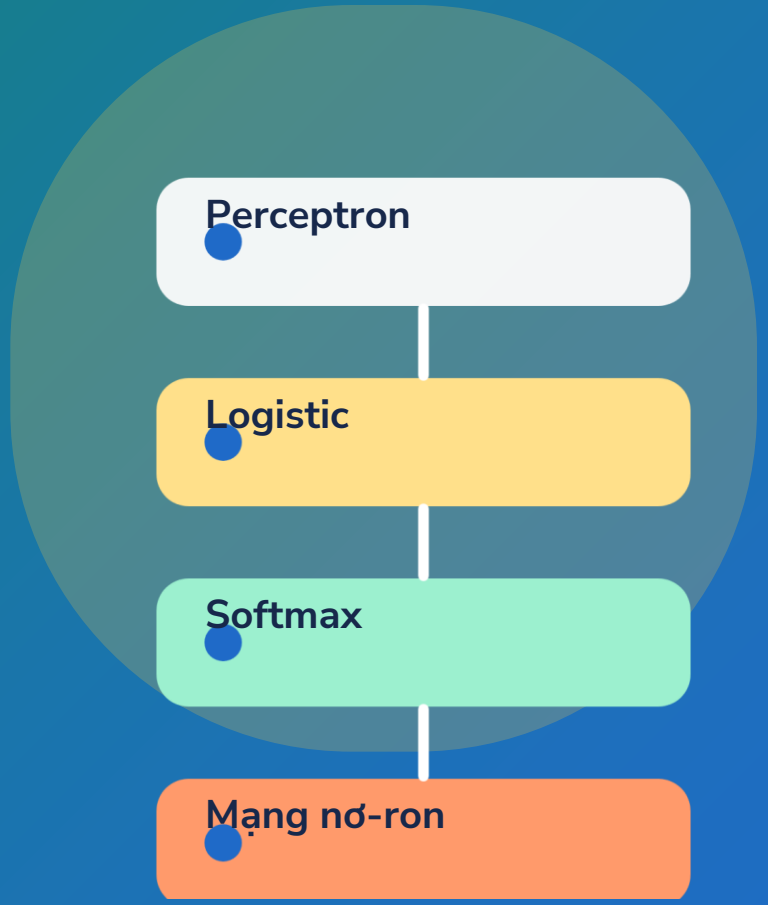
11 **Rò rỉ dữ liệu**: thống kê của tập kiểm tra ảnh hưởng phép chuẩn hóa khi huấn luyện ⇒ kết quả hơi lạc quan. Sửa: chia trước, tính μ, σ chỉ từ tập huấn luyện, dùng chung cho cả ba tập.

12 Điểm số lớp ra **s** = [1, **h**]$\mathbf{W}^{(L)}$ tuyến tính theo **h** ⇒ vùng thắng bị chặn bởi đường thẳng. Các lớp ẩn trước biến đổi mặt phẳng đầu vào để các lớp tách được bằng đường thẳng ở không gian mới.

CHƯƠNG: MẠNG NƠ-RON NHÂN TẠO · MỤC 6

Tổng kết và mở rộng

Mạng nơ-ron được gì, mất gì, và con đường đến học sâu



KẾ HOẠCH

Bốn phần của mục 6

6.1 Ưu điểm và hạn chế

Nói thẳng mạng nơ-ron được gì và mất gì, kèm quy tắc thực hành.

6.2 Bốn mô hình, một chuỗi

Perceptron, Logistic, Softmax, MLP: bảng tiêu chí và thí nghiệm trên bốn bộ dữ liệu.

6.3 Từ MLP đến học sâu

CNN, RNN, Transformer ở mức khái niệm; PyTorch và vi phân tự động.

6.4 Bài tập thực hành

Mười bài: sáu bài suy luận, bốn bài lập trình trên chữ số viết tay.

✓ **Kết thúc:** liên hệ toàn chương, tóm tắt, câu hỏi kiểm tra nhanh và đáp án.

MỞ ĐẦU

Năm mục trước, mỗi mục một câu hỏi

1 **Vì sao cần mạng?** Mô hình tuyến tính không giải được XOR, vòng tròn, xoắn ốc.

2 **Mạng gồm gì?** Lan truyền thuận, hàm kích hoạt, softmax, cross-entropy.

3 **Học thế nào?** Lan truyền ngược: tính $D^{(L)}$ rồi đi lùi từng lớp; mini-batch, Adam.

4 **Vì sao đôi khi thất bại?** Không lồi, bão hòa, quá khớp, rộng hay sâu.

5 **Làm sao biết mô hình tốt?** Đường cong học, chia tập, ma trận nhầm lẫn, thiên lệch và phương sai.

6 **Hôm nay:** nhìn lại toàn bộ, so sánh với mô hình quen thuộc, nhìn về phía trước.

KÝ HIỆU BỔ SUNG CHO MỤC 6

Năm ký hiệu mới, cùng quy ước cũ

$\sigma(z)$, softmax(z)	Hàm sigmoid và softmax, như mục 1 và mục 2.
F	Bộ lọc tích chập: ma trận nhỏ (ví dụ 3 × 3), tham số cần học của lớp tích chập.
h_t	Trạng thái ẩn tại bước t: vector "bộ nhớ" của mạng hồi quy.
Q, K, R	Truy vấn, khóa, giá trị trong cơ chế chú ý. Dùng **R** thay cho "value" để không trùng **V** của mục 3.
Ntham số	Tổng số phần tử của mọi ma trận trọng số và độ lệch.

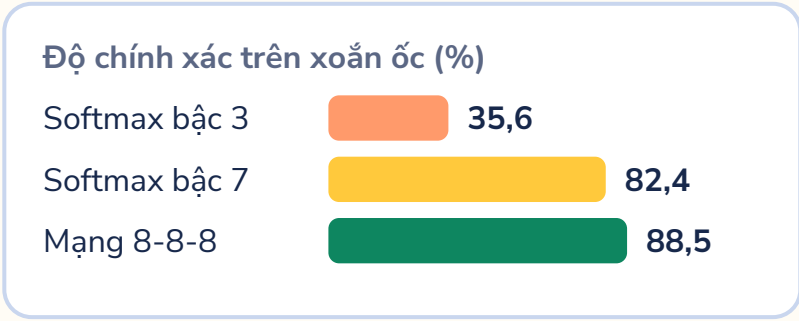
Quy ước chung: ma trận là chữ hoa in đậm (**X**, **W**), vector là chữ thường in đậm (**x**, **w**).

6.1 · ƯU ĐIỂM 1 VÀ 2

Biểu diễn phi tuyến và cơ sở lý thuyết

1. Ranh giới phi tuyến, không cần biết trước dạng

Softmax chỉ thắng ba vòng tròn khi ta **biết** phải thêm đặc trưng bậc 2. Với ba nhánh xoắn ốc, đa thức bậc 7 mới đạt 82,4% (bậc 3 chỉ 35,6%), còn mạng 8-8-8 đạt 87,6% đến 88,5% mà không ai thiết kế đặc trưng. Các lớp ẩn tự học đặc trưng; lớp cuối là một Softmax Regression.



2. Có cơ sở lý thuyết cho khả năng biểu diễn

Định lý xấp xỉ phổ quát: một lớp ẩn đủ rộng xấp xỉ được mọi hàm liên tục trên tập compact. Mục 4.5 bổ sung: **độ sâu** làm số vùng tuyến tính tăng theo hàm mũ (2k nơ-ron ReLU xếp k lớp tạo 2^k đoạn), còn độ rộng chỉ tăng tuyến tính.

6.1 · ƯU ĐIỂM 3, 4 VÀ 5

Khung chung, huấn luyện rẻ, mở rộng được

3. Một khung chung

Chỉ đổi lớp ra và hàm mất mát:

sigmoid + cross-entropy nhị phân

softmax + cross-entropy (nhiều lớp)

đầu ra tuyến tính + sai số bình phương (hồi quy)

Lan truyền ngược không đổi: $D^{(L)}$ rồi đi lùi từng lớp.

4. Huấn luyện hiệu quả

Chi phí tính **toàn bộ** gradient chỉ cỡ 2 đến 3 lần một lần lan truyền thuận, dù có hàng trăm nghìn tham số.

Nhờ vậy mạng lớn mới huấn luyện được.

Gradient còn kiểm tra được bằng sai phân hữu hạn (mục 3.4).

5. Mở rộng theo kiến trúc

Mỗi khối chỉ cần **khả vi**, nên có thể thay phép nhân ma trận đầy đủ bằng:

phép tích chập

đệ quy theo thời gian

cơ chế chú ý

mà không đổi thuật toán học (mục 6.3).

Ý chung: chỉ cần mỗi khối khả vi, lan truyền ngược lo phần còn lại.

6.1 · HẠN CHẾ 1 VÀ 2

Không lồi, gradient biến mất hoặc bùng nổ

1. Hàm mất mát không lồi

Softmax Regression có một cực tiểu toàn cục duy nhất nên không phụ thuộc khởi tạo. Mạng nơ-ron có điểm yên ngựa và nhiều cực tiểu tương đương do hoán vị (mục 4.2). Mạng XOR 2-2-1 đạt $J < 0{,}05$ ở bao nhiêu trong 40 khởi tạo?

34/40	39/40	40/40	40/40
m = 2	m = 3	m = 4	m = 8

2. Gradient biến mất hoặc bùng nổ ở mạng sâu

Chuẩn tín hiệu lỗi bị chặn bởi tích các $\|\mathbf{V}\|_2$ qua từng lớp (mục 4.3). Mạng sigmoid 8 lớp thất bại; ReLU với khởi tạo He thì học tốt.

43%	33,3%	99% đến 100%
sigmoid + Adam	sigmoid + SGD	ReLU + khởi tạo He

6.1 · HẠN CHẾ 3 VÀ 4

Dễ quá khớp, nhiều siêu tham số

3. Dễ quá khớp: số tham số lớn so với số mẫu

66,3% → 72,1%

60 mẫu (mục 4.4): không điều chuẩn hóa, rồi thêm L2. Mốc tham chiếu 75,3%.

100% so với 88,2%

Chữ số viết tay, 100 mẫu, mạng 64-64-10: độ chính xác huấn luyện so với kiểm tra.

Cần các công cụ của mục 4.4 và mục 5.4.

4. Nhiều siêu tham số

số lớp · số nơ-ron · hàm kích hoạt · tốc độ học η · mini-batch · hệ số L2 λ · dropout p · số epoch

Mỗi siêu tham số chọn bằng tập kiểm định (mục 5.2). Sai số chuẩn SE = $\sqrt{p(1-p)/n}$ khá lớn, nên chênh lệch nhỏ giữa các cấu hình thường chỉ là **nhiễu**.

6.1 · HẠN CHẾ 5, 6 VÀ 7

Chi phí, diễn giải, không phải lúc nào cũng thắng

5. Chi phí tính toán

0,6 s so với 0,2 s

Mạng của ta so với Softmax Regression trên cùng bài toán; khoảng cách tăng nhanh theo kích thước. Mạng lớn cần nhiều dữ liệu và phần cứng chuyên dụng.

6. Khó diễn giải

Softmax: hệ số nói được đặc trưng j làm tăng điểm số lớp k bao nhiêu.

Mạng nhiều lớp: trọng số không đọc trực tiếp được. Chỉ có công cụ gián tiếp: vùng quyết định, biểu diễn lớp ẩn (5.1), xem mẫu sai (5.4).

7. Không phải lúc nào cũng thắng

Ba vòng tròn (mục 5.5): Softmax bậc 2 đạt **86,3%**, mạng đạt 85,1% đến 85,5%, mà rẻ hơn và dễ diễn giải hơn.

Dữ liệu bảng: mô hình cây thường ngang hoặc hơn (nhận định chung, chưa kiểm chứng trong chương).

Nhận xét: khi dữ liệu ít, quan hệ đã biết hoặc bài toán đơn giản, mô hình tuyến tính tốt thường đủ dùng.

6.1 · BẢNG TÓM TẮT

Được gì, mất gì: bốn tiêu chí

Tiêu chí	Ưu điểm	Hạn chế
Biểu diễn	Học ranh giới phi tuyến và đặc trưng	Không rõ trước cần bao nhiêu nơ-ron, bao nhiêu lớp
Tối ưu	Lan truyền ngược rẻ; gradient kiểm tra được	Không lồi: nhiều cực tiểu, điểm yên ngựa, phụ thuộc khởi tạo
Tổng quát hóa	Điều chuẩn hóa mạnh (L2, dropout, dừng sớm)	Dễ quá khớp khi dữ liệu ít
Vận hành	Một khung cho nhiều bài toán, mở rộng được	Nhiều siêu tham số, tốn tính toán, khó diễn giải

Quy tắc thực hành

Bắt đầu từ mô hình đơn giản nhất (Softmax Regression, có thể kèm đặc trưng đa thức) làm mốc. Chuyển sang mạng chỉ khi mô hình đơn giản **thiếu khớp** (thiên lệch cao, mục 5.4), và mạng phải **thắng mốc rõ ràng** theo tiêu chuẩn thống kê của mục 5.2.

6.2 · MỘT DẠNG CHUNG

Bốn mô hình: biến đổi tuyến tính rồi kích hoạt

Với mẫu $\mathbf{x}$ và $\tilde{\mathbf{x}} = [1, \mathbf{x}^T]^T$ (thêm số 1 cho độ lệch):

Perceptron (2 lớp, $y \in \{-1, +1\}$)

$\hat{y} = \text{sign}(\mathbf{w}^T\tilde{\mathbf{x}})$

Mẫu sai thì $\mathbf{w} \leftarrow \mathbf{w} + \eta\, y\, \tilde{\mathbf{x}}$

Logistic Regression (2 lớp, $y \in \{0, 1\}$)

$\hat{p} = \sigma(\mathbf{w}^T\tilde{\mathbf{x}})$

$J = -(1/n) \sum [y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$

Softmax Regression (K lớp)

$\mathbf{p} = \text{softmax}(\mathbf{W}^T\tilde{\mathbf{x}})$, $\mathbf{W} \in \mathbb{R}^{(d+1)\times K}$

Gradient $= (1/n)\, \tilde{\mathbf{X}}^T(\mathbf{P} - \mathbf{Y})$

Mạng nơ-ron (MLP, L lớp)

$\mathbf{Z}^{(l)} = \tilde{\mathbf{A}}^{(l-1)}\mathbf{W}^{(l)}$, $\mathbf{A}^{(l)} = \varphi(\mathbf{Z}^{(l)})$

$\mathbf{P} = \text{softmax}(\mathbf{Z}^{(L)})$, $\mathbf{A}^{(0)} = \mathbf{X}$

6.2 · QUAN HỆ GIỮA CÁC MÔ HÌNH

Mỗi mô hình là bước kế tiếp của mô hình trước

Perceptron
hàm bước, cập nhật theo mẫu sai

→

Logistic
sigmoid, mất mát trơn, có xác suất

→

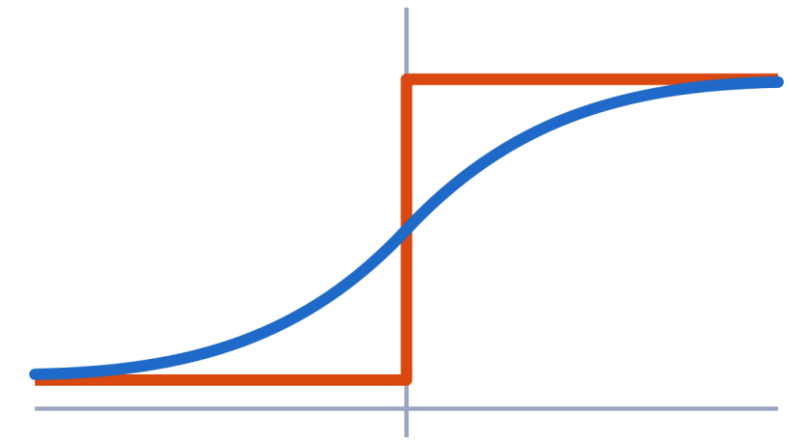
Softmax
K lớp, softmax, cross-entropy

→

Mạng nơ-ron
thêm lớp ẩn tự học đặc trưng

- Logistic Regression = Softmax Regression với **K = 2** (hai điểm số chỉ khác nhau một hằng số).
- Softmax Regression = mạng nơ-ron **không có lớp ẩn** (L = 1).
- Mạng nơ-ron = Softmax Regression trên biểu diễn $\mathbf{A}^{(L-1)}$ do các lớp ẩn **tự học** (mục 5.1).

Hàm bước không có gradient hữu ích



Đạo hàm của hàm bước bằng 0 ở hầu hết mọi nơi (không xác định tại điểm nhảy), nên gradient không cho biết hướng cải thiện.

Perceptron chỉ cập nhật theo **mẫu sai**, không có hàm mất mát trơn để cực tiểu hóa, và **không cho xác suất**.

Dữ liệu không tách được thì thuật toán **không bao giờ dừng**; kết quả cuối phụ thuộc mẫu gặp sau cùng.

Hệ quả: mạng nơ-ron hiện đại dùng kích hoạt khả vi (sigmoid, tanh, ReLU) và hàm mất mát trơn.

6.2 · BẢNG SO SÁNH (1/2)

Cấu trúc: lớp, đầu ra, mất mát, ranh giới

Tiêu chí	Perceptron	Logistic	Softmax	Mạng nơ-ron
Số lớp	Hai	Hai	Nhiều	Nhiều
Đầu ra	Nhãn ±1	Xác suất $\hat{p}$	Vector xác suất p	Vector xác suất p
Kích hoạt ra	Hàm bước	Sigmoid	Softmax	Softmax
Hàm mất mát	Không có (cập nhật theo mẫu sai)	Cross-entropy nhị phân	Cross-entropy	Cross-entropy
Ranh giới	Tuyến tính	Tuyến tính	Tuyến tính (từng đoạn thẳng giữa các lớp)	Phi tuyến

Ba mô hình đầu chỉ cho ranh giới thẳng. **Chỉ mạng nơ-ron** cho ranh giới cong.

6.2 · BẢNG SO SÁNH (2/2)

Vận hành: hội tụ, tham số, diễn giải, chi phí

Tiêu chí	Perceptron	Logistic	Softmax	Mạng nơ-ron
Hội tụ	Chỉ khi dữ liệu tách được tuyến tính	Mất mát lồi: cực tiểu toàn cục duy nhất	Mất mát lồi: cực tiểu toàn cục duy nhất	Không lồi: phụ thuộc khởi tạo
Số tham số	$d + 1$	$d + 1$	$(d + 1)K$	$\sum (m_{l-1} + 1)\, m_l$
Đặc trưng	Tự thiết kế	Tự thiết kế	Tự thiết kế	Tự học ở lớp ẩn
Diễn giải	Dễ	Dễ (hệ số, tỉ số odds)	Dễ	Khó
Chi phí huấn luyện	Rất thấp	Thấp	Thấp	Cao hơn

Quy ước: $m_0 = d$ và $m_L = K$. Ví dụ mạng 2-16-3 có $(2 + 1) \times 16 + (16 + 1) \times 3 = \mathbf{99}$ tham số.

6.2 · THÍ NGHIỆM

Cùng bốn bộ dữ liệu, cùng bốn mô hình

Bốn bộ dữ liệu, độ khó tăng dần

Ba cụm tách được: 600 huấn luyện, 600 kiểm tra.

XOR có nhiễu: 400 và 400. Ví dụ kinh điển mô hình tuyến tính không giải được.

Ba vòng tròn nhòe (mục 5): 1500 và 1500. Các vòng chồng lấn.

Ba nhánh xoắn ốc (mục 4): 600 huấn luyện, 1200 kiểm tra.

Cách cài đặt

Perceptron, Logistic: mô hình hai lớp, dùng **một-với-tất-cả** (OvR): K bộ phân loại, chọn lớp có điểm số lớn nhất.

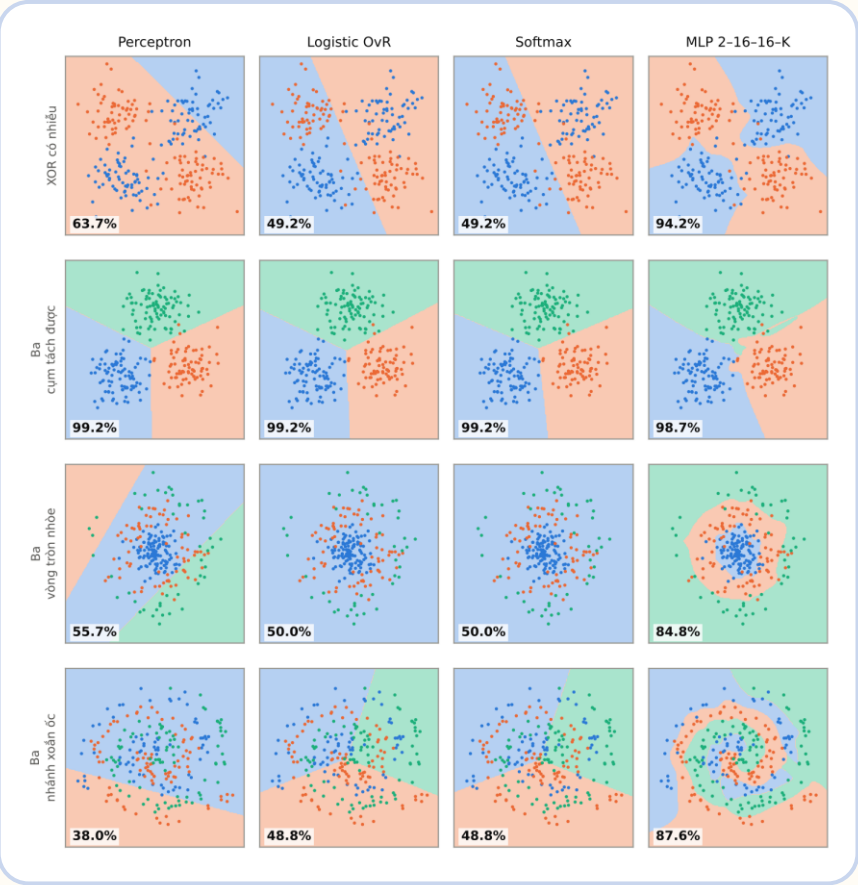
Softmax: cài đặt của mục 1, gradient descent đủ epoch.

Mạng nơ-ron: 2-16-16-K, tanh, Adam như mục 3 và 4.

Báo cáo: độ chính xác trên tập kiểm tra riêng; Perceptron và mạng chạy 5 hạt giống (trung bình ± độ lệch chuẩn).

6.2 · HÌNH 30

Vùng quyết định của bốn mô hình



Cách đọc: mỗi hàng một bộ dữ liệu (XOR, ba cụm, ba vòng tròn, xoắn ốc), mỗi cột một mô hình. Mỗi màu là vùng một lớp thắng; số ở góc là độ chính xác kiểm tra.

Ba cột đầu: ranh giới là đường thẳng. Trên vòng tròn, Logistic và Softmax tô gần như cả mặt phẳng một màu: 50,0%.

Cột MLP: ranh giới cong bám theo dữ liệu: bốn góc của XOR, ba vành đồng tâm, hai nhánh xoắn ốc cuộn vào nhau.

Lưu ý: hình vẽ hạt giống 0 (một lần chạy). Ví dụ Perceptron trên XOR 63,7% và MLP 94,2%; bảng kết quả là trung bình 5 hạt giống.

Hình 30. Vùng quyết định của bốn mô hình (cột) trên bốn bộ dữ liệu (hàng), hạt giống 0.

6.2 · KẾT QUẢ

Độ chính xác kiểm tra của bốn mô hình

Dữ liệu	Perceptron	Logistic OvR	Softmax	MLP 2-16-16-K
Ba cụm tách được	0,983 ± 0,012	0,992	0,992	0,983 ± 0,003
XOR có nhiễu	0,550 ± 0,050	0,492	0,492	0,944 ± 0,004
Ba vòng tròn nhòe	0,466 ± 0,130	0,500	0,500	0,847 ± 0,006
Ba nhánh xoắn ốc	0,380 ± 0,059	0,488	0,488	0,884 ± 0,006

Dữ liệu tách được: cả bốn mô hình ngang nhau (98% đến 99%).

Dữ liệu phi tuyến: ba mô hình tuyến tính đạt 0,38 đến 0,55; MLP đạt 0,847 đến 0,944.

Perceptron và MLP: trung bình trên 5 hạt giống ± độ lệch chuẩn. Logistic OvR và Softmax là tất định nên chỉ có một con số.

6.2 · ĐỌC KẾT QUẢ

Bốn nhận xét từ thí nghiệm

1. Dữ liệu tách được

Chênh 0,992 so với 0,983 chỉ cỡ hai lần sai số chuẩn (≈ 0,004). Không đủ cơ sở nói mô hình nào tốt hơn: **bài toán tuyến tính thì không có lý do dùng mô hình phức tạp hơn**.

2. Mô hình tuyến tính thiếu khớp

Logistic và Softmax chỉ đạt 0,49 đến 0,50: xấp xỉ đoán ngẫu nhiên hoặc đoán lớp đông nhất. Đây là **thiên lệch cao**, không phải huấn luyện kém: vùng tuyến tính không thể biểu diễn các ranh giới này.

3. Perceptron không ổn định

Vòng tròn: 0,466 ± 0,130; xoắn ốc: 0,380 ± 0,059, kém cả mô hình logistic. Chỉ cập nhật theo mẫu sai nên không bao giờ dừng khi dữ liệu không tách được. Vì thế ta thay bằng mất mát trơn.

4. MLP không phải lúc nào cũng hoàn hảo

Trên ba vòng tròn chồng lấn chỉ đạt 0,847, vì các vòng thực sự chồng lên nhau (nhiễu nội tại, mục 5.4). Không mô hình nào vượt được ngưỡng này.

Hạn chế của thí nghiệm: bốn bộ dữ liệu hai chiều, một cấu hình mạng cố định, không tinh chỉnh. Ta kết luận về *kiểu* hành vi, không về độ chính xác tuyệt đối.

6.2 · CHỌN MÔ HÌNH NÀO?

Quy trình chọn mô hình

1. **Làm mốc:** bắt đầu với Logistic hoặc Softmax Regression: nhanh, ổn định, có cực tiểu toàn cục, dễ diễn giải.

2. **Chẩn đoán:** dùng đường cong học và thiên lệch (mục 5.4). Lỗi huấn luyện **cũng cao** thì mô hình thiếu khớp: thêm đặc trưng (đa thức) hoặc chuyển sang mạng.

3. **Với mạng:** tăng dần kích thước, điều chuẩn hóa vừa đủ (mục 4.4), chọn siêu tham số bằng tập kiểm định (mục 5.2) và so với mốc ở bước 1 có tính sai số chuẩn.

4. **Perceptron:** chủ yếu có giá trị lịch sử và sư phạm, là mô hình đơn giản nhất cho thấy ý tưởng “cập nhật trọng số theo lỗi”.

Vì sao MLP chưa đủ?

Mọi thứ của chương (mất mát, lan truyền, mini-batch, điều chuẩn hóa, đánh giá) vẫn nguyên vẹn. Chỉ **cấu trúc mỗi lớp** thay đổi, để khai thác cấu trúc của dữ liệu.

Ảnh xám 28 × 28 vào MLP 784-128-64-10

$(784 + 1) \times 128 = 100\,480$

$(128 + 1) \times 64 = 8\,256$

$(64 + 1) \times 10 = 650$

Tổng: 109 386 tham số

Ảnh 224 × 224 ba kênh màu: riêng lớp đầu đã có hàng chục triệu tham số.

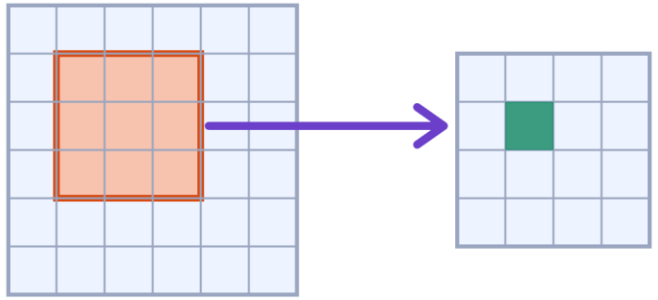
Vấn đề 1. Quá nhiều tham số: dễ quá khớp và tốn tính toán.

Vấn đề 2. Mất cấu trúc không gian: duỗi ảnh thì hai điểm ảnh kề nhau và hai điểm ảnh xa nhau bị đối xử như nhau. Chữ số dịch một điểm ảnh thành vector "xa lạ".

Văn bản là chuỗi có thứ tự; MLP với số đầu vào cố định không xử lý trực tiếp được.

Hướng giải: đưa giả định về cấu trúc dữ liệu vào mạng: CNN (lưới), RNN (chuỗi), Transformer (chú ý).

Một bộ lọc nhỏ trượt khắp ảnh



Ảnh 6 × 6, bộ lọc 3 × 3 → bản đồ đặc trưng 4 × 4

Đầu ra tại vị trí (i, j), bộ lọc F kích thước k × k

$z_{ij} = \sum_u \sum_v f_{uv} \cdot x_{i+u-1,\ j+v-1} + b$, rồi qua ReLU

Chia sẻ trọng số: cùng $k^2 + 1$ tham số dùng ở mọi vị trí. 32 bộ lọc 3 × 3 trên ảnh một kênh chỉ có (9 + 1) × 32 = **320** tham số, so với hơn 100 000 của lớp đầy đủ, bất kể kích thước ảnh.

Tương đương với phép dịch: dịch ảnh thì bản đồ đặc trưng dịch theo, mẫu được phát hiện ở mọi vị trí.

Gộp (pooling) lấy giá trị lớn nhất trong mỗi ô 2 × 2 để thu nhỏ bản đồ. Cuối cùng duỗi ra, qua lớp đầy đủ và softmax: **một Softmax Regression trên đặc trưng tự học**. Lan truyền ngược vẫn là quy tắc chuỗi; gradient theo **F** là tổng đóng góp của mọi vị trí bộ lọc trượt qua.

Trạng thái ẩn mang thông tin của quá khứ

Xử lý chuỗi $x_1, x_2, ..., x_T$ bằng trạng thái ẩn cập nhật tại từng bước

$$\mathbf{h}_t = \tanh(\mathbf{W}_x^T \mathbf{x}_t + \mathbf{W}_h^T \mathbf{h}_{t-1} + \mathbf{b})$$



Chia sẻ trọng số theo thời gian: cùng $\mathbf{W}_x$, $\mathbf{W}_h$ ở mọi bước. Đầu ra (phân loại cả chuỗi) lấy từ $\mathbf{h}_T$ qua softmax.

Gặp lại mục 4.3: trải theo thời gian được mạng T lớp dùng chung trọng số; tín hiệu lỗi bị nhân T lần với cùng $\mathbf{W}_h$ nên co về 0 hoặc phình ra.

Biện pháp: LSTM và GRU (có cổng) giữ tín hiệu ổn định hơn qua nhiều bước; cắt gradient (mục 4.4) tránh bùng nổ.

Mỗi vị trí nhìn trực tiếp mọi vị trí

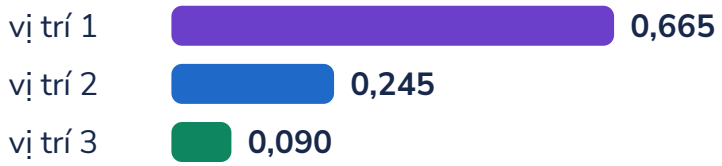
Ba phép biến đổi tuyến tính của chuỗi X (T × d)

$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{X}\mathbf{W}_K$, $\mathbf{R} = \mathbf{X}\mathbf{W}_R$

Chú ý

$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{R}) = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\,\mathbf{R}$

Ví dụ: điểm [2, 1, 0] → softmax



Softmax theo hàng: mỗi hàng của ma trận T × T là một phân phối xác suất, cho biết vị trí i lấy thông tin từ các vị trí khác theo tỉ lệ nào. Đầu ra là trung bình có trọng số các hàng của **R**.

Chia cho √dk: tránh điểm số quá lớn làm softmax bão hòa (cùng hiện tượng mục 4.3). Softmax xuất hiện *bên trong* mạng như cơ chế chọn lọc mềm.

Một khối Transformer = chú ý + MLP nhỏ cho từng vị trí + kết nối tắt; xếp chồng nhiều khối. Các vị trí xử lý song song nên huấn luyện hiệu quả hơn RNN; đây là nền tảng của các mô hình ngôn ngữ lớn.

6.3 · TÓM TẮT BA KIẾN TRÚC

Cùng ý tưởng, khác cấu trúc mỗi lớp

Kiến trúc	Giả định về dữ liệu	Thành phần thay cho ÃW	Ví dụ ứng dụng
MLP	Không có cấu trúc đặc biệt	Phép nhân ma trận đầy đủ	Dữ liệu bảng, phần phân loại cuối của các mạng khác
CNN	Cấu trúc lưới cục bộ (ảnh)	Tích chập với bộ lọc dùng chung	Nhận dạng ảnh
RNN	Chuỗi có thứ tự	Cập nhật trạng thái ẩn dùng chung theo thời gian	Chuỗi thời gian, văn bản (kiểu cũ)
Transformer	Chuỗi, mọi vị trí tương tác trực tiếp	Cơ chế chú ý (softmax trên điểm tương đồng)	Mô hình ngôn ngữ, ảnh, âm thanh

Mốc lịch sử (theo trí nhớ của tác giả): CNN kiểu LeNet từ thập niên 1980 đến 1990; LSTM cuối thập niên 1990; bước ngoặt học sâu trên ảnh quanh năm 2012; Transformer quanh năm 2017.

Cần đối chiếu tài liệu gốc trước khi trích dẫn các mốc năm.

6.3 · CÁC FRAMEWORK

Ngừng tự viết lan truyền ngược

Trong chương, ta tự viết lan truyền ngược bằng NumPy để hiểu thuật toán. Thực tế dùng PyTorch, TensorFlow hoặc JAX, với ba lợi ích:

1. Vi phân tự động

Chỉ viết lan truyền thuận; framework ghi đồ thị phép tính và áp dụng quy tắc chuỗi. Về bản chất là lan truyền ngược của mục 3, tổng quát cho mọi phép tính khả vi.

2. Tăng tốc phần cứng

Cùng một đoạn mã chạy trên GPU hoặc bộ tăng tốc chuyên dụng mà ít phải đổi.

3. Thư viện dựng sẵn

Lớp, hàm kích hoạt, bộ tối ưu (SGD, momentum, Adam), điều chuẩn hóa (L2, dropout) và công cụ đánh giá.

Biết lan truyền ngược "từ đầu" vẫn quan trọng: (i) gỡ lỗi (gradient biến mất, mất mát không giảm, quá khớp); (ii) hiểu các tùy chọn của framework (khởi tạo, bộ tối ưu, điều chuẩn hóa); (iii) đọc kiến trúc mới như tổ hợp của các khối quen thuộc.

Lưu ý trung thực: tác giả chưa cài được PyTorch nên mã ở slide sau **chưa được chạy**. Phần kiểm chứng bằng thực nghiệm là cài đặt NumPy (gradient trùng sai phân hữu hạn, mục 3.4). Các khẳng định về CNN, RNN, Transformer là kiến thức chung, không phải kết quả thí nghiệm của chương.

Lan truyền ngược gói trong một dòng

```
import torch, torch.nn as nn, torch.nn.functional as F
mang = nn.Sequential(
    nn.Linear(2, 16),   # (2 + 1) * 16 = 48 tham số
    nn.Tanh(),
    nn.Linear(16, 3),   # (16 + 1) * 3 = 51 tham số
)
toi_uu = torch.optim.Adam(mang.parameters(), lr=0.01)
for epoch in range(200):
    chi_so = torch.randperm(len(X))
    for k in range(0, len(X), 32):
        lo = chi_so[k:k + 32]
        toi_uu.zero_grad()
        loss = F.cross_entropy(mang(X[lo]), y[lo])
        loss.backward()
        toi_uu.step()
```

zero_grad(): xóa gradient của lần trước.

F.cross_entropy: nhận trực tiếp **điểm số z** (chưa softmax) và nhãn nguyên; tính softmax và cross-entropy ổn định số học (mục 2.4).

loss.backward(): toàn bộ lan truyền ngược của mục 3: việc tính $\mathbf{D}^{(l)}$ rồi nhân ngược.

step(): Adam cập nhật mọi $\mathbf{W}^{(l)}$ kèm độ lệch (mỗi lớp nn.Linear lưu cả hai).

Chưa được chạy trong môi trường của tác giả: hãy chạy thử và đối chiếu.

6.4 · BÀI TẬP SUY LUẬN (1/2)

Bài 1 đến 3: tham số, so sánh, chẩn đoán

1 **Tham số.** Tính số tham số của mạng 64-32-10 và mạng 64-64-10. So sánh với số mẫu huấn luyện (1074) và với Softmax Regression trên cùng dữ liệu (không lớp ẩn).

2 **So sánh mô hình.** Mô hình nào trong bảng mục 6.2 không thể giải XOR? Giải thích bằng ranh giới quyết định. Vì sao ba vòng tròn đồng tâm cũng không giải được bằng Softmax Regression trên đặc trưng gốc, nhưng giải được nếu thêm x_1^2, x_2^2?

3 **Chọn mô hình.** Softmax Regression: lỗi huấn luyện 12%, lỗi kiểm định 13%, lỗi tham chiếu 3%. Chẩn đoán và đề xuất bước tiếp theo. Sau khi chuyển sang mạng lớn: lỗi huấn luyện 0%, kiểm định 9%. Chẩn đoán lại và đề xuất hai cách xử lý.

Bài 4 đến 6: CNN, chú ý, RNN

4 **CNN và tham số.** Một lớp tích chập có 16 bộ lọc 5 × 5 trên ảnh một kênh. Tính số tham số. Một lớp đầy đủ nối ảnh 28 × 28 với 16 × 24 × 24 nơ-ron có bao nhiêu tham số? (Ảnh 28 × 28, bộ lọc 5 × 5, không đệm, cho bản đồ đặc trưng 24 × 24.)

5 **Chú ý.** Điểm tương đồng của một vị trí với ba vị trí trong chuỗi là $\mathbf{s} = [2, 1, 0]$ (đã chia cho $\sqrt{d_k}$). Tính softmax($\mathbf{s}$). Nếu nhân đôi thành $[4, 2, 0]$ thì các trọng số thay đổi thế nào, và điều đó nói gì về vai trò của hệ số $1/\sqrt{d_k}$?

6 **RNN.** Với RNN một chiều (h_t là số thực) $h_t = \tanh(w\, h_{t-1})$, bỏ đầu vào và độ lệch, $|w| < 1$: chỉ ra vì sao $\partial h_T / \partial h_0$ tiến về 0 khi T lớn. Gợi ý: quy tắc chuỗi và $|\tanh'| \le 1$.

Bài 7 đến 10 trên chữ số viết tay

Dữ liệu load_digits: 1797 ảnh xám 8 × 8 (vector 64 chiều, giá trị 0 đến 16), 10 chữ số. Chia phân tầng 60/20/20: 1074 huấn luyện, 359 kiểm định, 364 kiểm tra. Chia cho 16, rồi chuẩn hóa bằng thống kê của **riêng tập huấn luyện**. Mạng dùng tanh, softmax, cross-entropy.

7 **Mốc cơ bản.** Huấn luyện Softmax Regression và mạng 64-32-10; báo cáo độ chính xác kiểm tra. Chạy mạng với 5 hạt giống, báo cáo trung bình và độ lệch chuẩn.

8 **Ma trận nhầm lẫn C** = $\mathbf{Y}^{\mathrm{T}}\hat{\mathbf{Y}}$ trên tập kiểm tra (hạt giống 0). Hai chữ số nào bị nhầm nhiều nhất? Xem các ảnh bị sai bằng mắt.

9 **Bộ tối ưu.** Mạng 64-32-10, 50 epoch, mini-batch 32, 3 hạt giống: SGD (η = 0,1 và 0,5), momentum (η = 0,05), Adam (η = 0,001 và 0,01). So sánh độ chính xác kiểm định.

10 **Điều chuẩn hóa.** Lấy ngẫu nhiên 100 ảnh huấn luyện; mạng 64-64-10, 500 epoch, mini-batch 20, 5 hạt giống. So sánh: không điều chuẩn hóa, L2 (λ = 0,03), dropout (p = 0,3), dừng sớm. Chẩn đoán theo mục 5.4.

GỢI Ý ĐÁP ÁN · BÀI 1 ĐẾN 3

Tham số, XOR và vòng tròn, chẩn đoán

1 64-32-10: (64 + 1) × 32 + (32 + 1) × 10 = 2080 + 330 = **2410**. 64-64-10: 4160 + 650 = **4810**. Softmax: (64 + 1) × 10 = **650**. Hai mạng gấp khoảng 2,2 và 4,5 lần số mẫu (1074): cần điều chuẩn hóa; ở bài 10, 4810 tham số cho 100 mẫu là gấp khoảng 48 lần.

2 Perceptron, Logistic, Softmax đều có ranh giới thẳng nên **không** tách được {(0,0), (1,1)} khỏi {(0,1), (1,0)}; chỉ mạng có lớp ẩn giải được. Vòng tròn: ranh giới $x_1^2 + x_2^2 = r^2$ là phương trình **tuyến tính theo (x_1^2, x_2^2)** (thậm chí chỉ cần $x_1^2 + x_2^2$), nên Softmax trên đặc trưng bậc 2 biểu diễn được (86,3% ở mục 5.5), còn trên (x_1, x_2) gốc thì không.

3 Softmax: thiên lệch 12 − 3 = 9 điểm, khoảng cách kiểm định so với huấn luyện chỉ 1 điểm: **thiên lệch cao (thiếu khớp)**; thêm dữ liệu gần như không giúp, cần mô hình mạnh hơn (đa thức hoặc mạng). Mạng lớn: 0% so với 9%, thiên lệch gần 0, khoảng cách 9 điểm: **phương sai cao (quá khớp)**. Xử lý: L2, dropout hoặc dừng sớm; giảm kích thước mạng hoặc thêm dữ liệu.

GỢI Ý ĐÁP ÁN · BÀI 4 ĐẾN 6

CNN, cơ chế chú ý, RNN

4 Lớp tích chập: $(5 \times 5 + 1) \times 16 = \mathbf{416}$ tham số. Lớp đầy đủ: 784 đầu vào, $16 \times 24 \times 24 = 9216$ đầu ra, nên $(784 + 1) \times 9216 = \mathbf{7\,234\,560}$ tham số, gấp khoảng **17 000 lần** lớp tích chập vì lớp tích chập dùng chung 416 tham số ở mọi vị trí.

5 [2, 1, 0]: $e^2 = 7{,}389$; $e = 2{,}718$; $e^0 = 1$; tổng 11,107, nên softmax = **(0,665; 0,245; 0,090)**. [4, 2, 0]: 54,598; 7,389; 1; tổng 62,987, nên **(0,867; 0,117; 0,016)**. Nhân đôi điểm số làm phân phối **nhọn hơn** (86,7% thay vì 66,5%). Điểm số quá lớn (d**k** lớn) thì softmax gần one-hot, bão hòa, gradient → 0 (mục 4.3); hệ số $1/\sqrt{d\mathbf{k}}$ giữ độ lớn điểm số hợp lý.

6 $\partial h_t / \partial h_{t-1} = w(1 - h_t^2)$, có giá trị tuyệt đối $\leq |w| < 1$. Quy tắc chuỗi: $\partial \mathbf{h}T / \partial h_0 = \Pi\, w(1 - h_t^2)$, giá trị tuyệt đối $\leq \mathbf{|w|}^T \to \mathbf{0}$ khi $T \to \infty$. Tín hiệu từ bước đầu hầu như không ảnh hưởng đến h**T**: gradient theo các bước xa biến mất (cùng cơ chế mục 4.3).

GỢI Ý ĐÁP ÁN · BÀI 7 VÀ 8

Mốc cơ bản và ma trận nhầm lẫn

Bài 7. Softmax so với mạng 64-32-10 (5 hạt giống)

0,979 ± 0,004
Mạng, kiểm tra (huấn luyện 1,00; kiểm định 0,970 ± 0,008)

0,959
Softmax (η = 0,5, 1500 epoch), kiểm tra

≈ 2 SE
Chênh 2 điểm; SE ≈ 0,009 trên 364 mẫu

Đáng chú ý nhưng không áp đảo, khác hẳn khoảng cách hàng chục điểm ở mục 6.2: chữ số 8 × 8 khá gần tách được tuyến tính. Mỗi lần chạy mạng khoảng 0,5 giây.

Bài 8. Ma trận nhầm lẫn (hạt giống 0)

Mạng chỉ sai vài mẫu trên 364 ảnh. Nhầm nhiều nhất: chữ số **5 bị đoán là 3 (2 mẫu)**; các lỗi còn lại mỗi loại 1 mẫu (8 → 1, 6 → 4, 6 → 1, 5 → 9, 2 → 8). Quá ít để rút quy luật chắc chắn; nhiều ảnh 8 × 8 khó đọc kể cả với người: nhiễu nội tại (mục 5.4).

GỢI Ý ĐÁP ÁN · BÀI 9 VÀ 10

Bộ tối ưu và điều chuẩn hóa

Bài 9. Độ chính xác kiểm định sau 50 epoch (trung bình 3 hạt giống)

0,960 SGD, $\eta = 0{,}1$	**0,961** SGD, $\eta = 0{,}5$	**0,960** Momentum, $\eta = 0{,}05$	**0,964** Adam, $\eta = 0{,}001$	**0,961** Adam, $\eta = 0{,}01$

Mọi cấu hình nằm trong 96,0% đến 96,4%, nhỏ hơn sai số chuẩn trên 359 mẫu (≈ 0,010): **không thể** kết luận bộ tối ưu nào tốt hơn. Quan trọng hơn là chọn tốc độ học hợp lý (mục 3.5).

Bài 10. Mạng 64-64-10, 100 mẫu (huấn luyện / kiểm tra)

1,00 / 0,882 Không điều chuẩn	**1,00 / 0,891** L2, $\lambda = 0{,}03$	**1,00 / 0,880** Dropout, $p = 0{,}3$	**1,00 / 0,882** Dừng sớm

Cả bốn ghi nhớ 100% tập huấn luyện nhưng chỉ khoảng 88% kiểm tra: **phương sai cao** (khoảng cách ≈ 12 điểm). L2 hơn 0,9 điểm, nhỏ hơn SE (≈ 0,017): không đáng tin. Cách chắc chắn hơn là **thêm dữ liệu** (với 1074 mẫu, mạng 64-32-10 đạt 97,9%; so sánh thô).

KHÉP LẠI CHƯƠNG

Mục 6 nối lại toàn bộ chương

1. **Mục 1** hỏi: mô hình tuyến tính không giải được XOR, vòng tròn, xoắn ốc. **Mục 6.2 trả lời bằng số liệu:** Softmax chỉ đạt 0,49 đến 0,50, MLP đạt 0,847 đến 0,944.

2. **Mục 2 và 3** xây dựng mạng và thuật toán học. **Mục 6.3** cho thấy kiến trúc hiện đại là cùng ý tưởng “biến đổi tuyến tính rồi kích hoạt”, phần tuyến tính được cấu trúc lại; framework tự động hóa lan truyền ngược.

3. **Mục 4** giải thích khó khăn (không lồi, gradient biến mất, quá khớp); **mục 6.1** gom chúng thành phần hạn chế.

4. **Mục 5** cho công cụ đánh giá; **mục 6.4** dùng chính các công cụ đó trong bài tập.

TÓM TẮT MỤC 6 · PHẦN 1/2

Được gì, mất gì, bốn mô hình

1. Ưu điểm

Học ranh giới phi tuyến và đặc trưng; có định lý xấp xỉ phổ quát; một khung chung cho nhiều bài toán; huấn luyện được nhờ lan truyền ngược; mở rộng được theo kiến trúc.

2. Hạn chế

Không lồi nên kết quả phụ thuộc khởi tạo; gradient biến mất hoặc bùng nổ khi sâu; dễ quá khớp; nhiều siêu tham số; tốn tính toán; khó diễn giải; không phải lúc nào cũng thắng mô hình đơn giản.

3. Bốn mô hình là một chuỗi

Perceptron (hàm bước) → Logistic (sigmoid, mất mát trơn) → Softmax (nhiều lớp) → mạng nơ-ron (thêm lớp ẩn học đặc trưng). Dữ liệu phi tuyến: ba mô hình tuyến tính đạt 0,38 đến 0,55, MLP đạt 0,847 đến 0,944; dữ liệu tách được: cả bốn ngang nhau (98% đến 99%).

TÓM TẮT MỤC 6 · PHẦN 2/2

Học sâu và quy tắc thực hành

4. Học sâu

CNN dùng bộ lọc dùng chung cho cấu trúc lưới; RNN dùng trạng thái ẩn dùng chung theo thời gian cho chuỗi; Transformer dùng cơ chế chú ý với softmax. Tất cả vẫn huấn luyện bằng lan truyền ngược; PyTorch và các framework tự động hóa bước đó bằng vi phân tự động.

5. Quy tắc thực hành

Bắt đầu từ mô hình đơn giản làm mốc; dùng chẩn đoán thiên lệch và phương sai để quyết định có cần mô hình phức tạp hơn không; so sánh có tính đến sai số chuẩn.

Thông điệp cuối chương: hiểu lan truyền ngược từ đầu giúp bạn gỡ lỗi, hiểu framework và đọc các kiến trúc mới.

KIỂM TRA NHANH · CÂU 1 ĐẾN 6

Bạn đã nắm được chưa?

1. Trình bày mối quan hệ giữa Logistic Regression, Softmax Regression và mạng nơ-ron trong một câu.
2. Vì sao Perceptron không thể huấn luyện bằng gradient descent trên một hàm mất mát trơn?
3. Kể ba hạn chế của mạng nơ-ron và công cụ trong chương để xử lý từng hạn chế.
4. Một lớp tích chập với 8 bộ lọc 3×3 trên ảnh một kênh có bao nhiêu tham số?
5. Trong cơ chế chú ý, softmax áp dụng theo hàng hay theo cột của ma trận điểm tương đồng, và hàng đó có tính chất gì?
6. Vì sao biết lan truyền ngược "từ đầu" vẫn có ích khi đã có framework?

ĐÁP ÁN · CÂU 1 ĐẾN 6

Lời giải ngắn gọn

1. Softmax là mạng nơ-ron **không có lớp ẩn**; Logistic là Softmax với **hai lớp**; mạng thêm các lớp ẩn để tự học đặc trưng cho một Softmax Regression ở lớp cuối.

2. Hàm bước có đạo hàm bằng **0** ở hầu hết mọi nơi (không xác định tại điểm nhảy), nên gradient không cho thông tin về hướng cải thiện.

3. Ví dụ: **không lồi** → chạy nhiều hạt giống, báo cáo trung bình ± độ lệch chuẩn; **gradient biến mất** → ReLU, khởi tạo He, kiểm tra gradient; **quá khớp** → L2, dropout, dừng sớm, đường cong học.

4. $(3 \times 3 + 1) \times 8 = \mathbf{80}$ tham số.

5. Theo **hàng**: mỗi hàng là một phân phối xác suất (phần tử không âm, tổng bằng 1), cho biết một vị trí lấy thông tin từ các vị trí khác theo tỉ lệ nào.

6. Giúp **gỡ lỗi** (gradient biến mất, mất mát không giảm, quá khớp), **hiểu các tùy chọn** của framework và **đọc kiến trúc mới** như tổ hợp của các khối quen thuộc.